

Финансовый университет при Правительстве РФ

Государственная итоговая аттестация

01.03.02 «Прикладная математика и информатика»

профиль «Прикладное машинное обучение»

Подробный учебник для чайника

все 50 вопросов госэкзамена и 9 практических задач — с разбором и
решениями

Этот учебник создан, чтобы провести вас от полного непонимания до осмысленного ответа на государственном экзамене. Он разбирает каждый из 50 теоретических вопросов и каждую из 9 практических задач.

Как устроен разбор каждого вопроса. Сначала идёт блок «Простыми словами» — одна-две фразы с самой сутью, без формул. Затем основной разбор: ключевые понятия, формулы и определения, разбитые на смысловые части. Дальше — «Пример», показывающий идею на конкретных числах. Затем блок «Запомни главное» — самое важное плюс подсказка, о чём обычно спрашивает комиссия. Завершает разбор каждого вопроса **«Академический разбор»** — формальные определения, аксиомы, теоремы и точные формулы в той строгой записи, которой ждёт комиссия на экзамене.

Как готовиться. Первый проход — прочитайте все блоки «Простыми словами», чтобы увидеть общую картину. Второй проход — основной разбор, примеры и «Запомни главное». Третий — академические разборы, которые содержат точные формулировки для экзамена. Четвёртый — прорешайте практические задачи самостоятельно, подглядывая в решение только при затруднении. Формулы выписывайте от руки: так они запоминаются гораздо лучше.

Важная оговорка. Учебник даёт прочную основу и интуицию, но не заменяет официальные учебники из программы ГИА. Точные формулировки определений, полные доказательства теорем и строгие выкладки берите оттуда — комиссия ждёт именно их.

Оглавление

Часть 1. Дисциплины направления	5
1. Определённый интеграл Римана	5
2. Степенные ряды. Теорема Абеля. Ряд Тейлора	6
3. Алгебра матриц. Собственные значения и векторы	8
4. Евклидово пространство. Выпуклые множества. Гиперплоскости	10
5. Метрические пространства. Теорема Банаха о неподвижной точке	11
6. Логика высказываний. Логика предикатов. Логические законы	13
7. Булевы функции. Теорема о полноте (теорема Поста)	15
8. Вероятностные пространства. Формула полной вероятности и Байеса	16
9. Центральная предельная теорема. Закон больших чисел	18
10. Линейные дифференциальные уравнения первого порядка. Задача Коши	19
11. Показатели качества регрессионной модели	21
12. Метод наименьших квадратов (МНК)	23
13. Автокорреляция случайных возмущений	24
14. Антагонистические игры	26
15. Классификация игровых моделей. Чистые и смешанные стратегии	28
16. Функция полезности потребителя. CES-функция	30
17. Модель Леонтьева «затраты — выпуск»	31
18. Словари в Python	33
19. Алгоритмы сортировки. Бинарный поиск	35
20. Нормальные формы реляционных отношений	36
21. Структура SQL-запроса. Оператор SELECT	38
22. Запросы с несколькими таблицами. Соединения (JOIN)	39
23. Градиентные методы поиска экстремума	41
24. Целочисленное программирование. Метод Гомори. Метод ветвей и границ	43
25. Многокритериальная оптимизация. Парето-эффективная граница	44
Часть 2. Дисциплины профиля	47
1. Денежные потоки. Чистая приведённая стоимость. Ренты. IRR	47
2. Дюрация. Выпуклость облигаций. Иммунизация	48
3. Портфель ценных бумаг: доходность и риск	50
4. Биномиальная модель ценообразования. Модель Кокса-Росса-Рубинштейна	52
5. Модель Блэка-Шоулза	53
6. Модели индивидуального иска	55
7. Договоры страхования с вычетом, лимитом и франшизой	56
8. Индивидуальные модели риска. Вероятность разорения. Премии	58

9. Коллективные модели риска. Составное пуассоновское распределение	60
10. Эмпирическое распределение. Огива. Оценка Нельсона-Алена	61
11. Виды сетей: безмасштабные, тесного мира, фрактальные	63
12. Выявление сообществ в сетях. Модулярность	64
13. Типы задач машинного обучения	66
14. Классификация как задача машинного обучения	68
15. Метрики качества. Недо- и переобучение	70
16. Кластеризация. Метод ближайших соседей (kNN)	72
17. Обучение нейросетей. Обратное распространение ошибки	73
18. Свёрточные нейронные сети (CNN)	75
19. Рекуррентные нейронные сети. Модуль LSTM	77
20. Обработка текста. Модель Word2vec	79
21. Модели параллельного программирования	81
22. Map / Filter / Reduce. Dask.Bag	83
23. Формат XML и модель DOM. Библиотека BeautifulSoup	84
24. Организация массивов в NumPy	86
25. Pandas DataFrame. Операция GroupBy	88

Часть 3. Практические задачи 91

Задача 1. Портфель акций — двухкритериальная оптимизация	91
Задача 2. Заказ товара универмагом — игра с природой	92
Задача 3. Оценка инвестиционного проекта (NPV и IRR)	93
Задача 4. Страховой портфель — расчёт премии по принципу дисперсии	94
Задача 5. Оценка Каплана-Мейера	95
Задача 6. Динамическое программирование — распределение средств	96
Задача 7. Метрики качества классификации (3 класса)	97
Задача 8. Программа на pandas — сводка по двум таблицам	98
Задача 9. Дерево решений для разделения объектов на плоскости	99

Часть 1. Дисциплины направления

Общепрофессиональные и профессиональные дисциплины — математический фундамент.
25 вопросов.

В первую часть входят математический анализ, алгебра, теория вероятностей, эконометрика, теория игр, программирование, базы данных и методы оптимизации — то, на чём держится вся дальнейшая специализация.

1. Определённый интеграл Римана

ПРОСТЫМИ СЛОВАМИ

Определённый интеграл — это способ точно посчитать «накопленную сумму» бесконечно многих бесконечно малых кусочков. Самый наглядный образ — площадь фигуры под графиком функции.

Как строится интеграл

Возьмём функцию $f(x)$ на отрезке $[a, b]$. Разобьём отрезок на n маленьких частей. На каждой части выберем точку, посчитаем значение функции и умножим на ширину части — получим площадь узкого прямоугольника. Сложим все прямоугольники — это интегральная сумма Римана. Если устремить ширину частей к нулю и сумма стремится к одному и тому же числу, это число и есть определённый интеграл.

$$\int_{[a,b]} f(x) dx = \lim_{\Delta x \rightarrow 0} \sum f(\xi_i) \cdot \Delta x_i$$

Главные свойства

- Линейность: интеграл суммы равен сумме интегралов, константу можно выносить за знак интеграла.
- Аддитивность по отрезку: $\int_{[a,c]} = \int_{[a,b]} + \int_{[b,c]}$.
- Монотонность: если $f(x) \leq g(x)$, то и интеграл от f не больше интеграла от g .
- Перестановка пределов меняет знак: $\int_{[a,b]} = -\int_{[b,a]}$.

Формула Ньютона-Лейбница — мост между интегралом и производной

Чтобы не считать предел сумм вручную, используют первообразную $F(x)$ — функцию, производная которой равна $f(x)$. Тогда:

$$\int_{[a,b]} f(x) dx = F(b) - F(a)$$

Это и есть связь двух операций: дифференцирование измеряет мгновенную скорость, интегрирование собирает итог за весь промежуток.

Применение в экономике

Интегралом считают накопленную величину: суммарный доход при известной скорости поступления денег, объём выпуска при заданной производительности, потребительский излишек (площадь между кривой спроса и линией цены), излишек производителя.

ПРИМЕР

Пусть скорость поступления дохода равна $f(t) = 100 + 20t$ (тыс. руб. в год). Доход за первые 3 года: $\int_{[0,3]} (100 + 20t) dt = [100t + 10t^2]$ от 0 до 3 = $(300 + 90) - 0 = 390$ тыс. руб.

ЗАПОМНИ ГЛАВНОЕ

Запомни: определённый интеграл — это число (площадь), а неопределённый — семейство функций. Комиссия часто просит сформулировать свойства и формулу Ньютона-Лейбница, а также привести экономическую интерпретацию.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Определение. Определённым интегралом Римана от функции $f(x)$ на отрезке $[a, b]$ называется предел интегральных сумм при стремлении к нулю наибольшей длины частичного отрезка разбиения, не зависящий от способа разбиения и выбора промежуточных точек.

$$\int_a^b f(x) dx = \lim_{(\lambda \rightarrow 0)} \sum_{i=1}^n f(\xi_i) \cdot \Delta x_i,$$

где $\lambda = \max \Delta x_i$, $\xi_i \in [x_{i-1}, x_i]$

Функция, для которой такой предел существует, называется интегрируемой по Риману. Достаточные условия интегрируемости — непрерывность функции на отрезке или её ограниченность с конечным числом точек разрыва.

Свойства определённого интеграла

- линейность: постоянный множитель выносится, интеграл от суммы равен сумме интегралов;
- аддитивность по отрезку: $\int_a^c = \int_a^b + \int_b^c$ при $a \leq b \leq c$;
- перестановка пределов меняет знак: $\int_a^b = -\int_b^a$;
- если $a = b$, то $\int_a^a = 0$;
- монотонность: $f(x) \leq g(x) \Rightarrow \int_a^b f \leq \int_a^b g$;
- теорема о среднем: $\int_a^b f(x) dx = f(c) \cdot (b - a)$ при некотором $c \in [a, b]$ для непрерывной f .

Формула Ньютона-Лейбница

Если $F(x)$ — первообразная функции $f(x)$ на $[a, b]$, то

$$\int_a^b f(x) dx = F(b) - F(a)$$

Экономические приложения

- объём продукции за период $[0, t]$ при производительности труда $f(t)$ равен $\int_0^t f(\tau) d\tau$;
- по функции предельных издержек $MC(q)$ функция издержек восстанавливается как $C(q) = \int_{(q_0)}^q MC(s) ds + C(q_0)$;
- потребительский излишек — площадь между кривой спроса и линией равновесной цены;
- дисконтированная стоимость непрерывного потока платежей $f(t)$ за период $[0, T]$: $PV = \int_0^T f(t) \cdot e^{(-rt)} dt$.

2. Степенные ряды. Теорема Абеля. Ряд Тейлора

ПРОСТЫМИ СЛОВАМИ

Степенной ряд — это «бесконечный многочлен». Идея: сложную функцию (синус, экспоненту) рядом с точкой можно заменить суммой простых степеней x , с которой работать гораздо легче.

Что такое степенной ряд

Это бесконечная сумма слагаемых вида «коэффициент, умноженный на степень $(x - x_0)$ »:

$$\sum_{n=0.. \infty} a_n \cdot (x - x_0)^n$$

Область сходимости и теорема Абеля

Бесконечная сумма даёт конечное число не при любом x . Теорема Абеля гласит: если ряд сходится в некоторой точке, то он сходится и при всех x , которые ближе к центру x_0 . Отсюда вывод: у каждого ряда есть радиус сходимости R — внутри интервала $(x_0 - R, x_0 + R)$ ряд сходится, снаружи расходится, на границах нужна отдельная проверка.

$$R = \lim |a_n / a_{(n+1)}| \quad (\text{признак Даламбера})$$

Свойства суммы внутри интервала сходимости

Сумму ряда можно дифференцировать и интегрировать почленно — результат снова будет степенным рядом с тем же радиусом сходимости. То есть с рядом обращаются как с обычной гладкой функцией.

Ряд Тейлора

Ряд Тейлора — это рецепт, как разложить функцию в степенной ряд около точки x_0 . Коэффициенты берутся из производных функции:

$$f(x) = \sum f^{(n)}(x_0)/n! \cdot (x - x_0)^n$$

Если $x_0 = 0$, ряд называют рядом Маклорена. Базовые разложения, которые стоит знать наизусть:

$$\begin{aligned} e^x &= 1 + x + x^2/2! + x^3/3! + \dots \\ \sin x &= x - x^3/3! + x^5/5! - \dots \\ 1/(1-x) &= 1 + x + x^2 + x^3 + \dots \quad (\text{при } |x| < 1) \end{aligned}$$

ПРИМЕР

Приближим $e^{(0,1)}$. Берём три слагаемых ряда: $1 + 0,1 + 0,1^2/2 = 1 + 0,1 + 0,005 = 1,105$. Точное значение $\approx 1,10517$ — погрешность меньше 0,0002. Именно так калькулятор и считает экспоненту.

ЗАПОМНИ ГЛАВНОЕ

Запомни: радиус сходимости отвечает на вопрос «где ряд работает», а ряд Тейлора — «как функцию превратить в многочлен». На экзамене просят сформулировать теорему Абеля и записать разложения e^x , $\sin x$, $\cos x$, $\ln(1+x)$.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Определение. Степенным рядом с базисной точкой z_0 называют функциональный ряд вида

$$\sum_{n=0}^{\infty} a_n \cdot (z - z_0)^n,$$

где (a_n) — последовательность констант, z — переменная. При $z_0 = 0$ ряд имеет вид $\sum a_n \cdot z^n$.

Теорема Абеля

Если степенной ряд сходится в точке $x_0 \neq 0$, то он сходится абсолютно во всех точках интервала $(-|x_0|, |x_0|)$. Следствие: для каждого ряда существует число $R \geq 0$ — радиус сходимости, — такое что ряд сходится при $|x - x_0| < R$ и расходится при $|x - x_0| > R$.

$$R = 1 / \lim_{n \rightarrow \infty} \sqrt[n]{|a_n|} \quad (\text{формула Коши-Адамара})$$
$$R = \lim_{n \rightarrow \infty} |a_n / a_{n+1}| \quad (\text{по признаку Даламбера})$$

При $R = 0$ область сходимости является только точка $x = x_0$; при $R = +\infty$ — вся числовая прямая. На границах интервала $(x_0 - R, x_0 + R)$ поведение исследуется отдельно.

Интегрируемость и дифференцируемость суммы ряда

Внутри интервала сходимости степенной ряд можно дифференцировать и интегрировать почленно — результаты являются степенными рядами с тем же радиусом сходимости.

Ряд Тейлора и Маклорена

Если функция $f(x)$ бесконечно дифференцируема в окрестности точки x_0 , то её разложение в ряд Тейлора имеет вид

$$f(x) = \sum_{n=0}^{\infty} f^{(n)}(x_0)/n! \cdot (x - x_0)^n$$

При $x_0 = 0$ ряд называется рядом Маклорена. Базовые табличные разложения:

e^x	$= 1 + x + x^2/2! + x^3/3! + \dots$	$x \in \mathbb{R}$
$\sin x$	$= x - x^3/3! + x^5/5! - \dots$	$x \in \mathbb{R}$
$\cos x$	$= 1 - x^2/2! + x^4/4! - \dots$	$x \in \mathbb{R}$
$\ln(1+x)$	$= x - x^2/2 + x^3/3 - \dots$	$-1 < x \leq 1$
$1/(1-x)$	$= 1 + x + x^2 + x^3 + \dots$	$ x < 1$

Способы разложения функций

- непосредственное вычисление производных $f^{(n)}(x_0)$ и подстановка в формулу;
- использование табличных разложений с подстановкой;
- сложение, вычитание и умножение известных рядов;
- почленное дифференцирование или интегрирование известного ряда.

3. Алгебра матриц. Собственные значения и векторы

ПРОСТЫМИ СЛОВАМИ

Матрица — это таблица чисел, но удобнее думать о ней как о «машинке», которая преобразует пространство: растягивает, сжимает, поворачивает.

Операции с матрицами

Матрицы складывают поэлементно. Умножение сложнее: элемент результата — это сумма произведений строки первой матрицы на столбец второй.

$$(A \cdot B)_{ij} = \sum_k a_{ik} \cdot b_{kj}$$

Важно: умножение матриц некоммукативно ($A \cdot B$ обычно не равно $B \cdot A$). Геометрически умножение — это последовательное применение двух преобразований.

Линейное преобразование

Каждая матрица задаёт линейное преобразование: берёт вектор x и превращает его в вектор $A \cdot x$. «Линейное» означает, что прямые переходят в прямые, а начало координат остаётся на месте.

Собственные значения и собственные векторы

Собственный вектор v — это особое направление, которое преобразование НЕ поворачивает, а лишь растягивает или сжимает. Число λ (собственное значение) показывает, во сколько раз.

$$A \cdot v = \lambda \cdot v, \quad v \neq 0$$

Чтобы найти λ , решают характеристическое уравнение — приравнивают к нулю определитель:

$$\det(A - \lambda \cdot I) = 0$$

Корни этого уравнения — собственные значения. Подставив каждое λ обратно, находят соответствующие собственные векторы.

Зачем это нужно

- Метод главных компонент (PCA) — снижение размерности данных по собственным векторам ковариационной матрицы.
- Алгоритм PageRank ранжирует веб-страницы через собственный вектор.
- Устойчивость динамических систем определяется знаком собственных значений.

ПРИМЕР

Матрица $A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$. Характеристическое уравнение: $(2-\lambda)(3-\lambda) = 0$, корни $\lambda_1 = 2$, $\lambda_2 = 3$. Собственные векторы — оси координат: вектор $(1,0)$ растягивается в 2 раза, вектор $(0,1)$ — в 3 раза.

ЗАПОМНИ ГЛАВНОЕ

Запомни: собственные векторы — «естественные оси» преобразования. Просят записать характеристическое уравнение и объяснить смысл λ .

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Линейные операции над матрицами: сложение матриц одинакового размера и умножение матрицы на число. Умножение матриц: $(A \cdot B)_{ij} = \sum_k a_{ik} \cdot b_{kj}$, определено при согласованности размерностей; в общем случае некоммутативно.

Линейное преобразование. Отображение $\varphi: V \rightarrow W$ линейных пространств называется линейным, если $\varphi(\alpha x + \beta y) = \alpha \varphi(x) + \beta \varphi(y)$. В фиксированных базисах оно задаётся матрицей: $\varphi(x) = A \cdot x$.

Собственные значения и собственные векторы

Определение. Ненулевой вектор v называется собственным вектором квадратной матрицы A , если

$$A \cdot v = \lambda \cdot v, \quad v \neq 0,$$

число λ — соответствующее собственное значение (собственное число) матрицы.

Алгоритм нахождения

- составить характеристическую матрицу $A - \lambda E$ и характеристическое уравнение $\det(A - \lambda E) = 0$;
- решить полученное алгебраическое уравнение относительно λ — корни и есть собственные значения;
- для каждого λ найти фундаментальную систему решений однородной системы $(A - \lambda E) \cdot x = 0$; ненулевые решения — собственные векторы.

Свойства

- сумма собственных значений равна следу матрицы (сумме диагональных элементов);
- произведение собственных значений равно определителю матрицы;
- собственные векторы, отвечающие различным собственным значениям, линейно независимы;
- симметричная матрица имеет вещественные собственные значения и ортогональный базис из собственных векторов.

4. Евклидово пространство. Выпуклые множества. Гиперплоскости

ПРОСТЫМИ СЛОВАМИ

Это геометрический язык, на котором описывают и оптимизацию, и классификацию в машинном обучении.

Евклидово пространство

Это привычное пространство, обобщённое на любое число измерений. В нём определено скалярное произведение, через которое задаются длина вектора (норма) и угол между векторами.

$$\langle x, y \rangle = x_1 \cdot y_1 + x_2 \cdot y_2 + \dots + x_n \cdot y_n$$
$$\|x\| = \sqrt{\langle x, x \rangle}$$

Выпуклые множества

Множество выпукло, если для любых двух его точек весь отрезок между ними тоже лежит внутри множества — «фигура без вмятин».

$$\text{Для любых } x, y \text{ из } M \text{ и } \lambda \in [0, 1]: \lambda \cdot x + (1 - \lambda) \cdot y \in M$$

Выпуклость критически важна в оптимизации: у выпуклой функции на выпуклом множестве локальный минимум автоматически является глобальным — не застрянешь в «ложной яме».

Гиперплоскость

Гиперплоскость — это «плоский разрез» пространства: в 2D это прямая, в 3D — плоскость, в n -мерном пространстве — $(n-1)$ -мерная плоскость. Задаётся уравнением:

$$\langle w, x \rangle = b$$

Гиперплоскость делит пространство на два полупространства. Разделяющая гиперплоскость помещает одну группу точек по одну сторону, другую — по другую. Отделяющая гиперплоскость касается выпуклого множества, не заходя внутрь.

Связь с машинным обучением

Метод опорных векторов (SVM) ищет именно разделяющую гиперплоскость — такую, чтобы зазор (расстояние до ближайших точек обоих классов) был максимальным.

ПРИМЕР

На плоскости прямая $x_1 + x_2 = 1$ — это гиперплоскость. Точка $(0,0)$ даёт $0 < 1$ — она в одном полупространстве; точка $(1,1)$ даёт $2 > 1$ — в другом. Так прямая разделяет два класса точек.

ЗАПОМНИ ГЛАВНОЕ

Запомни: выпуклость гарантирует, что локальный минимум = глобальный. Просят дать определение выпуклого множества и записать уравнение гиперплоскости.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Определение. Вещественное линейное пространство называется евклидовым, если каждой паре его элементов x, y поставлено в соответствие вещественное число (x, y) — скалярное произведение — удовлетворяющее аксиомам:

- $(x, x) \geq 0$; $(x, x) = 0 \Leftrightarrow x = 0$ (положительная определённость);
- $(x, y) = (y, x)$ (симметричность);
- $(\lambda x, y) = \lambda \cdot (x, y)$ (однородность);
- $(x + y, z) = (x, z) + (y, z)$ (аддитивность).

Через скалярное произведение определяются норма $\|x\| = \sqrt{(x, x)}$ и расстояние $\rho(x, y) = \|x - y\|$. Справедливо неравенство Коши-Буняковского: $|(x, y)| \leq \|x\| \cdot \|y\|$.

Выпуклые множества

Определение. Множество $M \subseteq \mathbb{R}^n$ называется выпуклым, если вместе с любыми двумя своими точками оно содержит и весь соединяющий их отрезок:

$$x, y \in M, \lambda \in [0, 1] \Rightarrow \lambda x + (1 - \lambda)y \in M.$$

Пересечение любого семейства выпуклых множеств выпукло. Выпуклой комбинацией называется $\lambda_1 x_1 + \dots + \lambda_k x_k$ с $\lambda_i \geq 0$ и $\sum \lambda_i = 1$.

Гиперплоскость

Гиперплоскость в $\mathbb{R}^n$ — множество вида $H = \{x : (w, x) = b\}$, где $w \neq 0, b \in \mathbb{R}$. Она разбивает пространство на два замкнутых полупространства.

Разделяющая и опорная гиперплоскости

- Гиперплоскость H разделяет множества A и B , если они лежат в разных полупространствах: $(w, x) \leq b$ для $x \in A$ и $(w, x) \geq b$ для $x \in B$.
- Если A и B — непустые непересекающиеся выпуклые множества, то разделяющая гиперплоскость существует (первая теорема отделимости).
- Опорной (отделяющей в узком смысле) гиперплоскостью для выпуклого множества M в точке $x_0 \in \partial M$ называется такая, что M целиком лежит в одном полупространстве и $x_0 \in H$.

5. Метрические пространства. Теорема Банаха о неподвижной точке

ПРОСТЫМИ СЛОВАМИ

Это теоретическая гарантия того, что итерационные методы (повторяем вычисление снова и снова) действительно сходятся к ответу.

Метрическое пространство

Это любое множество объектов, для которых задано «расстояние» — функция $d(x, y)$, удовлетворяющая трём аксиомам:

- $d(x, y) \geq 0$, причём равно 0 только если $x = y$;
- симметрия: $d(x, y) = d(y, x)$;
- неравенство треугольника: $d(x, z) \leq d(x, y) + d(y, z)$.

Объектами могут быть точки, функции, последовательности, тексты — что угодно, лишь бы расстояние вело себя разумно.

Сжимающее отображение

Отображение f называется сжимающим, если оно сближает любые две точки — расстояние между образами строго меньше, чем между прообразами, с коэффициентом $q < 1$:

$$d(f(x), f(y)) \leq q \cdot d(x, y), \quad 0 \leq q < 1$$

Теорема Банаха

В полном метрическом пространстве у сжимающего отображения существует ровно одна неподвижная точка x^* (такая, что $f(x^*) = x^*$). Более того, к ней сходится последовательность итераций из любой стартовой точки:

$$x_{(n+1)} = f(x_n) \rightarrow x^* \text{ при } n \rightarrow \infty$$

Скорость сходимости геометрическая — ошибка убывает как q в степени n .

Зачем это нужно

Теорема обосновывает метод простых итераций для решения уравнений, численные методы, доказательство существования решения дифференциальных уравнений (теорема Пикара). Если процесс «сжимает», повторение гарантированно приводит к единственному ответу.

ПРИМЕР

Уравнение $x = \cos(x)$. Функция $\cos$ на отрезке $[0, 1]$ сжимает (её производная по модулю меньше 1). Запускаем итерации: $x_0 = 0,5 \rightarrow \cos(0,5) = 0,8776 \rightarrow \cos(0,8776) = 0,6390 \rightarrow \dots \rightarrow$ сходится к $x^* \approx 0,739$.

ЗАПОМНИ ГЛАВНОЕ

Запомни: сжатие + полнота пространства = единственная неподвижная точка. Просят сформулировать аксиомы метрики и саму теорему.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Определение. Метрическим пространством называется пара (X, ρ) , где X — непустое множество, а $\rho: X \times X \rightarrow \mathbb{R}$ — функция (метрика), удовлетворяющая аксиомам:

- $\rho(x, y) \geq 0$; $\rho(x, y) = 0 \Leftrightarrow x = y$ (неотрицательность);
- $\rho(x, y) = \rho(y, x)$ (симметричность);
- $\rho(x, z) \leq \rho(x, y) + \rho(y, z)$ (неравенство треугольника).

Пространство называется полным, если в нём всякая фундаментальная последовательность сходится (имеет предел в X).

Сжимающее отображение

Отображение $f: X \rightarrow X$ называется сжимающим, если существует число $q \in [0, 1)$, такое что для любых $x, y \in X$:

$$\rho(f(x), f(y)) \leq q \cdot \rho(x, y).$$

Число q называют коэффициентом сжатия.

Теорема Банаха о неподвижной точке

Формулировка. Пусть (X, ρ) — полное метрическое пространство и $f: X \rightarrow X$ — сжимающее отображение с коэффициентом $q < 1$. Тогда:

- в X существует единственная неподвижная точка x^* такая, что $f(x^*) = x^*$;
- при любой начальной точке $x_0 \in X$ итерации $x_{n+1} = f(x_n)$ сходятся к x^* ;
- имеет место оценка скорости сходимости: $\rho(x_n, x^*) \leq q^n / (1 - q) \cdot \rho(x_1, x_0)$.

Теорема используется для обоснования метода простых итераций, теоремы существования и единственности решения задачи Коши (теорема Пикара-Линделёфа), а также численных методов решения уравнений и систем.

6. Логика высказываний. Логика предикатов. Логические законы

ПРОСТЫМИ СЛОВАМИ

Логика — это грамматика правильных рассуждений, записанная символами. Основа условий в коде, работы процессора и баз данных.

Логика высказываний

Высказывание — утверждение, которое либо истинно (1), либо ложно (0). Высказывания соединяют логическими связками:

- отрицание $\neg A$ («не A »);
- конъюнкция $A \wedge B$ (« A и B ») — истинна, когда оба истинны;
- дизъюнкция $A \vee B$ (« A или B ») — истинна, когда хотя бы один;
- импликация $A \rightarrow B$ («из A следует B ») — ложна только когда A истинно, а B ложно;
- эквивалентность $A \leftrightarrow B$ — истинна, когда значения совпадают.

Логика предикатов

Логика высказываний не умеет говорить «для всех» и «существует». Логика предикатов это исправляет: предикат — это утверждение с переменной (например, $P(x) = \text{«}x \text{ — чётное»}$), а кванторы связывают переменную:

- квантор всеобщности $\forall x P(x)$ — «для всех x верно $P(x)$ »;
- квантор существования $\exists x P(x)$ — «существует x , для которого $P(x)$ ».

Логические законы

Закон — это формула, истинная при любых значениях переменных (тавтология).
Важнейшие:

Законы де Моргана: $\neg(A \wedge B) = \neg A \vee \neg B$, $\neg(A \vee B) = \neg A \wedge \neg B$

Закон исключённого третьего: $A \vee \neg A = 1$

Закон противоречия: $A \wedge \neg A = 0$

Дистрибутивность: $A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$

Отрицание квантора переворачивает его: $\neg \forall x P(x) = \exists x \neg P(x)$.

ПРИМЕР

Высказывание «неверно, что все студенты сдали экзамен» по закону отрицания квантора равносильно «существует студент, который не сдал». Это типичное преобразование, без которого не построить правильное возражение.

ЗАПОМНИ ГЛАВНОЕ

Запомни: законы де Моргана и отрицание кванторов. Просят построить таблицу истинности и доказать тождество.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Логика высказываний

Высказывание — повествовательное предложение, относительно которого можно объективно сказать, истинно оно (1) или ложно (0). Над высказываниями определены логические операции:

- отрицание $\neg A$ («не A»);
- конъюнкция $A \wedge B$ («A и B»): истинна только при $A = B = 1$;
- дизъюнкция $A \vee B$ («A или B»): ложна только при $A = B = 0$;
- импликация $A \rightarrow B$: ложна только когда $A = 1$, $B = 0$;
- эквивалентность $A \leftrightarrow B$: истинна, когда значения совпадают.

Формула, истинная при любых значениях входящих в неё высказываний, называется тождественно истинной (тавтологией); тождественно ложная — противоречием.

Логика предикатов

Предикат $P(x_1, \dots, x_n)$ — высказывательная функция от переменных, принимающих значения из некоторой предметной области. Над предикатами определены кванторы:

- квантор всеобщности $\forall x P(x)$ — «для всех x верно $P(x)$ »;
- квантор существования $\exists x P(x)$ — «существует такой x , для которого $P(x)$ ».

Логика предикатов расширяет возможности логики высказываний, позволяя формализовать рассуждения о бесконечных предметных областях и общих свойствах объектов.

Основные логические законы

коммутативность: $A \wedge B = B \wedge A$, $A \vee B = B \vee A$
ассоциативность: $(A \wedge B) \wedge C = A \wedge (B \wedge C)$, аналогично для $\vee$
дистрибутивность: $A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$
законы де Моргана: $\neg(A \wedge B) = \neg A \vee \neg B$, $\neg(A \vee B) = \neg A \wedge \neg B$
идемпотентность: $A \wedge A = A$, $A \vee A = A$
поглощение: $A \wedge (A \vee B) = A$, $A \vee (A \wedge B) = A$
исключённое третье: $A \vee \neg A = 1$
противоречие: $A \wedge \neg A = 0$
двойное отрицание: $\neg\neg A = A$
отрицание кванторов: $\neg\forall x P(x) \equiv \exists x \neg P(x)$

7. Булевы функции. Теорема о полноте (теорема Поста)

ПРОСТЫМИ СЛОВАМИ

Теорема Поста — это критерий, позволяет ли данный набор операций («конструктор») собрать вообще любую логическую схему.

Булева функция

Булева функция — функция, у которой и аргументы, и значение принимают только значения 0 и 1. От n переменных существует ровно 2 в степени n (2^n) различных функций. Любую можно задать таблицей истинности.

Важнейшие классы булевых функций

Теорема Поста опирается на пять специальных классов:

- T_0 — функции, сохраняющие ноль (на нулевом наборе дают 0);
- T_1 — функции, сохраняющие единицу;
- S — самодвойственные функции;
- M — монотонные (рост аргументов не уменьшает значение);
- L — линейные (выражаются через сложение по модулю 2).

Полнота системы и теорема Поста

Система функций называется полной, если из неё суперпозицией (подстановкой одних функций в другие) можно получить любую булеву функцию. Теорема Поста даёт точный критерий:

Система полна тогда и только тогда, когда она не содержится целиком ни в одном из пяти классов T_0 , T_1 , S , M , L . То есть для каждого класса в системе должна найтись функция, не входящая в этот класс.

Следствия

Система $\{\text{И}, \text{ИЛИ}, \text{НЕ}\}$ полна. Более того, полна даже одна функция «И-НЕ» (штрих Шеффера) или одна «ИЛИ-НЕ» (стрелка Пирса) — поэтому всю цифровую электронику можно построить на однотипных вентилях.

ПРИМЕР

Проверим систему {конъюнкция, отрицание}. Конъюнкция не самодвойственна, не линейна; отрицание не монотонно, не сохраняет 0 и 1. Значит, для каждого из пяти классов есть «нарушитель» — система полна.

ЗАПОМНИ ГЛАВНОЕ

Запомни: пять классов Поста и формулировку критерия полноты. Просят проверить полноту конкретной системы по таблице принадлежности классам.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Определение. Булева функция от n переменных — функция $f: \{0, 1\}^n \rightarrow \{0, 1\}$. Всего таких функций существует 2^{2^n} . Каждая может быть задана таблицей истинности или формулой (через операции $\wedge, \vee, \neg, \oplus$ и др.).

Важнейшие классы булевых функций (классы Поста)

- T_0 — класс функций, сохраняющих нуль: $f(0, \dots, 0) = 0$;
- T_1 — класс функций, сохраняющих единицу: $f(1, \dots, 1) = 1$;
- S — класс самодвойственных функций: $f(\neg x_1, \dots, \neg x_n) = \neg f(x_1, \dots, x_n)$;
- M — класс монотонных функций: рост каждого аргумента не уменьшает значения функции;
- L — класс линейных функций, представимых в виде $f = a_0 \oplus a_1 x_1 \oplus \dots \oplus a_n x_n$ (полином Жегалкина первой степени).

Каждый из пяти классов замкнут относительно суперпозиции — подстановки функций друг в друга.

Теорема Поста о полноте

Система булевых функций $\{f_1, \dots, f_m\}$ называется функционально полной, если суперпозициями функций из неё можно выразить любую булеву функцию.

Критерий Поста. Система функций функционально полна тогда и только тогда, когда она не содержится целиком ни в одном из пяти классов T_0, T_1, S, M, L . Иными словами, для каждого из пяти классов в системе должна найтись функция, не принадлежащая этому классу.

Примеры полных систем

- $\{\wedge, \vee, \neg\}$ — классическая система;
- $\{\wedge, \neg\}$ и $\{\vee, \neg\}$;
- одиночная функция «штрих Шеффера» (И-НЕ);
- одиночная функция «стрелка Пирса» (ИЛИ-НЕ);
- $\{\wedge, \oplus, 1\}$ — система для полинома Жегалкина.

8. Вероятностные пространства. Формула полной вероятности и Байеса

ПРОСТЫМИ СЛОВАМИ

Формула Байеса — это математика обучения на опыте: как пересчитать уверенность в гипотезе после получения новых данных.

Вероятностное пространство

Это тройка: пространство элементарных исходов Ω (всё, что может случиться), набор событий (подмножества Ω) и вероятность P , которая каждому событию ставит число от 0 до 1 по аксиомам Колмогорова: $P(\Omega) = 1$, вероятность неотрицательна, вероятность объединения несовместных событий равна сумме вероятностей.

Независимость событий

События A и B независимы, если наступление одного не меняет вероятность другого:

$$P(A \cap B) = P(A) \cdot P(B)$$

Формула полной вероятности

Пусть гипотезы $H_1, \dots, H_n$ разбивают всё пространство на непересекающиеся сценарии. Тогда вероятность события A:

$$P(A) = \sum_i P(A | H_i) \cdot P(H_i)$$

Формула Байеса

Байес позволяет «развернуть» условную вероятность — найти вероятность гипотезы ПОСЛЕ того, как событие A уже наступило:

$$P(H_i | A) = P(A | H_i) \cdot P(H_i) / P(A)$$

$P(H_i)$ называют априорной вероятностью (до данных), $P(H_i | A)$ — апостериорной (после данных). На этой формуле стоит наивный байесовский классификатор и всё байесовское машинное обучение.

ПРИМЕР

Болезнь есть у 1% людей. Тест верен в 99% случаев. Тест дал «+». Вероятность реально быть больным: $P = (0,99 \cdot 0,01) / (0,99 \cdot 0,01 + 0,01 \cdot 0,99) = 0,0099 / 0,0198 = 0,5$. Всего 50% — потому что болезнь редкая. Это классический контринтуитивный результат Байеса.

ЗАПОМНИ ГЛАВНОЕ

Запомни: априорная и апостериорная вероятности. Попросят вывести формулу Байеса из определения условной вероятности и решить задачу про тест.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Аксиоматика Колмогорова

Вероятностное пространство — тройка (Ω, F, P) , где Ω — пространство элементарных исходов, F — σ -алгебра событий (подмножеств Ω), P — вероятностная мера, удовлетворяющая аксиомам:

- неотрицательность: $P(A) \geq 0$ для любого $A \in F$;
- нормированность: $P(\Omega) = 1$;
- счётная аддитивность: для несовместных A_i ($A_i \cap A_j = \emptyset$ при $i \neq j$) $P(\bigcup A_i) = \sum P(A_i)$.

Условная вероятность и независимость

Условная вероятность события A при условии B (при $P(B) > 0$):

$$P(A | B) = P(A \cap B) / P(B)$$

События A и B называются независимыми, если $P(A \cap B) = P(A) \cdot P(B)$; это равносильно $P(A | B) = P(A)$.

Формула полной вероятности

Пусть события $H_1, H_2, \dots, H_n$ образуют полную группу: они попарно несовместны и $\sum P(H_i) = 1$. Тогда для любого события A :

$$P(A) = \sum_{i=1}^n P(A | H_i) \cdot P(H_i)$$

Формула Байеса

При тех же условиях после наблюдения A апостериорные вероятности гипотез вычисляются по формуле:

$$P(H_k | A) = P(A | H_k) \cdot P(H_k) / \sum_i P(A | H_i) \cdot P(H_i)$$

$P(H_k)$ — априорная вероятность гипотезы (до наблюдения), $P(H_k | A)$ — апостериорная (после). Формула Байеса лежит в основе наивного байесовского классификатора и всего байесовского статистического вывода.

9. Центральная предельная теорема. Закон больших чисел

ПРОСТЫМИ СЛОВАМИ

Это два закона, объясняющих, почему статистика вообще работает: среднее стабилизируется, и делает это по форме «колокола».

Закон больших чисел (ЗБЧ)

Если повторять случайный опыт много раз независимо, то среднее арифметическое результатов сходится к истинному математическому ожиданию:

$$\bar{X}_n = (X_1 + \dots + X_n)/n \rightarrow \mu \text{ при } n \rightarrow \infty$$

Поэтому при большом числе бросков монеты доля «орлов» близка к $1/2$, а среднее по большой выборке близко к истинному среднему. ЗБЧ обосновывает саму идею оценивать параметры по выборке.

Центральная предельная теорема (ЦПТ)

ЦПТ говорит больше: сумма (или среднее) большого числа независимых одинаково распределённых случайных величин распределена приблизительно по нормальному закону — почти независимо от того, как были устроены сами слагаемые.

$$(X_1 + \dots + X_n - n \cdot \mu) / (\sigma \cdot \sqrt{n}) \rightarrow N(0, 1)$$

Именно поэтому нормальное распределение («колокол Гаусса») встречается повсюду: рост людей, ошибки измерений, шум — это суммы множества мелких случайных вкладов.

Применение

- доверительные интервалы для среднего;
- проверка статистических гипотез;
- нормальная аппроксимация биномиального распределения;
- обоснование того, почему ошибки моделей часто нормальны.

ПРИМЕР

Игральная кость даёт числа 1-6 (распределение равномерное, совсем не колокол). Но если бросить 100 костей и взять среднее, и так повторить много раз — гистограмма этих средних будет почти идеальным колоколом со средним 3,5. Это и есть ЦПТ в действии.

ЗАПОМНИ ГЛАВНОЕ

Запомни: ЗБЧ — про сходимости среднего к μ , ЦПТ — про нормальность суммы. Просят сформулировать обе теоремы и объяснить, где применяется нормальная аппроксимация.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Закон больших чисел

Теорема Чебышёва (слабая форма ЗБЧ). Пусть $X_1, X_2, \dots$ — последовательность независимых случайных величин с конечными дисперсиями, ограниченными общей константой. Тогда для любого $\varepsilon > 0$:

$$P(|\bar{X}_n - \mu| < \varepsilon) \rightarrow 1 \quad \text{при } n \rightarrow \infty,$$

где $\bar{X}_n = (X_1 + \dots + X_n)/n$, μ — общее мат. ожидание.

Содержательно: при большом числе независимых испытаний выборочное среднее становится сколь угодно близким к истинному математическому ожиданию с вероятностью, стремящейся к единице. ЗБЧ обосновывает само понятие статистической оценки.

Центральная предельная теорема

Классическая ЦПТ (Линдберга-Леви). Пусть $X_1, X_2, \dots$ — независимые одинаково распределённые случайные величины с конечным математическим ожиданием μ и конечной положительной дисперсией σ^2 . Тогда нормированная сумма сходится по распределению к стандартной нормальной величине:

$$(X_1 + \dots + X_n - n\mu) / (\sigma\sqrt{n}) \rightarrow N(0, 1) \quad \text{при } n \rightarrow \infty.$$

Содержательно: распределение суммы (или среднего) большого числа независимых одинаково распределённых случайных величин близко к нормальному, независимо от исходного распределения слагаемых.

Применение

- обоснование доверительных интервалов для среднего и проверки статистических гипотез;
- нормальная аппроксимация биномиального распределения $\text{Bin}(n, p)$ при больших n ;
- оценка вероятности отклонений суммарных страховых выплат от ожидаемого значения (расчёт рисковой надбавки);
- обоснование нормальности шумов в эконометрических моделях.

10. Линейные дифференциальные уравнения первого порядка. Задача Коши

ПРОСТЫМИ СЛОВАМИ

Дифференциальное уравнение описывает закон движения процесса, а начальное условие — точку старта; вместе они дают единственную траекторию.

Что это за уравнение

Дифференциальное уравнение связывает функцию с её производной. Линейное уравнение первого порядка имеет стандартный вид — производная и сама функция входят «честно», без степеней и произведений:

$$y' + p(x) \cdot y = q(x)$$

Если $q(x) = 0$, уравнение называют однородным, иначе — неоднородным.

Как решать

Стандартный приём — метод интегрирующего множителя. Умножают уравнение на специальную функцию $\mu(x) = e^{\int p(x)dx}$, после чего левая часть сворачивается в производную произведения, и остаётся проинтегрировать. Альтернатива — метод Бернулли (замена $y = u \cdot v$) или метод вариации произвольной постоянной.

Общее решение содержит произвольную постоянную C — это целое семейство кривых.

Задача Коши

Задача Коши добавляет начальное условие $y(x_0) = y_0$. Оно определяет постоянную C и выбирает из всего семейства одну конкретную кривую.

Существование и единственность

Теорема (Пикара-Линделёфа): если функции $p(x)$ и $q(x)$ непрерывны в окрестности точки x_0 , то задача Коши имеет решение, и оно единственно. Доказательство опирается на теорему Банаха о сжимающем отображении (см. вопрос 5).

ПРИМЕР

Рост вклада: $y' = 0,1 \cdot y$, $y(0) = 1000$. Это уравнение с разделяющимися переменными, решение $y = 1000 \cdot e^{(0,1x)}$. Начальное условие зафиксировало множитель 1000 — получили единственную кривую непрерывного роста под 10%.

ЗАПОМНИ ГЛАВНОЕ

Запомни: общее решение — семейство, задача Коши — одна кривая. Просят записать стандартный вид, назвать метод решения и условия теоремы единственности.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Определение. Дифференциальное уравнение первого порядка называется линейным, если оно имеет вид

$$y' + p(x) \cdot y = q(x),$$

где $p(x)$, $q(x)$ — заданные функции. При $q(x) \equiv 0$ уравнение называется однородным, иначе — неоднородным.

Методы решения

Метод интегрирующего множителя. Умножая обе части на $\mu(x) = e^{\int p(x)dx}$, приводим левую часть к производной произведения:

$$\begin{aligned}(\mu(x) \cdot y)' &= \mu(x) \cdot q(x) \\ \Rightarrow y(x) &= (1/\mu(x)) \cdot \left(\int \mu(x) \cdot q(x) dx + C \right)\end{aligned}$$

Метод Бернулли ищет решение в виде произведения двух функций $y = u(x) \cdot v(x)$; подстановка приводит к двум более простым уравнениям.

Метод вариации произвольной постоянной (Лагранжа): сначала решают однородное уравнение, затем константу C заменяют на функцию $C(x)$ и подставляют в исходное.

Задача Коши

Задача Коши состоит в нахождении частного решения уравнения, удовлетворяющего начальному условию $y(x_0) = y_0$.

Теорема существования и единственности (Пикара-Линделёфа)

Если функция $f(x, y)$ непрерывна в некоторой окрестности точки (x_0, y_0) и удовлетворяет условию Липшица по y , то задача Коши $y' = f(x, y)$, $y(x_0) = y_0$ имеет в этой окрестности единственное решение. Для линейного уравнения $y' + p(x)y = q(x)$ достаточным является непрерывность $p(x)$ и $q(x)$ в окрестности x_0 ; в этом случае решение существует и единственно.

Доказательство теоремы основано на принципе сжимающих отображений (теорема Банаха): оператор Пикара $Ty(x) = y_0 + \int_{x_0}^x f(t, y(t)) dt$ является сжатием в соответствующем функциональном пространстве.

11. Показатели качества регрессионной модели

ПРОСТЫМИ СЛОВАМИ

Хорошая модель — не самая точная на обучающих данных, а та, что точна и при этом не переусложнена.

Коэффициент детерминации R^2

R^2 показывает, какую долю разброса зависимой переменной модель объяснила. Считается через суммы квадратов: общую (TSS), объяснённую (ESS) и остаточную (RSS):

$$R^2 = 1 - \text{RSS} / \text{TSS}, \quad R^2 \in [0, 1]$$

$R^2 = 1$ — идеальная подгонка, $R^2 = 0$ — модель не лучше простого среднего. Нецентрированный R^2 используют для регрессии без свободного члена.

Скорректированный R^2

Проблема обычного R^2 : он не убывает при добавлении любого, даже бесполезного признака. Скорректированный R^2 вводит штраф за число параметров k , поэтому может и снизиться:

$$R^2_{\text{adj}} = 1 - (\text{RSS}/(n-k)) / (\text{TSS}/(n-1))$$

Информационные критерии Акаике (AIC) и Шварца (BIC)

Они уравнивают точность модели и её сложность. Из нескольких моделей лучше та, у которой критерий МЕНЬШЕ:

$$\begin{aligned} \text{AIC} &= 2k - 2 \cdot \ln(L) \\ \text{BIC} &= k \cdot \ln(n) - 2 \cdot \ln(L) \end{aligned}$$

Здесь L — функция правдоподобия, k — число параметров, n — объём выборки. BIC сильнее штрафует сложность, особенно на больших выборках.

Тестирование значимости

- t-тест — проверяет значимость одного коэффициента (гипотеза «коэффициент равен нулю»);
- F-тест — проверяет значимость регрессии в целом или группы коэффициентов сразу;
- если p-значение меньше уровня значимости (обычно 0,05), коэффициент признаётся значимым.

ПРИМЕР

Две модели: первая с $R^2 = 0,80$ и 3 признаками, вторая с $R^2 = 0,81$ и 10 признаками. Прирост R^2 мизерный, а параметров втрое больше — скорректированный R^2 и BIC, скорее всего, выберут первую, более простую модель.

ЗАПОМНИ ГЛАВНОЕ

Запомни: обычный R^2 растёт всегда, скорректированный — нет. AIC/BIC минимизируют. Просят объяснить разницу и записать формулы.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Коэффициенты детерминации

Качество подгонки регрессионной модели измеряется через разложение общей суммы квадратов отклонений зависимой переменной на объяснённую и остаточную части:

$$\begin{aligned} \text{TSS} &= \sum (y_i - \bar{y})^2 && \text{— общая сумма квадратов} \\ \text{ESS} &= \sum (\hat{y}_i - \bar{y})^2 && \text{— объяснённая моделью} \\ \text{RSS} &= \sum (y_i - \hat{y}_i)^2 && \text{— остаточная (квадраты ошибок)} \\ \text{TSS} &= \text{ESS} + \text{RSS} && \text{(для модели со свободным членом)} \end{aligned}$$

Обычный коэффициент детерминации:

$$R^2 = \text{ESS}/\text{TSS} = 1 - \text{RSS}/\text{TSS}, \quad R^2 \in [0, 1]$$

Нецентрированный R^2 применяется в моделях без свободного члена (где разложение $\text{TSS} = \text{ESS} + \text{RSS}$, в общем случае, несправедливо):

$$R^2_{\text{uc}} = 1 - \sum (y_i - \hat{y}_i)^2 / \sum y_i^2$$

Скорректированный R^2 штрафует за число параметров k и может убывать при добавлении незначимых регрессоров:

$$R^2_{\text{adj}} = 1 - (\text{RSS}/(n - k)) / (\text{TSS}/(n - 1))$$

Информационные критерии

Критерии информации балансируют точность модели и её сложность; из двух моделей предпочтительнее та, для которой критерий меньше. L — функция правдоподобия модели.

Критерий Акаике: $AIC = 2k - 2 \cdot \ln L$
Критерий Шварца: $BIC = k \cdot \ln n - 2 \cdot \ln L$

BIC накладывает более жёсткий штраф на сложность, особенно при больших n .

Тестирование значимости

- t -тест проверяет гипотезу $H_0: \beta_j = 0$ (один коэффициент незначим). При H_0 статистика $t = \beta_j / SE(\beta_j)$ имеет распределение Стьюдента с $n - k$ степенями свободы;
- F -тест проверяет значимость регрессии в целом или подмножества коэффициентов ($H_0: \beta_1 = \beta_2 = \dots = \beta_q = 0$); статистика $F = ((RSS_R - RSS_U)/q) / (RSS_U/(n - k))$ имеет распределение Фишера;
- p -значение — наименьший уровень значимости, при котором гипотезу следует отвергнуть; обычно H_0 отвергается при $p < 0,05$.

12. Метод наименьших квадратов (МНК)

ПРОСТЫМИ СЛОВАМИ

МНК проводит линию или плоскость «по центру» облака точек, минимизируя суммарный квадрат промахов.

Модель множественной регрессии

Предполагается, что зависимая переменная — это взвешенная сумма факторов плюс случайная ошибка:

$$y_i = \beta_0 + \beta_1 \cdot x_{i1} + \dots + \beta_k \cdot x_{ik} + \varepsilon_i$$

Целевая функция МНК

Коэффициенты β подбирают так, чтобы сумма квадратов отклонений предсказаний от фактических значений была минимальной:

$$S(\beta) = \sum_i (y_i - \hat{y}_i)^2 \rightarrow \min$$

Квадраты берут по двум причинам: крупные ошибки штрафуются сильнее, и положительные отклонения не сокращаются с отрицательными.

Система нормальных уравнений

Приравняв производные целевой функции по каждому коэффициенту к нулю, получают систему нормальных уравнений. В матричной форме её решение:

$$\beta = (X^T X)^{-1} \cdot X^T \cdot y$$

Статистические свойства МНК-оценок

При выполнении классических предположений (ошибки имеют нулевое среднее, постоянную дисперсию, не коррелированы, факторы не случайны и линейно независимы) справедлива теорема Гаусса-Маркова: МНК-оценки несмещённые (в среднем равны истинным значениям) и эффективные — имеют наименьшую дисперсию среди всех линейных несмещённых оценок (свойство BLUE).

ПРИМЕР

Точки (1,2), (2,2), (3,4). Ищем прямую $y = a + b \cdot x$ по МНК. Подсчёт через нормальные уравнения даёт $b = 1$, $a = 2/3$. Прямая $y = 0,67 + x$ проходит так, что сумма квадратов вертикальных отклонений минимальна.

ЗАПОМНИ ГЛАВНОЕ

Запомни: МНК минимизирует сумму КВАДРАТОВ остатков; теорема Гаусса-Маркова даёт свойство BLUE. Попросят выписать целевую функцию и матричную формулу.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Модель множественной линейной регрессии

Рассматривается модель

$y = X \cdot \beta + \varepsilon$,
 $y \in \mathbb{R}^n$ — наблюдаемая зависимая переменная,
 $X \in \mathbb{R}^{(n \times k)}$ — матрица регрессоров (со столбцом единиц для β_0),
 $\beta \in \mathbb{R}^k$ — вектор неизвестных параметров,
 $\varepsilon \in \mathbb{R}^n$ — вектор случайных возмущений.

Целевая функция МНК

Метод наименьших квадратов минимизирует сумму квадратов остатков:

$$Q(\beta) = \sum_{i=1}^n (y_i - x_i^T \beta)^2 = (y - X\beta)^T (y - X\beta) \rightarrow \min$$

Система нормальных уравнений

Условие первого порядка $\partial Q / \partial \beta = 0$ даёт систему нормальных уравнений:

$$X^T X \cdot \hat{\beta} = X^T y$$

При условии полного ранга матрицы $X^T X$ её решение однозначно:

$$\hat{\beta} = (X^T X)^{-1} \cdot X^T y$$

Классические предположения и теорема Гаусса-Маркова

- $E[\varepsilon] = 0$ — нулевое среднее возмущений;
- $\text{Var}(\varepsilon) = \sigma^2 \cdot I$ — гомоскедастичность и отсутствие автокорреляции;
- регрессоры детерминированы (или экзогенны) и линейно независимы;
- для построения доверительных интервалов добавляется условие нормальности: $\varepsilon \sim N(0, \sigma^2 I)$.

Теорема Гаусса-Маркова. При выполнении первых трёх предположений МНК-оценка $\hat{\beta}$ является несмещённой ($E[\hat{\beta}] = \beta$) и эффективной — имеет наименьшую дисперсию среди всех линейных несмещённых оценок (BLUE — Best Linear Unbiased Estimator).

Ковариационная матрица оценок

$$\text{Cov}(\hat{\beta}) = \sigma^2 \cdot (X^T X)^{-1}$$

$\hat{\sigma}^2 = \text{RSS} / (n - k)$ — несмещённая оценка дисперсии

13. Автокорреляция случайных возмущений

ПРОСТЫМИ СЛОВАМИ

Автокорреляция не ломает сами прогнозы, но делает выводы о значимости недоверенными — её нужно находить и лечить.

Что это

Одно из классических предположений регрессии — случайные ошибки (возмущения) не коррелированы между собой. Автокорреляция — нарушение этого условия: ошибка в один момент связана с ошибкой в соседний:

$$\text{cov}(\varepsilon_t, \varepsilon_s) \neq 0 \quad \text{при} \quad t \neq s$$

Встречается почти во всех временных рядах: экономика инертна — если модель сегодня промахнулась вверх, завтра, скорее всего, тоже.

Причины

- пропущен важный объясняющий фактор, сам инерционный;
- неверно выбрана функциональная форма модели;
- инерционность самих экономических процессов;
- эффекты сглаживания и обработки данных.

Последствия

МНК-оценки коэффициентов остаются несмещёнными, но перестают быть эффективными. Главная беда — оценки стандартных ошибок становятся заниженными, поэтому t - и F -статистики завышены: модель кажется значимее, чем есть, а доверительные интервалы — уже реальных.

Тестирование

Статистика Дарбина-Уотсона (DW) проверяет автокорреляцию первого порядка:

$$DW \approx 2 \cdot (1 - \rho), \quad DW \in [0, 4]$$

DW около 2 — автокорреляции нет; ближе к 0 — положительная; ближе к 4 — отрицательная. Также применяют тест Бройша-Годфри (для более высоких порядков).

Методы устранения

- обобщённый МНК и процедура Кохрейна-Оркатта;
- добавление пропущенных факторов или лагов переменных;
- переход к разностям ряда;
- использование робастных стандартных ошибок (Ньюи-Уэста).

ПРИМЕР

Модель спроса по годовым данным дала $DW = 0,7$ — это сильная положительная автокорреляция. Вывод: t -статистики доверять нельзя, нужно применить обобщённый МНК и пересчитать значимость.

ЗАПОМНИ ГЛАВНОЕ

Запомни: автокорреляция → несмещённость сохраняется, эффективность теряется, стандартные ошибки занижены. Просьба интерпретировать значение DW.

Определение. Автокорреляция случайных возмущений — наличие статистической зависимости между возмущениями регрессионной модели в разные моменты времени:

$$\text{Cov}(\varepsilon_t, \varepsilon_s) \neq 0 \quad \text{при некоторых } t \neq s.$$

Чаще всего встречается автокорреляция первого порядка: $\varepsilon_t = \rho \cdot \varepsilon_{t-1} + u_t$, где u_t — белый шум, $|\rho| < 1$.

Причины

- инерционность экономических процессов (производство, спрос, доходы реагируют на возмущения с задержкой);
- пропущенная в модели существенная объясняющая переменная, которая сама автокоррелирована;
- неверная функциональная форма модели (нелинейная зависимость аппроксимируется линейной);
- сглаживание, интерполяция или агрегирование исходных данных.

Последствия

- МНК-оценки β остаются несмещёнными и состоятельными;
- теряется свойство эффективности (нарушено условие Гаусса-Маркова);
- стандартные ошибки коэффициентов оцениваются с искажением (обычно занижаются), вследствие чего t - и F -статистики завышены, а доверительные интервалы — уже реальных;
- выводы о значимости и прогнозы оказываются недостоверными.

Тестирование

Статистика Дарбина-Уотсона для проверки автокорреляции первого порядка:

$$DW = \sum_{t=2}^n (e_t - e_{t-1})^2 / \sum_{t=1}^n e_t^2 \approx 2 \cdot (1 - \hat{\rho})$$

$DW \approx 2$ — автокорреляции нет; $DW \rightarrow 0$ — сильная положительная; $DW \rightarrow 4$ — сильная отрицательная. Для автокорреляции высоких порядков используется **тест Бройша-Годфри** (LM-тест), более общий и применимый при наличии лагов зависимой переменной.

Методы устранения

- обобщённый метод наименьших квадратов (GLS): преобразование, превращающее ковариационную матрицу возмущений в скалярную;
- процедура Кохрейна-Оркатта — итеративная оценка ρ и пересчёт уравнения в разностях;
- включение в модель пропущенных факторов или лагов переменных;
- переход к моделям ARIMA и SARIMA;
- использование робастных стандартных ошибок Ньюи-Уэста (HAC).

14. Антагонистические игры

ПРОСТЫМИ СЛОВАМИ

В антагонистической игре каждый защищается от наихудшего сценария, и иногда эти защиты сходятся в устойчивую точку равновесия.

Допущения

Антагонистическая игра — это игра двух лиц с нулевой суммой: интересы сторон полностью противоположны, выигрыш одного в точности равен проигрышу другого. Никакого сотрудничества, оба игрока разумны и стремятся к лучшему для себя результату.

Элементы модели и формализация

- множество стратегий первого игрока (строки);
- множество стратегий второго игрока (столбцы);
- платёжная матрица A — её элемент a_{ij} показывает выигрыш первого игрока (и проигрыш второго), если выбраны стратегии i и j .

Принцип осторожности

Каждый игрок ожидает худшего. Первый игрок для каждой своей строки смотрит на минимальный выигрыш и выбирает строку с наибольшим таким минимумом — это нижняя цена игры:

$$\alpha = \max_i \min_j a_{ij} \quad (\text{максимин})$$

Второй игрок выбирает столбец с наименьшим максимальным проигрышем — верхняя цена игры:

$$\beta = \min_j \max_i a_{ij} \quad (\text{минимакс})$$

Всегда $\alpha \leq \beta$.

Ситуация равновесия (седловая точка)

Если $\alpha = \beta$, то соответствующая клетка — седловая точка, а общее значение — цена игры. В ней ни одному игроку невыгодно отклоняться в одиночку: это и есть критерий равновесия. Если седловой точки нет, игру решают в смешанных стратегиях (см. вопрос 15).

ПРИМЕР

Платёжная матрица $\begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$. По строкам минимумы: 1 и 2 → максимин $\alpha = 2$. По столбцам максимумы: 4 и 3 → минимакс $\beta = 3$. Так как $\alpha \neq \beta$, седловой точки нет — нужны смешанные стратегии.

ЗАПОМНИ ГЛАВНОЕ

Запомни: максимин $\leq$ минимакс; равенство = седловая точка = цена игры. Просят найти нижнюю и верхнюю цену по матрице.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Допущения антагонистической игры

- игра двух лиц;
- интересы участников строго противоположны (игра с нулевой суммой): выигрыш одного игрока равен проигрышу другого;
- оба игрока рациональны и стремятся к наилучшему для себя результату;
- сотрудничество между игроками исключено.

Порядок формализации

- перечисляются все возможные стратегии (чистые) первого игрока — пусть их m ;

- перечисляются стратегии второго игрока — пусть их n ;
- для каждой пары стратегий определяется значение выигрыша первого игрока (равное проигрышу второго);
- строится платёжная матрица $A = [a_{ij}]$ размера $m \times n$.

Элементы модели

- игроки — действующие стороны;
- стратегии — варианты действий;
- платёжная матрица — таблица выигрышей;
- цена игры — гарантированный результат при оптимальной игре обеих сторон.

Принцип осторожности и нижняя/верхняя цена игры

Первый игрок применяет принцип максимина: для каждой строки i находит минимальный возможный выигрыш и выбирает строку с наибольшим минимумом:

$$\alpha = \max_i \min_j a_{ij} \quad \text{— нижняя цена игры (максимин)}$$

Второй игрок применяет принцип минимакса: для каждого столбца j находит максимальный проигрыш и выбирает столбец с наименьшим максимумом:

$$\beta = \min_j \max_i a_{ij} \quad \text{— верхняя цена игры (минимакс)}$$

Всегда $\alpha \leq \beta$.

Критерий ситуации равновесия (седловой точки)

Если $\alpha = \beta$, то соответствующая клетка платёжной матрицы — седловая точка, а общее значение $v = \alpha = \beta$ — цена игры. В седловой точке ни одному игроку невыгодно изменять свою стратегию в одностороннем порядке. Если $\alpha < \beta$, игра не имеет решения в чистых стратегиях и требует перехода к смешанным.

15. Классификация игровых моделей. Чистые и смешанные стратегии

ПРОСТЫМИ СЛОВАМИ

Иногда лучшая стратегия — быть непредсказуемым, и это можно посчитать точно.

Классификация игр

- по числу игроков: двух лиц, n лиц;
- по сумме выигрышей: с нулевой суммой (антагонистические) и с ненулевой;
- по характеру ходов: одновременные и последовательные;
- по информации: с полной и неполной информацией;
- кооперативные (возможны соглашения) и некооперативные;
- особый класс — игры с природой, где второй «игрок» не враждебен, а просто случаен.

Чистые стратегии

Чистая стратегия — однозначный план: «всегда выбирай это действие». Недостаток: предсказуемого игрока соперник переиграет.

Смешанные стратегии

Смешанная стратегия — это выбор действий случайно, с заданными вероятностями. Стратегия первого игрока — вектор вероятностей $p = (p_1, \dots, p_n)$, где $p_i \geq 0$ и сумма равна 1.

$$\text{Цена игры} = \text{ожидаемый выигрыш} = \sum_i \sum_j p_i \cdot a_{ij} \cdot q_j$$

Функция выигрыша и теорема о минимаксе

Функция выигрыша сопоставляет паре стратегий ожидаемый результат. Фундаментальная теорема фон Неймана: у любой матричной игры существует решение в смешанных стратегиях, и для неё максимин равен минимаксу. То есть равновесие есть всегда, даже если в чистых стратегиях седловой точки нет.

ПРИМЕР

Камень-ножницы-бумага. В чистых стратегиях устойчивости нет: любую разгадают. Оптимальная смешанная стратегия — играть каждый вариант с вероятностью $1/3$; тогда соперник не может получить преимущество, цена игры равна нулю.

ЗАПОМНИ ГЛАВНОЕ

Запомни: смешанная стратегия = вероятности на чистых стратегиях; теорема фон Неймана гарантирует решение. Просьба привести классификацию и объяснить смысл смешивания.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Назначение игровых моделей

Игровые модели применяются для анализа конфликтных и конкурентных ситуаций: ценовая конкуренция, переговоры, распределение ресурсов, принятие решений в условиях неопределённости, проектирование стимулов и механизмов.

Классификация

- по числу игроков: игры двух лиц и n лиц;
- по сумме выигрышей: с нулевой суммой (антагонистические) и с ненулевой;
- по порядку ходов: с одновременными и с последовательными ходами;
- по информированности: с полной и неполной информацией;
- по возможности соглашений: кооперативные и некооперативные;
- по числу повторений: однократные и повторяющиеся;
- особый класс — игры с природой, где «противник» (среда) не враждебен, а действует случайно.

Чистые и смешанные стратегии

Чистая стратегия — однозначный план действий игрока. Множество чистых стратегий конечно и обозначается, например, $\{A_1, \dots, A_m\}$.

Смешанная стратегия — вероятностное распределение на множестве чистых стратегий: вектор $p = (p_1, \dots, p_m)$, где $p_i \geq 0$ и $\sum p_i = 1$. Игрок при каждом разыгрывании выбирает чистую стратегию случайно в соответствии с этим распределением.

Функция выигрыша

При смешанных стратегиях $p = (p_1, \dots, p_m)$ и $q = (q_1, \dots, q_n)$ ожидаемый выигрыш первого игрока:

$$H(p, q) = \sum_i \sum_j p_i \cdot a_{ij} \cdot q_j = p^T \cdot A \cdot q$$

Теорема о минимаксе фон Неймана

Для любой матричной игры с конечными множествами стратегий обоих игроков существует решение в смешанных стратегиях. При этом нижняя и верхняя цены игры в смешанных стратегиях совпадают и определяют цену игры v :

$$v = \max_p \min_q H(p, q) = \min_q \max_p H(p, q)$$

Оптимальные смешанные стратегии существуют для обоих игроков и находятся, в частности, методами линейного программирования.

16. Функция полезности потребителя. CES-функция

ПРОСТЫМИ СЛОВАМИ

Функция полезности — математическая модель предпочтений, а CES — гибкий шаблон, настраиваемый одним параметром замещаемости.

Функция полезности

Функция полезности $U(x_1, \dots, x_n)$ сопоставляет набору товаров число, выражающее удовлетворённость потребителя. Сами числа условны — важен только порядок (это порядковая полезность).

Свойства

- монотонность: больше товара — не меньшая полезность;
- убывающая предельная полезность: каждая следующая единица товара добавляет меньше удовольствия, чем предыдущая;
- выпуклость кривых безразличия: потребитель предпочитает сбалансированные наборы крайним.

Кривая безразличия — набор точек одинаковой полезности. Предельная норма замещения (MRS) показывает, сколько одного товара потребитель готов отдать за единицу другого, оставаясь на той же кривой.

CES-функция полезности

CES — функция с постоянной эластичностью замещения. Для двух товаров:

$$U = (a \cdot x_1^\rho + b \cdot x_2^\rho)^{1/\rho}, \quad \rho \leq 1$$

Параметр ρ задаёт, насколько легко один товар заменяет другой. Эластичность замещения $\sigma = 1/(1-\rho)$.

Частные случаи CES

- $\rho \rightarrow 1$: товары — совершенные заменители (полезность линейна);
- $\rho \rightarrow 0$: в пределе получается функция Кобба-Дугласа $U = x_1^a \cdot x_2^b$;
- $\rho \rightarrow -\infty$: совершенные дополнители (функция Леонтьева, $U = \min(x_1, x_2)$).

ПРИМЕР

Кофе и чай — близкие заменители (высокое ρ): потребитель легко переключается. Левый и правый ботинок — дополнители ($\rho \rightarrow -\infty$): полезность определяется минимумом, один без другого бесполезен.

ЗАПОМНИ ГЛАВНОЕ

Запомни: CES обобщает Кобба-Дугласа и крайние случаи через один параметр ρ . Просят записать CES и назвать её предельные случаи.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Определение. Функция полезности потребителя $U(x_1, \dots, x_n)$ ставит в соответствие набору товаров $(x_1, \dots, x_n)$ вещественное число, выражающее степень удовлетворённости потребителя. В порядковом подходе важен лишь порядок значений: $U_1 > U_2 \Leftrightarrow$ первый набор предпочтительнее второго; конкретные числовые значения не имеют самостоятельного смысла.

Свойства функции полезности

- монотонность: $\partial U / \partial x_i \geq 0$ — рост потребления не уменьшает полезности (благо «не насыщает» в неотрицательном смысле);
- убывающая предельная полезность: $\partial^2 U / \partial x_i^2 < 0$ — каждая следующая единица товара даёт меньший прирост полезности;
- выпуклость кривых безразличия — потребитель предпочитает сбалансированные наборы крайним.

Кривая безразличия — линия уровня функции полезности: множество наборов с одинаковой полезностью. **Предельная норма замещения** (MRS) показывает, от скольких единиц одного товара потребитель готов отказаться ради дополнительной единицы другого:

$$MRS(x_1, x_2) = (\partial U / \partial x_1) / (\partial U / \partial x_2)$$

CES-функция полезности

CES (Constant Elasticity of Substitution) — функция полезности с постоянной эластичностью замещения. Для двух товаров:

$$U(x_1, x_2) = (a \cdot x_1^\rho + b \cdot x_2^\rho)^{1/\rho}, \quad \rho \leq 1, \rho \neq 0$$

$a, b > 0$ — весовые коэффициенты,
эластичность замещения $\sigma = 1 / (1 - \rho)$

Частные случаи

- $\rho = 1$: $U = a \cdot x_1 + b \cdot x_2$ — линейная функция, товары как совершенные заменители, $\sigma = \infty$;
- $\rho \rightarrow 0$: в пределе получается функция Кобба-Дугласа $U = x_1^a \cdot x_2^b$, $\sigma = 1$;
- $\rho \rightarrow -\infty$: $U = \min(x_1, x_2)$ — функция Леонтьева, совершенные дополнители, $\sigma = 0$.

17. Модель Леонтьева «затраты — выпуск»

ПРОСТЫМИ СЛОВАМИ

Экономика — это сеть взаимозависимых отраслей, и линейная алгебра позволяет точно рассчитать, кто сколько должен произвести.

Идея модели

Экономика разбита на n отраслей. Чтобы что-то произвести, отрасль потребляет продукцию других отраслей (для автомобиля нужны металл, электричество, резина). Модель Леонтьева описывает эти взаимные потоки.

Матрица прямых затрат

Элемент a_{ij} — это количество продукции отрасли i , нужное для выпуска единицы продукции отрасли j . Из этих чисел составляют технологическую матрицу A .

Балансовое уравнение

Валовой выпуск x каждой отрасли должен покрыть и внутреннее потребление (Ax), и конечный спрос людей (d):

$$x = Ax + d$$

Решая относительно x , получают, какой валовой выпуск нужен для заданного конечного спроса:

$$x = (I - A)^{-1} \cdot d$$

Матрицу $(I - A)^{-1}$ называют матрицей полных затрат: она учитывает не только прямые, но и косвенные потребности (металл нужен для станка, которым делают другой металл).

Продуктивность

Модель называется продуктивной, если при любом неотрицательном спросе d решение x неотрицательно. Это выполняется, когда матрица $(I - A)^{-1}$ существует и неотрицательна (критерий — выполнение условия Хокинса-Саймона).

ПРИМЕР

Две отрасли, $A = \begin{bmatrix} 0,2 & 0,3 \\ 0,4 & 0,1 \end{bmatrix}$, конечный спрос $d = (100; 80)$. Решаем $(I - A) \cdot x = d$. Получаем валовой выпуск, который после всех внутренних расходов оставит ровно 100 и 80 единиц конечной продукции.

ЗАПОМНИ ГЛАВНОЕ

Запомни: балансовое уравнение $x = Ax + d$ и решение $x = (I - A)^{-1}d$. Просьт пояснить смысл матрицы полных затрат и условие продуктивности.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Задача управления производственным сектором

Экономика разбивается на n взаимосвязанных отраслей. Каждая отрасль производит однородную продукцию и потребляет продукцию других отраслей в качестве промежуточных ресурсов. Управляющая задача состоит в определении объёмов валового выпуска каждой отрасли, обеспечивающих заданный конечный спрос потребителей.

Матрица прямых затрат

Коэффициент прямых затрат a_{ij} — количество продукции отрасли i , расходуемое на производство одной единицы продукции отрасли j . Из коэффициентов составляется технологическая матрица $A = [a_{ij}]$.

Балансовое уравнение модели Леонтьева

Валовой выпуск x_j отрасли j распределяется на промежуточное потребление (для всех отраслей) и конечный продукт d_j :

$$x_i = \sum_{j=1}^n a_{ij} \cdot x_j + d_i, \quad i = 1, \dots, n$$

в матричной форме: $x = A \cdot x + d$

Отсюда:

$$(E - A) \cdot x = d \quad \Rightarrow \quad x = (E - A)^{-1} \cdot d$$

Матрицу $B = (E - A)^{-1}$ называют **матрицей полных затрат**: её элемент b_{ij} показывает суммарную потребность в продукции отрасли i для производства единицы конечной продукции отрасли j с учётом всех прямых и косвенных связей.

Продуктивность модели

Матрица A называется продуктивной, если для любого вектора конечного спроса $d \geq 0$ существует неотрицательное решение $x \geq 0$. Достаточные условия продуктивности:

- существование обратной матрицы $(E - A)^{-1}$ с неотрицательными элементами;
- выполнение условия Хокинса-Саймона: все главные миноры матрицы $(E - A)$ положительны;
- сумма элементов каждого столбца матрицы A меньше единицы.

18. Словари в Python

ПРОСТЫМИ СЛОВАМИ

Словарь нужен там, где данные удобнее искать по имени, а не по номеру, и где важна быстрота поиска.

Назначение

Словарь (dict) хранит пары «ключ — значение». В отличие от списка, где элемент ищут по числовому индексу, в словаре доступ идёт по осмысленному ключу — как поиск слова в обычном словаре.

Создание и изменение

```
d = {'имя': 'Анна', 'возраст': 25}
d['город'] = 'Москва'      # добавление
d['возраст'] = 26          # изменение
del d['город']              # удаление
```

Основные операции и методы

- $d[key]$ — получить значение (ошибка, если ключа нет);
- $d.get(key, default)$ — безопасно, с значением по умолчанию;
- $key \in d$ — проверка наличия ключа;
- $d.keys()$, $d.values()$, $d.items()$ — ключи, значения, пары;
- $d.update(other)$ — слияние словарей;
- $len(d)$ — число пар.

Способы итерации

```
for key in d: ...           # по ключам
for value in d.values(): ... # по значениям
for key, value in d.items(): ... # по парам
```

Как устроен внутри

Словарь реализован хеш-таблицей: ключ пропускается через хеш-функцию, которая указывает, где искать значение. Поэтому доступ, добавление и проверка наличия выполняются в среднем за постоянное время $O(1)$ — независимо от размера словаря. Из-за хеширования ключи обязаны быть неизменяемыми (строки, числа, кортежи); список ключом быть не может.

ПРИМЕР

Подсчёт частоты слов в тексте: заводим пустой словарь, идём по словам, для каждого делаем `counts[word] = counts.get(word, 0) + 1`. За один проход получаем частотность — быстро и компактно.

ЗАПОМНИ ГЛАВНОЕ

Запомни: `dict` = хеш-таблица, доступ $O(1)$, ключи неизменяемы. Просят перечислить методы и сравнить словарь со списком.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Назначение и особенности словаря

Словарь (`dict`) в Python — встроенный изменяемый контейнер ассоциативного типа, хранящий пары «ключ — значение». Ключи должны быть неизменяемыми (хешируемыми) объектами: строки, числа, кортежи. Значения могут быть любого типа. Внутренняя реализация — хеш-таблица, обеспечивающая среднее время доступа, вставки и удаления $O(1)$.

Создание и изменение

```
d = {}           # пустой словарь
d = {'name': 'Anna', 'age': 25} # литерал
d = dict(name='Anna', age=25)   # через конструктор
d['city'] = 'Moscow'            # добавление пары
d['age'] = 26                   # изменение значения
del d['city']                   # удаление по ключу
```

Основные операции и методы

- `d[key]` — получение значения; `KeyError`, если ключ отсутствует;
- `d.get(key, default)` — безопасное получение с дефолтом;
- `d.setdefault(key, default)` — получить или установить;
- `key in d` — проверка наличия ключа за $O(1)$;
- `len(d)` — количество пар;
- `d.keys()`, `d.values()`, `d.items()` — представления ключей, значений, пар;
- `d.update(other)` — слияние с другим словарём или итерируемым объектом;
- `d.pop(key, default)`, `d.popitem()` — удаление с возвратом;
- `d.clear()` — очистить словарь.

Способы итерации по словарю

```
for key in d: ...                # по ключам (по умолчанию)
for value in d.values(): ...      # по значениям
for key, value in d.items(): ...  # по парам
{k: v for k, v in d.items() if ...} # словарное включение
```

С версии Python 3.7 порядок вставки ключей сохраняется. Сравнение словарей выполняется по содержимому (порядок не учитывается). Доступны методы `|` и `|=` для объединения словарей (Python 3.9+).

19. Алгоритмы сортировки. Бинарный поиск

ПРОСТЫМИ СЛОВАМИ

Один раз потратиться на сортировку, чтобы потом искать молниеносно.

Сортировка

Сортировка — упорядочивание элементов. Сложность алгоритма оценивают числом операций в зависимости от размера данных n (нотация «О большое»).

Простые алгоритмы — $O(n^2)$

- пузырьковая сортировка — соседние элементы меняются местами, пока массив не упорядочится;
- сортировка выбором — на каждом шаге ищется минимум;
- сортировка вставками — элемент вставляется в уже отсортированную часть.
- Достоинства: просты, не требуют доп. памяти. Недостаток: медленны на больших данных.

Быстрые алгоритмы — $O(n \cdot \log n)$

- быстрая сортировка (quicksort) — выбирается опорный элемент, массив делится на «меньше» и «больше», части сортируются рекурсивно; в среднем $O(n \cdot \log n)$, в худшем $O(n^2)$;
- сортировка слиянием (merge sort) — массив делится пополам, половины сортируются и сливаются; стабильно $O(n \cdot \log n)$.

Бинарный поиск в отсортированном массиве

Если массив отсортирован, элемент ищут так: смотрят в середину; если искомое меньше — отбрасывают правую половину, если больше — левую. Каждый шаг сокращает зону поиска вдвое.

Сложность бинарного поиска: $O(\log n)$

В неотсортированном массиве остаётся только линейный перебор — $O(n)$.

ПРИМЕР

Массив из миллиона элементов. Линейный поиск в худшем случае — до 1 000 000 сравнений. Бинарный поиск — около $\log_2(1\,000\,000) \approx 20$ сравнений. Разница в 50 000 раз — вот зачем нужна сортировка.

ЗАПОМНИ ГЛАВНОЕ

Запомни: $O(n^2)$ против $O(n \cdot \log n)$ для сортировок, $O(\log n)$ против $O(n)$ для поиска. Просят сравнить алгоритмы и объяснить идею бинарного поиска.

Основные алгоритмы сортировки

Простые сортировки — $O(n^2)$:

- пузырьковая (bubble sort) — последовательно сравнивает соседние элементы и меняет их местами; $n - 1$ проход;
- сортировка выбором (selection sort) — на каждом шаге находит минимум в неотсортированной части и ставит в начало;
- сортировка вставками (insertion sort) — каждый новый элемент вставляется в уже отсортированную часть.

Эффективные сортировки — $O(n \cdot \log n)$:

- быстрая сортировка (quicksort) — выбирается опорный элемент (pivot), массив разбивается на «меньшие» и «большие», части сортируются рекурсивно; в среднем $O(n \cdot \log n)$, в худшем $O(n^2)$;
- сортировка слиянием (merge sort) — массив делится пополам, половины сортируются рекурсивно, затем сливаются; гарантированно $O(n \cdot \log n)$, стабильна, но требует дополнительной памяти $O(n)$;
- пирамидальная сортировка (heap sort) — использует структуру «куча»; гарантированно $O(n \cdot \log n)$, на месте.

Достоинства и недостатки. Простые алгоритмы наглядны, стабильны, требуют $O(1)$ дополнительной памяти; недостаток — квадратичная сложность. Эффективные алгоритмы значительно быстрее на больших массивах, но сложнее в реализации.

Бинарный (двоичный) поиск

Алгоритм поиска элемента в отсортированном массиве. На каждой итерации сравнивается искомый элемент со средним элементом активной области; в зависимости от результата область поиска сокращается вдвое.

Сложность бинарного поиска: $O(\log n)$

Сложность линейного поиска в неотсортированном массиве: $O(n)$

Бинарный поиск требует предварительной сортировки. Если поиск выполняется многократно, затраты на сортировку $O(n \cdot \log n)$ окупаются.

20. Нормальные формы реляционных отношений

ПРОСТЫМИ СЛОВАМИ

Нормализация устраняет избыточность: каждый факт хранится один раз, и обновлять его нужно тоже в одном месте.

Зачем нормализовать

Если данные дублируются (адрес клиента записан в каждой его покупке), возникают аномалии: при обновлении легко исправить не все копии, при удалении можно потерять нужный факт, при вставке — не суметь добавить запись без лишних данных.

Первая нормальная форма (1НФ)

В каждой ячейке таблицы — одно атомарное (неделимое) значение, нет повторяющихся групп и списков внутри поля. У таблицы есть первичный ключ.

Вторая нормальная форма (2НФ)

Таблица в 1НФ, и каждый неключевой атрибут зависит от ВСЕГО составного ключа, а не от его части. Устраняет частичные зависимости.

Третья нормальная форма (3НФ)

Таблица в 2НФ, и нет транзитивных зависимостей — неключевой атрибут не зависит от другого неключевого. Каждый факт определяется только ключом. Усиленный вариант — нормальная форма Бойса-Кодда (БКНФ).

Применимость

Обычно базы нормализуют до 3НФ — это хороший баланс. Но для хранилищ данных и аналитики ради скорости чтения иногда сознательно идут на денормализацию (допускают дублирование), потому что в аналитике данные чаще читают, чем меняют.

ПРИМЕР

Таблица «Заказы», где в каждой строке повторяются название и телефон клиента, нарушает 3НФ. Решение: вынести клиентов в отдельную таблицу, а в «Заказах» оставить только идентификатор клиента — теперь телефон хранится один раз.

ЗАПОМНИ ГЛАВНОЕ

Запомни: 1НФ — атомарность, 2НФ — нет частичных зависимостей, 3НФ — нет транзитивных. Просят кратко охарактеризовать формы и объяснить смысл денормализации.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Нормализация — процесс приведения реляционных отношений (таблиц) к виду, при котором устранены избыточность данных и аномалии вставки, обновления и удаления. Нормальные формы — ступенчатая иерархия требований к структуре таблиц.

Первая нормальная форма (1НФ)

Таблица находится в 1НФ, если:

- все её атрибуты атомарны (неделимы);
- отсутствуют повторяющиеся группы и многозначные поля;
- у таблицы есть первичный ключ.

Вторая нормальная форма (2НФ)

Таблица находится в 2НФ, если она находится в 1НФ и каждый неключевой атрибут функционально полно зависит от всего первичного ключа (отсутствуют частичные зависимости от частей составного ключа).

Третья нормальная форма (3НФ)

Таблица находится в 3НФ, если она находится в 2НФ и в ней отсутствуют транзитивные зависимости неключевых атрибутов от ключа (неключевой атрибут не должен зависеть от другого неключевого).

Нормальная форма Бойса-Кодда (БКНФ)

Усиление 3НФ: каждая нетривиальная функциональная зависимость имеет своим левым делом потенциальный ключ. Большинство таблиц, находящихся в 3НФ, удовлетворяют и БКНФ.

Высшие нормальные формы

4НФ устраняет нетривиальные многозначные зависимости, **5НФ** — нетривиальные зависимости соединения. На практике применяются редко.

Применимость

Для оперативных систем (OLTP) обычно нормализуют до 3НФ или БКНФ — это снижает избыточность и предотвращает аномалии. Для аналитических хранилищ данных (OLAP, схемы «звезда» и «снежинка») сознательно применяют денормализацию: дублирование данных повышает скорость чтения и упрощает запросы за счёт увеличения объёма.

21. Структура SQL-запроса. Оператор SELECT

ПРОСТЫМИ СЛОВАМИ

SQL выполняется не в том порядке, в каком пишется, — и знание реального порядка избавляет от типичных ошибок.

Основные блоки SELECT

- SELECT — какие столбцы или выражения вернуть;
- FROM — из каких таблиц брать данные;
- WHERE — условие отбора строк (до группировки);
- GROUP BY — группировка строк по значениям столбцов;
- HAVING — условие отбора уже для групп (после группировки);
- ORDER BY — сортировка результата.

Порядок написания и порядок выполнения

Пишут запрос так: SELECT — FROM — WHERE — GROUP BY — HAVING — ORDER BY. А логически выполняется он в другом порядке:

- 1) FROM — определяются источники
- 2) WHERE — отбираются строки
- 3) GROUP BY — строки собираются в группы
- 4) HAVING — отбираются группы
- 5) SELECT — вычисляются столбцы
- 6) ORDER BY — сортируется результат

Почему это важно

Из реального порядка следуют практические правила: в WHERE нельзя использовать агрегатные функции (групп ещё нет) — для условий по группам есть HAVING; псевдоним столбца из SELECT нельзя использовать в WHERE, но можно в ORDER BY.

Агрегатные функции

GROUP BY применяют вместе с агрегатными функциями COUNT, SUM, AVG, MIN, MAX — они сворачивают каждую группу строк в одно итоговое значение.

ПРИМЕР

«Города, где средняя зарплата выше 50 000»: SELECT city, AVG(salary) FROM staff GROUP BY city HAVING AVG(salary) > 50000. Условие на среднее — именно в HAVING, потому что в WHERE групп ещё не существует.

ЗАПОМНИ ГЛАВНОЕ

Запомни: порядок выполнения FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. Попросят объяснить разницу WHERE и HAVING.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Структура запроса SELECT

Запрос SELECT извлекает данные из таблиц. Полная синтаксическая структура (порядок написания):

```
SELECT    [DISTINCT] список_столбцов_или_выражений
FROM      таблицы [JOIN ...]
WHERE     условие_отбора_строк
GROUP BY  список_столбцов_группировки
HAVING    условие_отбора_групп
ORDER BY  столбцы [ASC|DESC]
LIMIT     n      [OFFSET m]
```

Назначение основных блоков

- FROM — задаёт источники данных (таблицы и их соединения);
- WHERE — фильтрует строки до группировки;
- GROUP BY — объединяет строки в группы по значениям указанных столбцов;
- HAVING — фильтрует уже сгруппированные данные;
- SELECT — формирует итоговый список столбцов и выражений (в том числе агрегатные функции);
- ORDER BY — сортирует результат.

Последовательность логического выполнения

Логический порядок выполнения отличается от порядка написания:

- 1) FROM — формируется рабочая таблица из источников
- 2) WHERE — отбираются строки
- 3) GROUP BY — строки группируются
- 4) HAVING — отбираются группы
- 5) SELECT — вычисляются итоговые столбцы и выражения
- 6) ORDER BY — сортируется результат
- 7) LIMIT — отрезается заданная часть результата

Следствия порядка: в WHERE нельзя использовать агрегатные функции и псевдонимы из SELECT (групп и итоговых выражений ещё не существует); псевдонимы доступны в ORDER BY.

Агрегатные функции

Применяются к группе строк и возвращают одно значение: COUNT, SUM, AVG, MIN, MAX, STDDEV, VAR. При наличии GROUP BY применяются к каждой группе отдельно.

22. Запросы с несколькими таблицами. Соединения (JOIN)

ПРОСТЫМИ СЛОВАМИ

JOIN собирает разрозненные таблицы в одну осмысленную, а вид JOIN решает судьбу строк без пары.

Зачем соединять таблицы

В нормализованной базе данные разложены по таблицам (клиенты отдельно, заказы отдельно). Чтобы получить связную картину, таблицы соединяют по общему полю — обычно по ключу.

Виды соединений (JOIN)

- INNER JOIN — только строки, у которых есть пара в обеих таблицах; строки без пары отбрасываются;
- LEFT JOIN — все строки левой таблицы; если пары справа нет, правые столбцы заполняются NULL;
- RIGHT JOIN — симметрично, все строки правой таблицы;
- FULL JOIN — все строки обеих таблиц, непарные дополняются NULL;
- CROSS JOIN — все возможные сочетания строк (декартово произведение).

Синтаксис

```
SELECT c.name, o.sum
FROM clients c
LEFT JOIN orders o ON c.id = o.client_id
```

Операции над множествами строк

- UNION — объединение результатов двух запросов (дубликаты убираются; UNION ALL — оставляет);
- INTERSECT — пересечение, только общие строки;
- EXCEPT (MINUS) — разность, строки первого запроса без строк второго.

ПРИМЕР

INNER JOIN клиентов и заказов покажет только клиентов, сделавших хотя бы один заказ. LEFT JOIN покажет ВСЕХ клиентов, в том числе тех, у кого заказов нет (в столбцах заказа будет NULL) — так удобно искать неактивных клиентов.

ЗАПОМНИ ГЛАВНОЕ

Запомни: INNER теряет непарные строки, LEFT сохраняет левую таблицу. Просят описать виды JOIN и отличить их от UNION/INTERSECT.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Соединения отношений (JOIN)

Соединения позволяют объединять строки из нескольких таблиц по условию связи (обычно — равенство значений ключевых полей).

- INNER JOIN — внутреннее соединение: только пары строк, для которых выполнено условие соединения;
- LEFT OUTER JOIN (LEFT JOIN) — левое внешнее соединение: все строки левой таблицы; правые поля без пары заполняются NULL;
- RIGHT OUTER JOIN — правое внешнее соединение, симметричное левому;
- FULL OUTER JOIN — полное внешнее соединение: все строки обеих таблиц, недостающие значения — NULL;
- CROSS JOIN — декартово произведение: все возможные пары строк;
- SELF JOIN — соединение таблицы с собой (через псевдонимы).

Синтаксис соединения

```
SELECT t1.col, t2.col
FROM   t1
INNER JOIN t2 ON t1.key = t2.key
WHERE ...
```

Операции над множествами строк

- UNION — объединение результатов двух запросов с устранением дубликатов; UNION ALL сохраняет дубликаты;
- INTERSECT — пересечение: строки, присутствующие в обоих результатах;
- EXCEPT (или MINUS) — разность: строки первого запроса, не входящие во второй;
- обязательное требование — совпадение числа и типов столбцов в объединяемых запросах.

Характеристики соединений

- INNER JOIN — сужает результат (теряет строки без пары);
- LEFT/RIGHT JOIN — сохраняют все строки соответствующей таблицы;
- FULL JOIN — не теряет ни одной строки;
- CROSS JOIN — резко увеличивает результат ($n \times m$ строк), применяется редко;
- выбор вида JOIN определяет поведение со «строками без пары».

23. Градиентные методы поиска экстремума

ПРОСТЫМИ СЛОВАМИ

Чтобы дойти до низшей точки, на каждом шаге двигайся туда, где спуск круче всего.

Постановка задачи

Нужно найти точку, в которой функция многих переменных принимает минимум (или максимум): минимум ошибки модели, максимум прибыли. Аналитически решить систему $\nabla f = 0$ часто невозможно — используют итерационные методы.

Градиент

Градиент ∇f — это вектор частных производных. Он указывает направление наискорейшего РОСТА функции в данной точке, а его длина — крутизну.

Метод наискорейшего спуска

Чтобы найти минимум, делают шаги в сторону, противоположную градиенту:

$$x_{(k+1)} = x_k - \alpha \cdot \nabla f(x_k)$$

Здесь α — длина шага (скорость обучения). Слишком большой шаг приводит к «перескокам», слишком малый — к медленной сходимости. Образ: спуск с горы по самому крутому склону, шаг за шагом, пока не придёшь в низину.

Методы при наличии ограничений

- метод проекции градиента — после обычного шага точку проецируют обратно в допустимую область;
- метод условного градиента (Франка-Вульфа) — направление движения выбирается сразу с учётом ограничений, решением вспомогательной линейной задачи.

Значение

Градиентный спуск — основа обучения почти всех нейронных сетей и большинства моделей машинного обучения.

ПРИМЕР

$f(x, y) = x^2 + y^2$, минимум в точке $(0,0)$. Старт $(2, 2)$, шаг $\alpha = 0,1$. Градиент в старте $(4, 4)$. Новая точка: $(2,2) - 0,1 \cdot (4,4) = (1,6; 1,6)$. Повторяя шаги, последовательность сходится к нулю.

ЗАПОМНИ ГЛАВНОЕ

Запомни: градиент указывает рост, спуск идёт против него; шаг α важно подобрать. Просят записать формулу шага и пояснить роль α .

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Градиентные методы — итерационные алгоритмы поиска локального экстремума дифференцируемой функции многих переменных $f: \mathbb{R}^n \rightarrow \mathbb{R}$. Они используют направление антиградиента (для минимизации) или градиента (для максимизации) функции.

Градиент

Градиент $\nabla f(x)$ — вектор частных производных функции. Он указывает направление наискорейшего возрастания функции в точке x ; антиградиент $-\nabla f(x)$ — направление наискорейшего убывания.

Метод наискорейшего спуска (подъёма)

Общая итерационная формула для минимизации:

$$x_{(k+1)} = x_k - \alpha_k \cdot \nabla f(x_k)$$

Длина шага $\alpha_k > 0$ выбирается одним из способов:

- постоянный шаг $\alpha_k = \text{const}$ (требует малого α для сходимости);
- дробный шаг с автоматическим уменьшением до выполнения условий убывания;
- оптимальный шаг: $\alpha_k = \operatorname{argmin}_{(\alpha > 0)} f(x_k - \alpha \cdot \nabla f(x_k))$ — одномерная задача минимизации вдоль направления спуска;
- правило Армихо или Вольфе — гибридный подход.

Критерий остановки: $\|\nabla f(x_k)\| < \varepsilon$ или $\|x_{(k+1)} - x_k\| < \varepsilon$.

Метод проекции градиента

Применяется при наличии ограничений $x \in \Omega$. После обычного градиентного шага точка проецируется обратно на допустимое множество:

$$x_{(k+1)} = P_{\Omega}(x_k - \alpha_k \cdot \nabla f(x_k)),$$

где P_{Ω} — оператор проекции на множество Ω . Эффективен, когда проекция вычисляется просто (например, для прямоугольных ограничений или симплекса).

Метод условного градиента (Франка-Вульфа)

На каждом шаге направление движения находится решением вспомогательной задачи линейного программирования:

$$\begin{aligned} s_k &= \operatorname{argmin}_{(s \in \Omega)} (\nabla f(x_k), s) \\ x_{(k+1)} &= x_k + \alpha_k \cdot (s_k - x_k), \quad \alpha_k \in [0, 1] \end{aligned}$$

Метод подходит для задач с выпуклыми компактными ограничениями, особенно когда минимизация линейной функции на Ω выполняется эффективно.

24. Целочисленное программирование. Метод Гомори. Метод ветвей и границ

ПРОСТЫМИ СЛОВАМИ

Целочисленные задачи нельзя решать округлением — нужны умные методы, аккуратно сужающие поиск.

Постановка задачи

Задача целочисленного программирования — это задача линейного программирования (линейная целевая функция и линейные ограничения), но с дополнительным требованием: переменные должны быть целыми. Так бывает, когда нельзя купить 2,7 станка или нанять 3,5 человека.

Простое округление решения «без целочисленности» обычно даёт либо неоптимальный, либо вовсе недопустимый ответ — поэтому нужны специальные методы.

Метод Гомори (метод отсечений)

Сначала задачу решают без условия целочисленности (релаксация). Если решение дробное, к ограничениям добавляют новое линейное неравенство — отсечение Гомори. Оно отсекает текущее дробное решение, но не отсекает ни одной целой точки. Задачу решают заново, и так повторяют, пока решение не станет целым.

Метод ветвей и границ

Задачу разбивают на подзадачи (ветвление). Если переменная получилась дробной, например $x = 3,4$, создают две ветви: « $x \leq 3$ » и « $x \geq 4$ ». Для каждой ветви решают релаксацию — она даёт границу (оценку лучшего возможного значения). Ветви, чья граница хуже уже найденного целого решения, отбрасываются целиком — это избавляет от полного перебора.

ветвление: $x \leq \lfloor x^* \rfloor$ или $x \geq \lceil x^* \rceil$

ПРИМЕР

Релаксация дала $x = 3,4$. Округление вниз (3) может быть не оптимальным, вверх (4) — нарушить ограничение. Метод ветвей и границ честно проверит обе ветви $x \leq 3$ и $x \geq 4$ и отсечёт бесперспективную.

ЗАПОМНИ ГЛАВНОЕ

Запомни: Гомори добавляет отсечения, ветви и границы строит дерево подзадач с отбрасыванием ветвей. Попросят объяснить, почему нельзя просто округлять.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Постановка задачи целочисленного программирования

Задача целочисленного линейного программирования (ЗЦЛП) формулируется как задача линейного программирования с дополнительным условием целочисленности всех или части переменных:

$$\begin{aligned} \max \text{ (или min)} \quad & c^T \cdot x \\ \text{при условиях} \quad & A \cdot x \leq b, \quad x \geq 0, \\ & x_j \in \mathbb{Z} \text{ для } j \in J \end{aligned}$$

При $J = \{1, \dots, n\}$ получаем полностью целочисленную задачу, при $J \subset \{1, \dots, n\}$ — частично целочисленную. Особый случай — булевы (0-1) переменные.

Метод Гомори (метод отсечений)

- решается релаксированная задача — обычная задача ЛП без условия целочисленности;
- если оптимальное решение целочисленно, оно является решением ЗЦЛП; иначе — берётся переменная x_j с дробным значением;
- по этой переменной строится правильное отсечение Гомори — новое линейное неравенство, отсекающее текущее дробное решение, но не отсекающее ни одной целочисленной точки допустимой области;
- отсечение добавляется к системе ограничений, и задача решается заново;
- процесс повторяется до получения целочисленного решения (теорема Гомори гарантирует конечность процесса).

Метод ветвей и границ

- решается релаксация исходной задачи; если решение целочисленно — оно оптимально;
- выбирается переменная x_j с дробным значением x_j^* ; задача разделяется на две подзадачи (ветвление):
- подзадача 1: к ограничениям добавлено $x_j \leq \lfloor x_j^* \rfloor$, подзадача 2: $x_j \geq \lceil x_j^* \rceil$;
- для каждой подзадачи находится релаксированное решение — оценка границы возможного значения целевой функции (bound);
- ветви, чья граница хуже уже найденного целочисленного решения, отсекаются (pruning);
- процесс продолжается, пока не будут просмотрены все перспективные ветви; рекорд — лучшее найденное целочисленное решение.

Метод ветвей и границ универсален и широко применяется в коммерческих решателях (CPLEX, Gurobi). Метод Гомори часто комбинируется с методом ветвей и границ (метод branch-and-cut).

25. Многокритериальная оптимизация. Парето-эффективная граница

ПРОСТЫМИ СЛОВАМИ

При многих целях нет одного «правильного» ответа — есть множество компромиссов, а финальный выбор делает человек.

Постановка задачи

В реальных задачах целей несколько, и они конфликтуют: хочется и дёшево, и качественно; и доходно, и безопасно. Улучшая один критерий, ухудшаешь другой — поэтому единственного оптимума нет.

Эффективность по Парето

Решение называется Парето-эффективным (Парето-оптимальным), если его нельзя улучшить ни по одному критерию, не ухудшив при этом какой-то другой. Решение, которое можно улучшить «бесплатно» (по одному критерию, не теряя в остальных), называют доминируемым — оно заведомо плохое.

Парето-граница

Множество всех Парето-эффективных решений образует Парето-границу (множество компромиссов). Внутри этой границы выбор уже зависит от предпочтений лица, принимающего решение.

Методы решения

- метод взвешенной суммы — критерии сводят в один: $F = \sum w_i \cdot f_i$, с весами, отражающими важность;
- метод главного критерия — оптимизируют один (главный) критерий, остальные переводят в ограничения «не хуже заданного уровня»;
- метод идеальной точки — ищут решение, ближайшее к недостижимому идеалу;
- построение всей Парето-границы и передача выбора человеку.

ПРИМЕР

Выбор инвестпортфеля: критерии — высокая доходность и низкий риск. Портфель с доходностью 8% и риском 5 доминирует портфель с доходностью 7% и риском 6. А вот портфель «10% и риск 9» и портфель «6% и риск 3» оба на Парето-границе — между ними выбирает инвестор по своей готовности рисковать.

ЗАПОМНИ ГЛАВНОЕ

Запомни: Парето-эффективность = «нельзя улучшить одно, не ухудшив другое». Попросят дать определение и назвать методы свёртки критериев.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Постановка многокритериальной задачи

Многокритериальная задача оптимизации имеет несколько целевых функций, оптимизируемых одновременно:

$$f(x) = (f_1(x), f_2(x), \dots, f_m(x)) \rightarrow \min \text{ (или } \max)$$

при $x \in X$ (множество допустимых решений)

В общем случае единого оптимума, минимизирующего все критерии одновременно, не существует — критерии конфликтуют.

Доминирование по Парето

Решение $y \in X$ **доминирует по Парето** решение $x \in X$ (в задаче минимизации), если выполнены условия:

$$f_i(y) \leq f_i(x) \text{ для всех } i = 1, \dots, m,$$

и существует k , для которого $f_k(y) < f_k(x)$.

Парето-эффективным (Парето-оптимальным) называется решение, которое не доминируется никаким другим: его нельзя улучшить ни по одному критерию, не ухудшив хотя бы один другой.

Парето-эффективная граница

Множество всех Парето-эффективных решений в пространстве переменных называется **Парето-множеством**; его образ в пространстве критериев — **Парето-границей** (frontier). Парето-граница содержит все рациональные компромиссы; внутри неё выбор зависит от предпочтений лица, принимающего решение.

Методы решения

- **Метод взвешенной суммы (свёртки):** $F(x) = \sum w_i \cdot f_i(x) \rightarrow \min$; веса $w_i \geq 0$ отражают важность критериев. При фиксированных весах решение принадлежит Парето-границе.
- **Метод главного критерия:** оптимизируется один (главный) критерий, остальные переводятся в ограничения $f_i(x) \leq L_i$.
- **Метод идеальной точки:** определяется недостижимый идеал $f^* = (\min f_1, \dots, \min f_m)$; ищется решение, ближайшее к f^* в выбранной метрике.
- **Метод последовательных уступок:** критерии упорядочены по важности; сначала оптимизируется главный, затем следующий с ограничением «уступки» по первому, и т.д.
- **Полное построение Парето-границы** с предоставлением выбора человеку — в задачах с двумя-тремя критериями.

Часть 2. Дисциплины профиля

Профиль «Прикладное машинное обучение». 25 вопросов.

Вторая часть — профессиональное ядро программы: финансовая и страховая математика, анализ сетей, машинное и глубокое обучение, параллельные вычисления и инструменты обработки данных.

1. Денежные потоки. Чистая приведённая стоимость. Ренты. IRR

ПРОСТЫМИ СЛОВАМИ

Чтобы честно сравнивать платежи в разные годы, их приводят к одному моменту времени.

Денежный поток и стоимость денег во времени

Денежный поток — это набор платежей CF_t , разнесённых по моментам времени. Главный принцип финансовой математики: рубль сегодня дороже рубля завтра, потому что сегодняшний рубль можно вложить под процент.

Приведённая стоимость (PV)

Чтобы сравнивать платежи разных лет, их дисконтируют — приводят к сегодняшнему моменту, деля на $(1 + r)$ в степени t :

$$PV = \sum_t CF_t / (1 + r)^t$$

Чистая приведённая стоимость (NPV)

NPV — это приведённая стоимость всех будущих доходов минус начальные вложения. Критерий: проект выгоден, если $NPV > 0$.

$$NPV = -CF_0 + \sum_{(t=1..n)} CF_t / (1 + r)^t$$

Ренты (аннуитеты)

Рента — поток одинаковых регулярных платежей R (кредитные взносы, пенсия). Приведённая стоимость постоянной ренты:

$$PV = R \cdot (1 - (1 + r)^{-n}) / r$$

Ренты можно объединять и заменять эквивалентными — лишь бы совпадали их приведённые стоимости.

Внутренняя норма доходности (IRR)

IRR — это такая ставка дисконтирования, при которой NPV проекта обращается в ноль:

$$NPV(IRR) = 0$$

Проект выгоден, если IRR выше стоимости капитала. IRR находят численно (подбором), так как уравнение нелинейное.

ПРИМЕР

Проект: вложить 100, получить 60 через год и 70 через два года при $r = 10\%$. $NPV = -100 + 60/1,1 + 70/1,21 = -100 + 54,5 + 57,9 = 12,4 > 0$ — проект выгоден.

ЗАПОМНИ ГЛАВНОЕ

Запомни: $NPV > 0 \rightarrow$ принять; IRR — ставка, обнуляющая NPV. Просят формулы PV, NPV, ренты и критерий по IRR.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Денежный поток и приведённая стоимость

Денежный поток — последовательность платежей CF_t , разнесённых по моментам времени $t = 0, 1, \dots, n$. Принцип финансовой математики: рубль, доступный сегодня, имеет большую ценность, чем рубль, ожидаемый в будущем. Для сопоставления платежи дисконтируются — приводятся к одному моменту времени по ставке дисконтирования r :

$$PV = \sum_{t=0}^n CF_t / (1 + r)^t \quad - \text{приведённая стоимость потока}$$

Чистая приведённая стоимость (NPV)

NPV — разность приведённой стоимости будущих доходов и приведённой стоимости вложений. Для инвестиционного проекта с начальными затратами $CF_0 < 0$:

$$NPV = -CF_0 + \sum_{t=1}^n CF_t / (1 + r)^t$$

Критерий принятия решения: проект эффективен, если $NPV > 0$; при сравнении проектов предпочитается проект с большим NPV.

Рента (аннуитет)

Рента — поток одинаковых регулярных платежей R за период n . Приведённая стоимость постнумерандо-ренты:

$$PV = R \cdot (1 - (1 + r)^{-n}) / r = R \cdot a(n, r)$$

Объединение и замена рент: две ренты эквивалентны, если их приведённые стоимости равны. На этом принципе строятся сравнение различных схем платежей и реструктуризация долга.

Внутренняя норма доходности (IRR)

IRR — такая ставка дисконтирования, при которой NPV проекта обращается в ноль:

$$NPV(IRR) = -CF_0 + \sum_{t=1}^n CF_t / (1 + IRR)^t = 0$$

Экономический смысл IRR — максимальная стоимость капитала, при которой проект ещё выгоден. **Критерий:** проект эффективен, если IRR превышает стоимость капитала (требуемую норму доходности). Уравнение для IRR нелинейное и в общем случае решается численно.

2. Дюрация. Выпуклость облигаций. Иммунизация

ПРОСТЫМИ СЛОВАМИ

Дюрация измеряет процентный риск облигации, а иммунизация позволяет его нейтрализовать.

Облигация и процентный риск

Облигация даёт купонные платежи и возврат номинала. Её цена — это приведённая стоимость всех будущих платежей. Когда рыночная ставка растёт, цена облигации падает, и наоборот — это процентный риск.

Дюрация

Дюрация — это средневзвешенный срок поступления платежей, где веса — доли приведённых стоимостей платежей в цене облигации:

$$D = \sum_t t \cdot PV_t / P$$

Дюрация одновременно служит мерой чувствительности цены к ставке: чем больше дюрация, тем сильнее облигация реагирует. Приблизённо изменение цены:

$$\Delta P/P \approx -D_{\text{mod}} \cdot \Delta r$$

Выпуклость

Дюрация описывает зависимость цены от ставки линейно, но настоящая кривая изогнута. Выпуклость C — поправка второго порядка, уточняющая оценку, особенно при больших изменениях ставки:

$$\Delta P/P \approx -D_{\text{mod}} \cdot \Delta r + \frac{1}{2} \cdot C \cdot (\Delta r)^2$$

Иммунизация (теорема об иммунизации)

Если инвестору нужно расплатиться через определённый срок, он может защититься от колебаний ставки, подобрав портфель облигаций так, чтобы его дюрация равнялась горизонту инвестирования. Тогда потери от изменения цены и выигрыш от реинвестирования купонов компенсируют друг друга.

ПРИМЕР

Дюрация облигации равна 5. Если ставка вырастет на 1%, цена облигации упадёт примерно на 5%. Облигация с дюрацией 2 при том же изменении подешевеет лишь на 2% — она менее рискованна.

ЗАПОМНИ ГЛАВНОЕ

Запомни: дюрация — и средний срок, и мера риска; иммунизация = дюрация портфеля равна горизонту. Просят формулу дюрации и смысл выпуклости.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Дюрация (Маколея)

Дюрация потока платежей $P_1, P_2, \dots, P_n$ при ставке y — средневзвешенный срок поступления платежей, где весами служат доли приведённых стоимостей в общей цене:

$$D = \sum_{t=1}^n t \cdot P_t / (1+y)^t / \sum_{t=1}^n P_t / (1+y)^t$$

Дюрация измеряется в единицах времени (годах). Для облигации дюрация — одновременно средний срок возврата средств и мера чувствительности цены к изменению ставки.

Модифицированная дюрация и формула чувствительности

$$D_{\text{mod}} = D / (1 + y)$$

$$\Delta P / P \approx - D_{\text{mod}} \cdot \Delta y$$

Чем больше дюрация, тем сильнее цена облигации реагирует на изменение процентной ставки.

Свойства дюрации

- для бескупонной облигации дюрация равна сроку до погашения;
- для купонной облигации дюрация меньше срока до погашения;
- дюрация портфеля — взвешенное среднее дюраций активов;
- дюрация убывает с ростом купонной ставки и с ростом доходности к погашению.

Выпуклость облигации

Линейная аппроксимация по дюрации работает только для малых изменений ставки.

Выпуклость C — поправка второго порядка:

$$C = (1/P) \cdot \sum_{t=1}^n t \cdot (t+1) \cdot P_t / (1+y)^{(t+2)}$$

$$\Delta P / P \approx - D_{\text{mod}} \cdot \Delta y + \frac{1}{2} \cdot C \cdot (\Delta y)^2$$

Теорема об иммунизации

Пусть инвестор имеет обязательство выплатить сумму L через N лет (горизонт инвестирования). Если он формирует портфель облигаций с приведённой стоимостью, равной L , и дюрацией, равной N , то при малом параллельном сдвиге кривой доходности изменение стоимости портфеля и изменение приведённой стоимости обязательства взаимно компенсируются — обязательство остаётся выполнимым. Это явление называется **иммунизацией портфеля**.

3. Портфель ценных бумаг: доходность и риск

ПРОСТЫМИ СЛОВАМИ

Риск можно снизить без потери доходности, грамотно комбинируя непохожие активы.

Доходность и риск портфеля

Портфель — набор активов с долями капитала w_i (сумма долей равна 1). Ожидаемая доходность портфеля — взвешенное среднее доходностей активов:

$$r_p = \sum_i w_i \cdot r_i$$

Риск измеряют дисперсией (или стандартным отклонением) доходности. Для портфеля дисперсия учитывает и ковариации активов:

$$\sigma^2_p = \sum_i \sum_j w_i \cdot w_j \cdot \sigma_{ij}$$

Диверсификация

Ключевая идея: если активы ведут себя по-разному (низкая или отрицательная ковариация), риск портфеля оказывается меньше, чем у каждого актива по отдельности — провалы одного компенсируются ростом другого. Это снижение риска «бесплатно».

Эффективное множество

Эффективное множество (граница эффективности Марковица) — это портфели, дающие максимум ожидаемой доходности при каждом уровне риска. Рациональный инвестор выбирает только из них.

Оптимальный портфель с безрисковым активом

Если добавить безрисковый актив (гособлигации), эффективная граница превращается в прямую — линию рынка капитала. Оптимальный выбор любого инвестора — это комбинация безрискового актива и одного и того же рискованного «рыночного» портфеля; различается лишь пропорция (теорема разделения).

ПРИМЕР

Два актива с доходностью 10%, но движущихся в противоположные стороны. По отдельности каждый рискован. Портфель 50/50: средняя доходность те же 10%, но колебания гасятся — риск резко падает. Это и есть выгода диверсификации.

ЗАПОМНИ ГЛАВНОЕ

Запомни: риск портфеля зависит от ковариаций; диверсификация снижает риск. Просят формулы r_p и σ_p^2 и смысл эффективной границы.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Доходность и риск портфеля

Портфель — набор из n активов с долями капитала $w_1, \dots, w_n$, где $\sum w_i = 1$, $w_i \geq 0$ (при отсутствии коротких продаж). Ожидаемая доходность портфеля — взвешенное среднее доходностей активов:

$$E[r_p] = \sum_i w_i \cdot E[r_i]$$

Риск измеряется дисперсией доходности и учитывает ковариации активов:

$$\begin{aligned} \text{Var}(r_p) &= \sum_i \sum_j w_i \cdot w_j \cdot \sigma_{ij} \\ &= w^T \cdot \Sigma \cdot w, \end{aligned}$$

где Σ — ковариационная матрица доходностей.

Эффект диверсификации

При объединении активов с низкой или отрицательной ковариацией дисперсия портфеля оказывается меньше, чем у каждого актива в отдельности. Это снижение риска без потери ожидаемой доходности и есть выгода диверсификации.

Задача Марковица и эффективное множество

Задача оптимизации портфеля:

$$\begin{aligned} \min \quad & w^T \cdot \Sigma \cdot w \\ \text{при} \quad & \mu^T \cdot w = E^* \quad (\text{заданная ожидаемая доходность}), \\ & 1^T \cdot w = 1, \quad (\text{опционально } w \geq 0) \end{aligned}$$

Эффективное множество (граница Марковица) — множество портфелей, дающих максимум ожидаемой доходности при заданном уровне риска (или, эквивалентно, минимум риска при заданной доходности). Рациональный инвестор выбирает только из этого множества.

Оптимальный портфель при наличии безрискового актива

Пусть на рынке имеется безрисковый актив с доходностью r_f . Тогда эффективное множество в координатах «риск-доходность» превращается в прямую — **линию рынка капитала** (CML), касающуюся границы Марковица в одной точке — рыночном портфеле М. Любой эффективный портфель есть комбинация безрискового актива и рыночного портфеля:

$$r_p = \alpha \cdot r_f + (1 - \alpha) \cdot r_M$$

Это утверждение известно как **теорема разделения**: выбор рискованного портфеля (М) не зависит от индивидуальных предпочтений инвестора; различается только доля α в безрисковом активе.

4. Биномиальная модель ценообразования. Модель Кокса-Росса-Рубинштейна

ПРОСТЫМИ СЛОВАМИ

Сложную задачу оценки опциона решают пошагово, перебирая «вверх/вниз» и считая цену с конца к началу.

Что такое опцион

Опцион — это право (но не обязанность) купить (call) или продать (put) актив по заранее оговорённой цене исполнения K . Вопрос: сколько стоит такое право сегодня?

Идея биномиальной модели

Время до исполнения разбивают на маленькие шаги. На каждом шаге цена актива может только вырасти в u раз или упасть в d раз («би» — два исхода). Получается дерево возможных цен.

Риск-нейтральная вероятность

Цену опциона считают не через реальные вероятности роста, а через риск-нейтральную вероятность q — она подобрана так, чтобы ожидаемая доходность актива равнялась безрисковой ставке:

$$q = (e^{(r \cdot \Delta t)} - d) / (u - d)$$

Расчёт цены: движение назад по дереву

На последнем шаге выплата опциона известна (например, для call это $\max(S - K, 0)$). Двигаясь от конца к началу, цену в каждом узле находят как дисконтированное ожидание двух будущих значений:

$$C = e^{(-r \cdot \Delta t)} \cdot (q \cdot C_{up} + (1 - q) \cdot C_{down})$$

Модель Кокса-Росса-Рубинштейна (CRR)

CRR — это конкретный выбор параметров дерева через волатильность σ : $u = e^{(\sigma \cdot \sqrt{\Delta t})}$, $d = 1/u$. При увеличении числа шагов результат CRR сходится к формуле Блэка-Шоулза.

ПРИМЕР

Один шаг: цена акции 100, может стать 120 (u) или 80 (d). Опцион call с $K = 100$: в верхнем узле выплата 20, в нижнем 0. Зная q , цену опциона находят как дисконтированное среднее этих двух значений.

ЗАПОМНИ ГЛАВНОЕ

Запомни: дерево «вверх/вниз», риск-нейтральная вероятность q , расчёт от конца к началу. Просят формулу q и описать алгоритм.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Биномиальная модель ценообразования

Биномиальная модель описывает динамику цены актива дискретно: за один шаг времени Δt цена S может только умножиться на u (вверх) или на d (вниз), где $u > 1 > d > 0$. Дерево цен строится итеративно.

Риск-нейтральная вероятность

Принцип отсутствия арбитража требует, чтобы ожидаемая доходность актива при **риск-нейтральной мере** равнялась безрисковой ставке r . Отсюда:

$$q = (e^{(r \cdot \Delta t)} - d) / (u - d)$$

q — риск-нейтральная вероятность роста; $1 - q$ — падения. Условие отсутствия арбитража: $d < e^{(r \cdot \Delta t)} < u$.

Оценка опционов

Цена опциона европейского типа в любой узловой точке вычисляется методом обратной индукции — от конца к началу:

$$C_t = e^{(-r \cdot \Delta t)} \cdot (q \cdot C_{(t+\Delta t)}^{(up)} + (1 - q) \cdot C_{(t+\Delta t)}^{(down)})$$

На последнем шаге используются известные выплаты опциона: для call $\max(S - K, 0)$, для put $\max(K - S, 0)$. Для американских опционов в каждом узле дополнительно сравнивается полученное значение со стоимостью немедленного исполнения, и выбирается большее.

Модель Кокса-Росса-Рубинштейна (CRR)

CRR — конкретная параметризация биномиальной модели через волатильность σ актива:

$$u = e^{(\sigma \cdot \sqrt{\Delta t})}, \\ d = 1 / u = e^{(-\sigma \cdot \sqrt{\Delta t})}$$

При числе шагов $n \rightarrow \infty$ цена опциона по модели CRR сходится к цене по формуле Блэка-Шоулза.

5. Модель Блэка-Шоулза

ПРОСТЫМИ СЛОВАМИ

Если риск можно полностью захеджировать, то у опциона есть единственная справедливая цена, и её даёт формула.

Суть модели

Модель Блэка-Шоулза даёт формулу справедливой цены европейского опциона при предположении, что цена базового актива меняется случайно и непрерывно (геометрическое броуновское движение). По сути это предельный случай биномиальной модели при бесконечном числе шагов.

Уравнение Блэка-Шоулза

Цена любого производного инструмента подчиняется дифференциальному уравнению Блэка-Шоулза в частных производных. Идея его вывода — хеджирование: продавец опциона постоянно подстраивает портфель из акций и денег (дельта-хеджирование) так, чтобы свести риск к нулю. Раз риска нет, портфель приносит безрисковую ставку — отсюда и уравнение.

Формула цены опциона call

$$\begin{aligned}C &= S \cdot N(d1) - K \cdot e^{(-r \cdot T)} \cdot N(d2) \\d1 &= [\ln(S/K) + (r + \sigma^2/2) \cdot T] / (\sigma \cdot \sqrt{T}) \\d2 &= d1 - \sigma \cdot \sqrt{T}\end{aligned}$$

Здесь $N(\cdot)$ — функция стандартного нормального распределения.

Пять величин, от которых зависит цена

- S — текущая цена базового актива;
- K — цена исполнения;
- T — время до исполнения;
- r — безрисковая ставка;
- σ — волатильность актива (единственный параметр, который не наблюдается напрямую, его оценивают).

ПРИМЕР

Чем выше волатильность σ , тем дороже опцион: большой разброс цены увеличивает шанс крупного выигрыша, а потери ограничены уплаченной премией. Поэтому опционы на «спокойные» акции дешевле, чем на «бурные».

ЗАПОМНИ ГЛАВНОЕ

Запомни: вывод через безрисковое хеджирование; цена зависит от 5 параметров. Попросят назвать эти параметры и объяснить роль волатильности.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Модель Блэка-Шоулза-Мертона — непрерывная модель ценообразования производных финансовых инструментов. Цена базового актива $S(t)$ описывается стохастическим дифференциальным уравнением (геометрическое броуновское движение):

$$dS = \mu \cdot S \cdot dt + \sigma \cdot S \cdot dW,$$

где μ — снос, σ — волатильность, dW — приращение винеровского процесса.

Уравнение Блэка-Шоулза

Цена $V(S, t)$ производного инструмента, выплата которого зависит от S , удовлетворяет дифференциальному уравнению в частных производных:

$$\partial V / \partial t + r \cdot S \cdot \partial V / \partial S + \frac{1}{2} \cdot \sigma^2 \cdot S^2 \cdot \partial^2 V / \partial S^2 - r \cdot V = 0$$

Вывод основан на построении безрискового хеджирующего портфеля из опциона и базового актива (дельта-хеджирование): подбирая пропорции, можно мгновенно нейтрализовать риск; такой портфель должен приносить безрисковую ставку, иначе возникает арбитраж.

Формула цены европейского опциона

Решение уравнения с граничным условием call-опциона $V(S, T) = \max(S - K, 0)$:

$$\begin{aligned} C &= S \cdot N(d_1) - K \cdot e^{(-r \cdot T)} \cdot N(d_2) \\ d_1 &= [\ln(S/K) + (r + \sigma^2/2) \cdot T] / (\sigma \cdot \sqrt{T}) \\ d_2 &= d_1 - \sigma \cdot \sqrt{T} \end{aligned}$$

Цена европейского put-опциона получается из паритета пут-колл:

$$P = K \cdot e^{(-r \cdot T)} \cdot N(-d_2) - S \cdot N(-d_1)$$

Параметры, от которых зависит цена опциона

- S — текущая цена базового актива;
- K — цена исполнения (страйк);
- T — срок до исполнения;
- r — безрисковая процентная ставка;
- σ — волатильность базового актива.

Все параметры, кроме σ , наблюдаемы напрямую. Волатильность σ оценивается по историческим данным или находится как **подразумеваемая** (implied) из рыночных цен опционов.

6. Модели индивидуального иска

ПРОСТЫМИ СЛОВАМИ

Выплата по одному полису = (случилось ли страховое событие) $\times$ (каков размер), и обе части — случайные величины.

Что такое индивидуальный иск

Индивидуальный иск — это случайная величина суммарной выплаты по ОДНОМУ договору страхования за период. Она случайна, потому что заранее неизвестно ни наступит ли страховой случай, ни каким будет ущерб.

Структура модели

Иск представляют произведением двух случайных величин:

$$S = I \cdot B$$

- I — индикатор страхового случая: равен 1, если случай наступил (с вероятностью q), и 0 иначе;
- B — размер выплаты при условии, что случай произошёл (случайная величина с своим распределением).

Числовые характеристики

Математическое ожидание и дисперсия иска вычисляются по правилам для произведения и через формулы полного ожидания:

$$E[S] = q \cdot E[B]$$

$$\text{Var}[S] = q \cdot E[B^2] - (q \cdot E[B])^2$$

Значение

Модель индивидуального иска — фундамент актуарной математики: поняв, как устроена случайность по одному договору, переходят к риску всего портфеля (см. вопросы 8 и 9).

ПРИМЕР

Договор: вероятность страхового случая $q = 0,05$; выплата при наступлении в среднем 200 тыс. руб. Ожидаемый иск $E[S] = 0,05 \cdot 200 = 10$ тыс. руб. — это базовая (нетто) часть страховой премии.

ЗАПОМНИ ГЛАВНОЕ

Запомни: иск = индикатор $\times$ размер; $E[S] = q \cdot E[B]$. Просят записать структуру модели и формулы математического ожидания.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Индивидуальный иск

Индивидуальный иск S — случайная величина суммарной выплаты по одному договору страхования за период действия. Случайность порождается двумя источниками: фактом наступления страхового случая и размером выплаты при его наступлении.

Структура модели

Иск представляют произведением двух независимых случайных величин:

$$S = I \cdot B,$$

I — индикатор страхового случая: $I = 1$ с вероятностью q ,
 $I = 0$ с вероятностью $1 - q$;
 B — размер выплаты при наступлении ($B \geq 0$).

Числовые характеристики

По правилу полного ожидания:

$$E[S] = q \cdot E[B]$$

$$E[S^2] = q \cdot E[B^2]$$

$$\text{Var}[S] = q \cdot E[B^2] - (q \cdot E[B])^2 = q \cdot \text{Var}[B] + q \cdot (1 - q) \cdot (E[B])^2$$

Расширения

- модель с несколькими видами страховых случаев: $S = \sum_k I_k \cdot B_k$;
- модель с количеством страховых случаев: $S = X_1 + X_2 + \dots + X_N$, где N — случайное число случаев (это уже коллективная модель);
- модели с зависимыми I и B (вероятность наступления зависит от потенциального ущерба);
- модели с ограничениями выплат (франшиза, лимит — см. следующий вопрос).

Модель индивидуального иска — фундамент актуарной математики: на её основе вычисляются нетто-премия, рисковая надбавка и характеристики портфеля договоров (см. вопрос 8).

7. Договоры страхования с вычетом, лимитом и франшизой

ПРОСТЫМИ СЛОВАМИ

Франшиза и лимит перекалывают часть риска на клиента, и это меняет распределение выплаты, а значит, и цену полиса.

Зачем нужны оговорки

Страховщик обычно возмещает не весь ущерб, а с условиями, которые делят риск между ним и клиентом. Это удешевляет полис и снижает поток мелких хлопотных выплат.

Франшиза (вычет)

Безусловная франшиза d : ущерб ниже d не возмещается совсем, а из ущерба выше d вычитается фиксированная сумма d . Выплата как функция ущерба X :

$$\text{Выплата} = \max(X - d, 0)$$

Условная франшиза: если ущерб ниже d — не платят, если выше — платят полностью.

Лимит ответственности

Лимит L — это потолок выплаты: даже если ущерб больше, страховщик заплатит не более L :

$$\text{Выплата} = \min(X, L)$$

Совместное действие

С франшизой и лимитом сразу выплата «обрезана» и снизу, и сверху:

$$\text{Выплата} = \min(\max(X - d, 0), L)$$

Задача актуария

Нужно пересчитать вероятностные характеристики выплаты — её среднее и дисперсию — уже с учётом этих обрезаний. Математически это работа с «усечённой» и «цензурированной» случайной величиной.

ПРИМЕР

Ущерб 500 тыс. руб., франшиза 50, лимит 400. Сначала вычитаем франшизу: $500 - 50 = 450$. Затем применяем лимит: $\min(450, 400) = 400$. Клиент получит 400 тыс. руб., остальное — его собственный риск.

ЗАПОМНИ ГЛАВНОЕ

Запомни: франшиза обрезает снизу, лимит — сверху. Просят записать формулу выплаты с обеими оговорками.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Условия страхового договора часто включают оговорки, перераспределяющие риск между страховщиком и страхователем. Они уменьшают премию и устраняют поток мелких выплат.

Безусловная франшиза (вычет)

Безусловная франшиза размером $d > 0$: страховщик платит сумму, на которую ущерб X превышает d ; если ущерб не превышает d , выплата нулевая.

$$Y = \max(X - d, 0) = (X - d)_+$$

Условная франшиза

Если ущерб не превышает порога d , выплата нулевая; если превышает — выплачивается полностью X . Условная франшиза удобна с операционной точки зрения, но создаёт «скачок» выплат у границы d .

Лимит ответственности

Лимит L — верхняя граница выплаты: страховщик платит $\min(X, L)$, даже если ущерб больше L .

$$Y = \min(X, L)$$

Совместное действие франшизы и лимита

$$Y = \min(\max(X - d, 0), L)$$

Вероятностные и числовые характеристики

Функция распределения и моменты выплаты Y зависят от исходного распределения ущерба X . Для безусловной франшизы:

$$\begin{aligned} E[Y] &= E[(X - d)_+] = \int_d^\infty (x - d) \cdot f_X(x) dx \\ &= \int_d^\infty (1 - F_X(x)) dx, \end{aligned}$$

где f_X и F_X — плотность и функция распределения ущерба. Аналогично выводятся формулы для лимита и комбинации франшизы и лимита.

8. Индивидуальные модели риска. Вероятность разорения. Премии

ПРОСТЫМИ СЛОВАМИ

Премия должна не просто покрывать средние выплаты, а оставлять запас, чтобы компания почти наверняка не разорилась.

Индивидуальная модель риска

Портфель страховой компании рассматривается как сумма независимых исков по каждому из n договоров:

$$S = S_1 + S_2 + \dots + S_n$$

Поскольку договоры независимы, математическое ожидание и дисперсия суммарных выплат складываются из вкладов отдельных договоров. Распределение S при больших n приближают нормальным (по центральной предельной теореме).

Вероятность разорения

Компания располагает средствами: собранными премиями плюс резервным капиталом u . Разорение — событие, когда суммарные выплаты превысят эти средства. Цель — чтобы вероятность разорения была мала (например, не больше 5%):

$$P(S > \text{премии} + u) \leq \varepsilon$$

Принципы назначения премий

Нетто-премия равна ожидаемой выплате $E[S]$. Но фактические выплаты колеблются, поэтому к нетто-премии добавляют рисковую надбавку. Основные принципы:

- принцип математического ожидания: премия = $(1 + \theta) \cdot E[S]$;
- принцип дисперсии: премия = $E[S] + \alpha \cdot \text{Var}[S]$;
- принцип стандартного отклонения: премия = $E[S] + \beta \cdot \sqrt{\text{Var}[S]}$.

Размер надбавки подбирают так, чтобы обеспечить заданную вероятность разорения.

ПРИМЕР

Ожидаемые выплаты по портфелю $E[S] = 1$ млн руб. Чтобы вероятность разорения не превысила 5%, по нормальной аппроксимации к нетто-премии добавляют надбавку порядка $1,645 \cdot \sqrt{\text{Var}[S]}$ — итоговая премия оказывается выше миллиона.

ЗАПОМНИ ГЛАВНОЕ

Запомни: премия = ожидаемые выплаты + рисковая надбавка; цель — малая вероятность разорения. Просят назвать принципы назначения премий.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Индивидуальная модель риска

Портфель страховой компании состоит из n независимых договоров. Случайная величина суммарных выплат за период:

$$S = S_1 + S_2 + \dots + S_n,$$

где S_i — индивидуальные иски. Из независимости договоров:

$$E[S] = \sum E[S_i], \quad \text{Var}[S] = \sum \text{Var}[S_i]$$

При большом числе однородных или мелких договоров по центральной предельной теореме S имеет приближённо нормальное распределение.

Вероятность разорения

Компания располагает средствами: собранными премиями Π и начальным резервным капиталом u . **Разорение** наступает, если суммарные выплаты превысят располагаемые средства:

$$\psi = P(S > \Pi + u)$$

Требование надёжности: $\psi \leq \varepsilon$ для заданного малого ε (например, $\varepsilon = 0,05$). По нормальной аппроксимации:

$$\Pi + u \geq E[S] + z_{(1-\varepsilon)} \cdot \sqrt{\text{Var}[S]},$$

где $z_{(1-\varepsilon)}$ — квантиль стандартного нормального распределения уровня $1 - \varepsilon$ (например, $z_{0,95} \approx 1,645$).

Принципы назначения страховых премий

Премия должна покрывать ожидаемые выплаты и обеспечивать запас на колебания (рисковая надбавка). Основные принципы:

- принцип эквивалентности: $\Pi = E[S]$ (нетто-премия, без надбавки);
- принцип математического ожидания: $\Pi = (1 + \theta) \cdot E[S]$, где $\theta > 0$ — относительная надбавка;
- принцип дисперсии: $\Pi = E[S] + \alpha \cdot \text{Var}[S]$;
- принцип стандартного отклонения: $\Pi = E[S] + \beta \cdot \sqrt{\text{Var}[S]}$;
- принцип нулевой полезности: премия определяется уравнением $u(W) = E[u(W + \Pi - S)]$ для функции полезности u ;
- принцип Эшера и экспоненциальный принцип (для тяжёлых хвостов).

9. Коллективные модели риска. Составное пуассоновское распределение

ПРОСТЫМИ СЛОВАМИ

Коллективная модель отделяет «как часто платим» от «сколько платим за раз», и их комбинация даёт распределение суммарного убытка.

Отличие от индивидуальной модели

Индивидуальная модель складывает иски по каждому договору. Коллективная смотрит на портфель в целом: суммарная выплата — это сумма СЛУЧАЙНОГО числа N страховых случаев, каждый из которых имеет случайный размер X :

$$S = X_1 + X_2 + \dots + X_N$$

Число случаев — распределение Пуассона

Количество страховых случаев N обычно моделируют распределением Пуассона с параметром λ — оно хорошо описывает число редких независимых событий за период.

Составное пуассоновское распределение

Если N распределено по Пуассону, а размеры выплат X независимы и одинаково распределены, то S имеет составное пуассоновское распределение. Его характеристики:

$$\begin{aligned} E[S] &= \lambda \cdot E[X] \\ \text{Var}[S] &= \lambda \cdot E[X^2] \end{aligned}$$

Преимущества коллективной модели

- проще считается, чем индивидуальная;
- хорошо описывает большие портфели;
- сумма независимых составных пуассоновских величин снова составная пуассоновская — удобно объединять портфели.

ПРИМЕР

За год ожидается в среднем $\lambda = 50$ страховых случаев, средняя выплата $E[X] = 30$ тыс. руб. Ожидаемые суммарные выплаты $E[S] = 50 \cdot 30 = 1500$ тыс. руб. Дисперсию считают через $E[X^2]$.

ЗАПОМНИ ГЛАВНОЕ

Запомни: S = сумма случайного числа выплат; $E[S] = \lambda \cdot E[X]$, $\text{Var}[S] = \lambda \cdot E[X^2]$. Простят отличить коллективную модель от индивидуальной.

Коллективная модель риска

В коллективной модели страховой портфель рассматривается как целое: суммарные выплаты — это сумма случайного числа страховых случаев со случайными размерами выплат:

$$S = X_1 + X_2 + \dots + X_N,$$

где N — случайное число страховых случаев за период, $X_1, X_2, \dots$ — независимые одинаково распределённые случайные величины размеров выплат, не зависящие от N .

Распределение Пуассона для числа случаев

Число случаев N часто моделируется распределением Пуассона с параметром $\lambda > 0$:

$$P(N = k) = (\lambda^k / k!) \cdot e^{(-\lambda)}, \quad k = 0, 1, 2, \dots$$

$$E[N] = \text{Var}[N] = \lambda$$

Параметр λ имеет смысл среднего ожидаемого числа страховых случаев за период.

Составное пуассоновское распределение

Если N распределена по Пуассону с параметром λ , а X_i независимы и одинаково распределены, то S имеет составное пуассоновское распределение. Его числовые характеристики:

$$E[S] = \lambda \cdot E[X]$$

$$\text{Var}[S] = \lambda \cdot E[X^2]$$

$$M_S(t) = \exp(\lambda \cdot (M_X(t) - 1)),$$

где $M_X(t)$ — производящая функция моментов X .

Свойства

- сумма независимых составных пуассоновских величин с параметрами $(\lambda_1, F_1), (\lambda_2, F_2)$ — снова составная пуассоновская с параметрами $(\lambda_1 + \lambda_2, \text{смешанное распределение})$;
- это свойство делает удобным агрегирование однотипных портфелей и страховых линий;
- коллективная модель проще считается и хорошо описывает большие портфели по сравнению с индивидуальной.

10. Эмпирическое распределение. Огива. Оценка Нельсона-Алена

ПРОСТЫМИ СЛОВАМИ

Распределение можно оценить прямо из данных, а для неполных (цензурированных) данных есть особые аккуратные методы.

Эмпирическое распределение

Когда есть не теоретическая модель, а только собранные наблюдения, по ним строят эмпирическую функцию распределения — оценку прямо из данных. Для несгруппированных данных это ступенчатая функция: в каждой точке она равна доле наблюдений, не превышающих эту точку.

Сгруппированные данные: огива и гистограмма

- **гистограмма** — столбиковая диаграмма частот по интервалам, показывает форму плотности распределения;
- **огива (кумулята)** — ломаная накопленных частот, приближённая эмпирическая функция распределения для сгруппированных данных.

Цензурированные данные

Цензурирование — это когда точное значение неизвестно, известно лишь, что оно больше некоторого порога. Пример из страхования: выплата превысила лимит полиса — фактический ущерб мы не видим, знаем только «больше лимита». Обычные формулы для таких данных не годятся.

Оценка Нельсона-Алена

Оценка Нельсона-Алена — метод оценки кумулятивной функции риска (интенсивности), корректно учитывающий цензурирование. В каждый момент наблюдаемого события t_i прибавляется отношение числа событий d_i к числу объектов n_i , ещё находящихся «под риском»:

$$\hat{H}(t) = \sum_{(t_i \leq t)} d_i / n_i$$

Родственная оценка функции выживания — оценка Каплана-Мейера (см. практическое задание 5).

ПРИМЕР

Выборка сроков службы деталей, где часть деталей ещё работает (наблюдение оборвано — цензурировано). Нельсон-Ален оценит накопленный риск отказа, не выбрасывая и не искажая «недожившие» наблюдения.

ЗАПОМНИ ГЛАВНОЕ

Запомни: огива — для сгруппированных данных, Нельсон-Ален — для цензурированных. Просят объяснить, что такое цензурирование.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Эмпирическое распределение

По выборке наблюдений $x_1, \dots, x_n$ строится эмпирическая функция распределения — оценка истинного распределения прямо из данных:

$$F_n(t) = (1/n) \cdot \#\{i: x_i \leq t\}$$

Это ступенчатая функция; по теореме Гливенко-Кантелли $F_n(t)$ равномерно сходится к истинной $F(t)$ при $n \rightarrow \infty$.

Сгруппированные данные

- **гистограмма** — столбиковая диаграмма частот по интервалам разбиения; даёт визуальное представление о форме плотности;
- **огива (кумулята)** — ломаная, соединяющая точки (верхняя_граница_интервала, накопленная_частота); приближённая эмпирическая функция распределения для сгруппированных данных.

Цензурирование данных

Цензурирование — ситуация, когда точное значение переменной неизвестно, известно лишь, что оно лежит в некотором интервале (чаще всего — больше определённого порога). Пример: страховая выплата, превысившая лимит полиса; срок жизни ещё живого пациента в момент окончания исследования.

Функция риска (интенсивность отказов)

Кумулятивная функция риска связана с функцией выживания:

$$H(t) = -\ln S(t), \quad S(t) = e^{-H(t)}, \\ h(t) = H'(t) = f(t) / S(t)$$

Оценка Нельсона-Алена

Оценка Нельсона-Алена строит кумулятивную функцию риска по цензурированным данным:

$$\hat{H}(t) = \sum_{(t_i \leq t)} d_i / n_i,$$

где t_i — моменты наблюдаемых событий (нецензурированных), d_i — число событий в момент t_i , n_i — число объектов «под риском» (с наблюдением $\geq t_i$). Цензурированные наблюдения не дают событий, но уменьшают n_i на следующих шагах. Родственная оценка функции выживания — оценка Каплана-Мейера: $\hat{S}(t) = \prod_{(t_i \leq t)} (1 - d_i/n_i)$.

11. Виды сетей: безмасштабные, тесного мира, фрактальные

ПРОСТЫМИ СЛОВАМИ

Реальные сети не случайны — у них есть характерные структуры, и тип структуры определяет их поведение.

Сеть (граф)

Сеть — это узлы (вершины) и связи (рёбра) между ними: соцсеть, интернет, дорожная сеть, нейроны мозга. Структуру сети характеризуют распределением степеней (число связей у узла), длиной путей, кластеризацией.

Безмасштабные сети

В безмасштабной сети у большинства узлов мало связей, но есть редкие узлы-концентраторы (хабы) с огромным числом связей. Распределение степеней подчиняется степенному закону:

$$P(k) \sim k^{-\gamma}$$

Такие сети формируются за счёт предпочтительного присоединения («богатые становятся богаче»). Они устойчивы к случайным сбоям, но уязвимы к целенаправленной атаке на хабы.

Сети тесного мира

В сети «тесного мира» от любого узла до любого другого ведёт короткая цепочка (эффект «шести рукопожатий»), и при этом сохраняется высокая локальная кластеризация — друзья ваших друзей часто знакомы между собой. Это сочетание коротких путей и плотных локальных связей.

Фрактальные сети

Фрактальная сеть самоподобна: её фрагмент по структуре напоминает всю сеть. Такие сети характеризуются масштабной инвариантностью — статистические свойства не меняются при изменении масштаба рассмотрения.

ПРИМЕР

Интернет и социальные сети — безмасштабные: есть суперпопулярные узлы-хабы. Поэтому случайный сбой сервера почти безвреден, а падение крупного хаба ощущается всеми.

ЗАПОМНИ ГЛАВНОЕ

Запомни: безмасштабная — хабы и степенной закон; тесный мир — короткие пути и кластеризация. Просят дать характеристику каждого типа.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Сеть (граф) $G = (V, E)$ задаётся множеством узлов V (вершин) и связей E (рёбер). Реальные сети редко бывают случайными — у них выраженная структура, которая влияет на поведение и устойчивость.

Безмасштабные сети

Сеть называется **безмасштабной** (scale-free), если распределение степеней узлов подчиняется степенному закону:

$$P(k) \sim k^{-(\gamma)}, \quad \text{обычно } 2 < \gamma < 3$$

Большинство узлов имеют малое число связей, но существуют редкие узлы-концентраторы (hubs) с большим числом связей. Механизм формирования — **предпочтительное присоединение** (модель Барабаши-Альберт): новый узел с большей вероятностью присоединяется к уже популярному узлу. Характерные примеры: интернет, социальные сети, цитирование. Особенности: высокая устойчивость к случайным сбоям, но уязвимость к целенаправленной атаке на хабы.

Сети тесного мира

Сеть **тесного мира** (small-world, модель Уоттса-Строгаца) сочетает два свойства:

- короткие пути: средняя длина кратчайшего пути растёт медленно — пропорционально $\log |V|$;
- высокий коэффициент кластеризации: соседи узла часто связаны между собой.

Феномен «шести рукопожатий» в социальных сетях — классическое проявление топологии тесного мира.

Фрактальные сети

Фрактальная сеть обладает свойством самоподобия: её статистические характеристики не меняются при изменении масштаба рассмотрения. Это проявляется в степенных законах связи между размером сети и числом коробок при box-counting. Фрактальная размерность характеризует геометрическую структуру сети.

12. Выявление сообществ в сетях. Модулярность

ПРОСТЫМИ СЛОВАМИ

Сообщество — это область сети с плотными внутренними связями, и его можно найти, оптимизируя модулярность.

Задача выявления сообществ

В больших сетях узлы сбиваются в группы — сообщества: внутри группы связей много, между группами мало (кружки по интересам в соцсети, тематические кластеры сайтов). Задача — найти такое разбиение автоматически, без заранее заданных меток.

Модулярность

Модулярность Q — числовая мера качества разбиения. Она показывает, насколько связей ВНУТРИ групп больше, чем было бы в случайной сети с теми же степенями узлов. Чем выше Q (ближе к 1), тем чётче выражены сообщества. Хорошее разбиение — то, что максимизирует Q .

Алгоритм Гирвана-Ньюмана

Действует «сверху вниз»: вычисляет, через какие рёбра проходит больше всего кратчайших путей (рёберная центральность), и удаляет такие рёбра-мосты. Сеть постепенно распадается на сообщества. Точный, но медленный для больших сетей.

Лувенский алгоритм

Действует «снизу вверх» и очень быстро: сначала каждый узел — своё сообщество, затем узлы жадно объединяются так, чтобы модулярность росла; полученные сообщества «схлопываются» в суперузлы и повторяют. Хорошо масштабируется на миллионы узлов.

Метод распространения меток

Каждый узел получает случайную метку, затем многократно перенимает метку, которая чаще всего встречается среди его соседей. Метки сами собой собираются в сообщества. Очень быстрый, но результат может слегка меняться от запуска к запуску.

ПРИМЕР

В соцсети алгоритм находит, что одни узлы плотно связаны между собой (одногруппники), другие — между собой (коллеги). Высокая модулярность подтверждает, что разбиение на эти сообщества осмысленно.

ЗАПОМНИ ГЛАВНОЕ

Запомни: модулярность измеряет качество разбиения; Гирван-Ньюман — сверху вниз, Лувен — снизу вверх. Просьба описать смысл модулярности и алгоритмы.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Задача выявления сообществ

Сообщество в сети — подмножество узлов с высокой плотностью связей внутри и низкой — наружу. Задача автоматического выявления сообществ относится к обучению без учителя: метки сообществ заранее не известны.

Модулярность

Модулярность Q — числовая мера качества разбиения сети на сообщества. Сравнивает фактическое число связей внутри сообществ с ожидаемым в случайной сети с теми же степенями узлов:

$$Q = (1 / 2m) \cdot \sum_{(i,j)} [A_{ij} - (k_i \cdot k_j) / (2m)] \cdot \delta(c_i, c_j),$$

где A_{ij} — матрица смежности, k_i — степень узла, m — общее число рёбер, c_i — сообщество узла i , δ — символ Кронекера. $Q \in (-0,5; 1]$; чем больше Q , тем более чётко выражены сообщества. Многие алгоритмы выявления сообществ оптимизируют именно Q .

Алгоритм Гирвана-Ньюмана

Действует «сверху вниз»:

- для всех рёбер вычисляется их **рёберная центральность** (число кратчайших путей, проходящих через ребро);
- удаляется ребро с наибольшей центральностью;
- после удаления центральности пересчитываются;
- процедура повторяется, сеть постепенно распадается на сообщества.

Преимущества: высокая точность. Недостатки: высокая вычислительная сложность $O(m^2n)$, плохо масштабируется на большие сети.

Лувенский алгоритм (Louvain method)

Действует «снизу вверх» — жадная оптимизация модулярности:

- изначально каждый узел — отдельное сообщество;
- для каждого узла рассматривается перенос в сообщество одного из его соседей; перенос выполняется, если это увеличивает модулярность; перебор узлов повторяется до локального максимума;
- сообщества «схлопываются» в суперузлы, и процедура повторяется на новой сети.

Высокая скорость и хорошее качество позволяют применять алгоритм к сетям из миллионов узлов.

Метод распространения меток (Label Propagation)

Каждый узел получает уникальную метку; затем на каждой итерации узел принимает метку, которая чаще всего встречается среди его соседей (при равенстве — случайный выбор). Процедура повторяется до стабилизации. Очень быстр (почти линейен по числу рёбер), но результат может несколько меняться от запуска к запуску из-за случайных разрешений равенств.

13. Типы задач машинного обучения

ПРОСТЫМИ СЛОВАМИ

Обучение модели = поиск параметров, при которых ошибка на данных наименьшая.

Что такое машинное обучение

Машинное обучение — создание программ, которые не следуют жёстко прописанным правилам, а извлекают закономерности из данных и применяют их к новым случаям.

Обучение с учителем

В данных есть «правильные ответы» (метки), и модель учится их предсказывать. Два главных вида:

- классификация — предсказание категории (спам/не спам, болезнь/здоров);
- регрессия — предсказание числа (цена квартиры, спрос, температура).

Обучение без учителя

Правильных ответов нет, модель сама ищет структуру в данных:

- кластеризация — разбиение объектов на группы похожих;
- снижение размерности — сжатие данных с сохранением сути;
- поиск ассоциативных правил.

Отдельно выделяют обучение с подкреплением, где агент учится действиям через систему наград.

Обучение как оптимизация

Любое обучение сводится к оптимизации: задают функцию потерь (ошибки), измеряющую расхождение предсказаний с реальностью, и алгоритм подбирает параметры модели, минимизируя эту функцию.

$$\text{параметры}^* = \operatorname{argmin} L(\text{параметры}; \text{данные})$$

ПРИМЕР

Предсказать цену квартиры по площади — регрессия (ответ число). Определить, мошенническая транзакция или нет — классификация (ответ категория). Разбить клиентов на сегменты без меток — кластеризация.

ЗАПОМНИ ГЛАВНОЕ

Запомни: с учителем = есть метки (классификация/регрессия), без учителя = нет меток (кластеризация). Просят привести классификацию задач и примеры.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Общая постановка

Машинное обучение — раздел искусственного интеллекта, изучающий алгоритмы, способные улучшать своё качество автоматически на основании опыта (данных). Формально задача обучения сводится к подбору параметров θ модели f_θ , минимизирующих функцию потерь на обучающих данных:

$$\theta^* = \operatorname{argmin}_\theta \sum_{i=1}^N L(f_\theta(x_i), y_i) + \Omega(\theta),$$

где L — функция потерь, Ω — регуляризатор (штраф за сложность модели).

Типы задач машинного обучения

Обучение с учителем (supervised learning): в обучающей выборке для каждого объекта известна метка y . Два главных вида:

- **классификация** — y дискретная переменная: $y \in \{1, \dots, K\}$ (бинарная — два класса, многоклассовая — более двух, многометочная — несколько меток одновременно);
- **регрессия** — y непрерывная переменная: $y \in \mathbb{R}$.

Обучение без учителя (unsupervised learning): метки отсутствуют, модель ищет структуру в данных:

- **кластеризация** — разбиение объектов на группы похожих;
- снижение размерности (PCA, t-SNE, автоэнкодеры);
- поиск ассоциативных правил;
- оценка плотности распределения данных.

Обучение с подкреплением (reinforcement learning): агент взаимодействует со средой, получая награды; задача — научиться политике действий, максимизирующей суммарную награду.

Полуконтролируемое обучение сочетает размеченные и неразмеченные данные; **самообучение (self-supervised)** создаёт собственные «метки» из структуры данных.

Машинное обучение как оптимизация

Любая задача обучения сводится к оптимизационной: задают функцию потерь (ошибку), измеряющую расхождение предсказаний с истиной, и подбирают параметры модели, минимизируя её. Основной инструмент — градиентные методы (см. вопрос 23 части 1). Качество модели оценивают на отдельной тестовой выборке для контроля переобучения.

14. Классификация как задача машинного обучения

ПРОСТЫМИ СЛОВАМИ

Классификатор учится на размеченных примерах, минимизируя свои ошибки, чтобы затем верно распознавать новые объекты.

Постановка задачи

Классификация — предсказание категории (класса) объекта по его признакам. Каждый объект описан вектором чисел-признаков и относится к одному из конечного набора классов.

Математическая формализация

Есть обучающая выборка — пары «объект x , класс y ». Нужно найти функцию (классификатор) $a(x)$, которая по признакам выдаёт класс. Качество измеряют функцией потерь $L(a(x), y)$ — штрафом за несовпадение предсказания с истиной.

$$a^* = \operatorname{argmin} \sum_i L(a(x_i), y_i)$$

Различают бинарную классификацию (два класса), многоклассовую и многометочную (объект может принадлежать нескольким классам сразу).

Целевая функция

Часто модель выдаёт не сам класс, а вероятности классов; тогда потери измеряют перекрёстной энтропией (логистические потери). Минимизация потерь и есть процесс обучения.

Примеры моделей классификации

- логистическая регрессия — линейная граница, выдаёт вероятности;
- дерево решений — последовательность вопросов «да/нет»;
- метод опорных векторов (SVM) — максимизирует зазор между классами;
- случайный лес и градиентный бустинг — ансамбли деревьев;
- нейронные сети.

ПРИМЕР

Фильтр спама: признаки письма — частоты слов, наличие ссылок, адрес отправителя. Модель обучается на тысячах писем, помеченных «спам»/«не спам», и затем классифицирует новые входящие письма.

ЗАПОМНИ ГЛАВНОЕ

Запомни: классификация = предсказание категории; обучение = минимизация функции потерь. Просьба формализовать задачу и привести примеры моделей.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Постановка задачи классификации

Задача классификации — задача обучения с учителем, в которой требуется предсказать категорию (класс) объекта по его признакам.

Математическая формализация

Заданы пространство объектов $X \subseteq \mathbb{R}^d$ и конечное множество классов $Y = \{1, 2, \dots, K\}$. Обучающая выборка состоит из независимо порождённых одинаково распределённых пар:

$$\{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}, \quad x_i \in X, \quad y_i \in Y$$

Требуется построить функцию-классификатор $a: X \rightarrow Y$, минимизирующую вероятность ошибки на новых объектах:

$$\text{Risk}(a) = P(a(X) \neq Y),$$

оцениваемую через эмпирический риск на обучающей выборке.

Целевая функция (функция потерь)

Качество классификации измеряется функцией потерь $L(a(x), y)$. Типичные виды потерь:

- 0-1 потери: $L = [a(x) \neq y]$ — индикатор ошибки;
- логистические (перекрёстная энтропия): $L = - \sum_k y_k \cdot \log \hat{p}_k$ — используется, когда модель выдаёт вероятности классов;
- hinge loss: $L = \max(0, 1 - y \cdot f(x))$ — для метода опорных векторов (SVM).

Эмпирический риск

$$\hat{R}(\theta) = (1/N) \cdot \sum_{i=1}^N L(a_\theta(x_i), y_i) \rightarrow \min_\theta$$

Часто к $\hat{R}$ добавляют регуляризатор $R(\theta) + \lambda \cdot \|\theta\|^2$ — штраф за сложность, предотвращающий переобучение.

Основные модели классификации

- логистическая регрессия — линейный классификатор, выдающий вероятности;
- метод опорных векторов (SVM) — максимизирует зазор между классами;
- наивный байесовский классификатор — основан на формуле Байеса с предположением условной независимости признаков;
- k ближайших соседей (kNN) — голосование ближайших соседей;
- дерево решений и его ансамбли (случайный лес, градиентный бустинг XGBoost, LightGBM, CatBoost);
- нейронные сети.

Пример: оптимальный байесовский классификатор

При известных условных распределениях $p(x | y)$ и априорных $p(y)$ минимум вероятности ошибки достигается на правиле Байеса:

$$a^*(x) = \operatorname{argmax}_{(y \in Y)} p(x | y) \cdot p(y)$$

15. Метрики качества. Недо- и переобучение

ПРОСТЫМИ СЛОВАМИ

Цель — модель, которая хорошо работает на новых данных, а не та, что выучила обучающие наизусть.

Метрики для регрессии

$$\begin{aligned} \text{MAE} &= (1/n) \cdot \sum |y_i - \hat{y}_i| && \text{— средняя абсолютная ошибка} \\ \text{MSE} &= (1/n) \cdot \sum (y_i - \hat{y}_i)^2 && \text{— средний квадрат ошибки} \end{aligned}$$

MSE сильнее штрафует крупные промахи; MAE устойчивее к выбросам.

Метрики для классификации

Точность (ассурасу) — доля верных ответов — обманчива на несбалансированных данных. Информативнее метрики на основе матрицы ошибок (TP, FP, FN, TN):

$$\begin{aligned} \text{Precision} &= \text{TP} / (\text{TP} + \text{FP}) && \text{— точность} \\ \text{Recall} &= \text{TP} / (\text{TP} + \text{FN}) && \text{— полнота} \\ \text{F1} &= 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall}) \end{aligned}$$

- Precision — из помеченных «да» сколько реально «да» (цена ложной тревоги);
- Recall — из всех настоящих «да» сколько модель поймала (цена пропуска);
- F1 — гармоническое среднее, баланс точности и полноты;
- ROC-AUC — качество модели по всем порогам сразу, площадь под ROC-кривой; 0,5 — случайное угадывание, 1 — идеал.

Недообучение и переобучение

- недообучение: модель слишком проста, плохо работает и на обучающих, и на новых данных (высокое смещение);
- переобучение: модель «зазубрила» обучающие данные вместе с шумом, отлично работает на них, но проваливается на новых (высокий разброс).

Кривые обучения

Кривые обучения — графики ошибки на обучающей и проверочной выборках в зависимости от объёма данных или сложности модели. Большой разрыв между кривыми — признак переобучения; обе ошибки высоки — недообучение.

ПРИМЕР

Диагностика редкой болезни (1% больных). Модель «всем здоров» даёт ассурасу 99%, но Recall = 0 — не поймала ни одного больного. Здесь релевантна именно полнота (Recall), а не точность.

ЗАПОМНИ ГЛАВНОЕ

Запомни: Precision — про ложные тревоги, Recall — про пропуски, F1 — баланс. Просят формулы и объяснение недо-/переобучения по кривым.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Метрики качества регрессии

Для задачи регрессии $y \in \mathbb{R}$:

$$\begin{aligned} \text{MAE} &= (1/n) \cdot \sum |y_i - \hat{y}_i| && \text{— средняя абс. ошибка} \\ \text{MSE} &= (1/n) \cdot \sum (y_i - \hat{y}_i)^2 && \text{— средний квадрат ошибки} \\ \text{RMSE} &= \sqrt{\text{MSE}} && \text{— корень из MSE} \\ \text{MAPE} &= (1/n) \cdot \sum |y_i - \hat{y}_i| / |y_i| && \text{— средняя абс. в процентах} \\ R^2 &= 1 - \sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2 \end{aligned}$$

MSE сильнее штрафует крупные промахи; MAE устойчивее к выбросам.

Метрики качества классификации

Для бинарной классификации различают четыре исхода (**матрица ошибок**):

- TP (true positive) — истинно положительные;
- FP (false positive) — ложно положительные (ложная тревога);
- FN (false negative) — ложно отрицательные (пропуск);
- TN (true negative) — истинно отрицательные.

$$\begin{aligned} \text{Accuracy} &= (TP + TN) / (TP + TN + FP + FN) \\ \text{Precision} &= TP / (TP + FP) && \text{— точность} \\ \text{Recall} &= TP / (TP + FN) && \text{— полнота} \\ \text{F1} &= 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall}) \end{aligned}$$

Accuracy обманчива на несбалансированных данных; Precision и Recall раскрывают её. F1 — гармоническое среднее Precision и Recall.

ROC-кривая и AUC

ROC-кривая (Receiver Operating Characteristic) — график зависимости TPR (= Recall) от FPR = FP/(FP + TN) при варьировании порога классификации. **ROC-AUC** — площадь под ROC-кривой, общая мера качества по всем порогам: 0,5 — случайное угадывание, 1 — идеальная классификация. Аналог для несбалансированных задач — PR-AUC (площадь под Precision-Recall кривой).

Многоклассовое усреднение

- macro-усреднение — простое среднее метрик по классам (все классы важны одинаково);
- weighted-усреднение — взвешенное среднее с весами по числу объектов класса (support);
- micro-усреднение — суммирование TP, FP, FN по всем классам с последующим расчётом.

Недообучение и переобучение

- **недообучение** (underfitting): высокий bias — модель слишком проста, плохо работает и на обучающих, и на тестовых данных;
- **переобучение** (overfitting): высокий variance — модель «зазубрила» обучающие данные вместе с шумом, отлично работает на них, проваливается на новых.

Кривые обучения — графики ошибки на обучающей и валидационной выборках в зависимости от размера обучающей выборки или сложности модели. Большой разрыв между ними — признак переобучения; обе ошибки высоки — недообучение. Для контроля используют отложенную выборку, кросс-валидацию, регуляризацию, раннюю остановку обучения.

16. Кластеризация. Метод ближайших соседей (kNN)

ПРОСТЫМИ СЛОВАМИ

kNN не строит сложную модель — он классифицирует новый объект «по большинству» среди ближайших к нему.

Кластеризация

Кластеризация — обучение без учителя: разбить объекты на группы (кластеры) похожих, когда меток нет. Похожесть определяют через расстояние (евклидово, манхэттенское и др.). Популярные методы — k-средних, иерархическая кластеризация, DBSCAN.

Метод k ближайших соседей (kNN)

kNN — метрический метод. Чтобы классифицировать новый объект, находят k ближайших к нему объектов обучающей выборки и относят новый объект к классу, который преобладает среди этих соседей (голосование большинством). Для регрессии берут среднее значение соседей.

Важная особенность: kNN — «ленивый» метод, он не строит модель заранее, а просто хранит все данные и вычисляет расстояния в момент запроса.

Подбор числа k

Параметр k критичен: слишком маленькое k (например, 1) делает модель чувствительной к шуму и выбросам; слишком большое — сглаживает и теряет детали границ. Оптимальное k подбирают по качеству на отложенной (проверочной) выборке или кросс-валидацией.

Реализация в Scikit-Learn

В библиотеке Scikit-Learn методы метрической классификации реализованы классами KNeighborsClassifier и KNeighborsRegressor. Важно предварительно масштабировать признаки — иначе признак с крупными значениями исказит расстояние.

ПРИМЕР

Новый клиент банка. Берём $k = 5$ ближайших по доходу и возрасту клиентов. Если 4 из 5 оказались надёжными заёмщиками — нового тоже относим к надёжным.

ЗАПОМНИ ГЛАВНОЕ

Запомни: kNN — голосование ближайших соседей; k подбирают по валидации; признаки нужно масштабировать. Попросят объяснить выбор k.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Кластеризация как задача машинного обучения

Кластеризация — задача обучения без учителя: разбить множество объектов на группы (кластеры) таким образом, чтобы внутри кластера объекты были похожи (по выбранной метрике), а между кластерами — различны.

Меры близости

- евклидово расстояние: $d(x, y) = \sqrt{\sum (x_i - y_i)^2}$;
- манхэттенское: $d(x, y) = \sum |x_i - y_i|$;
- косинусное расстояние: $1 - (x \cdot y) / (\|x\| \cdot \|y\|)$;
- расстояние Махаланобиса с учётом ковариационной матрицы.

Основные алгоритмы кластеризации

- **к-средних** (k-means): итеративно минимизирует суммарное квадратичное отклонение объектов от центров кластеров;
- **иерархическая кластеризация**: агломеративная (снизу вверх) или дивизивная (сверху вниз), строит дендрограмму;
- **DBSCAN** — плотностный алгоритм, находит кластеры произвольной формы и автоматически определяет их число, выделяет шумовые точки;
- **спектральная кластеризация** на собственных векторах матрицы графа.

Метод k ближайших соседей (kNN)

kNN — метрический метод классификации и регрессии. Для нового объекта x находится k ближайших объектов обучающей выборки по выбранной метрике; в задаче классификации x относится к классу, который преобладает среди этих k соседей (голосование большинством); в регрессии — среднее значений их меток.

kNN относится к **ленивым (lazy)** алгоритмам — он не обучает модель, а просто хранит данные и выполняет вычисления в момент запроса. Сложность предсказания одного объекта в наивной реализации $O(N \cdot d)$; для ускорения применяют KD-деревья и Ball-деревья.

Обобщения kNN

- взвешенный kNN — вклад соседей пропорционален обратному расстоянию;
- kNN с обучаемой метрикой (metric learning);
- Parzen-окно — частный случай взвешенного kNN с ядром-плотностью.

Подбор параметра k

Слишком маленькое k делает классификатор чувствительным к шуму (переобучение); слишком большое — сглаживает границы классов и теряет локальные детали (недообучение). Оптимум подбирается по кросс-валидации.

Реализация в scikit-learn

В библиотеке `scikit-learn` доступны `KNeighborsClassifier` и `KNeighborsRegressor` с параметрами `n_neighbors`, `weights`, `metric`, `algorithm`. Перед применением рекомендуется масштабировать признаки (`StandardScaler` или `MinMaxScaler`), иначе признаки с большими значениями исказят расстояние.

17. Обучение нейросетей. Обратное распространение ошибки

ПРОСТЫМИ СЛОВАМИ

Backpropagation эффективно вычисляет, как подкрутить каждый вес, чтобы уменьшить общую ошибку.

Многослойная нейронная сеть

Нейросеть — цепочка слоёв. Каждый нейрон считает взвешенную сумму входов, прибавляет смещение и пропускает результат через нелинейную функцию активации (ReLU, сигмоида). Настраиваемые параметры — веса и смещения, их могут быть миллионы.

Задача обучения

Обучить сеть — подобрать веса так, чтобы функция потерь была минимальной. Делается это градиентным спуском. Но возникает вопрос: как для каждого из миллионов весов узнать его вклад в общую ошибку (частную производную)?

Алгоритм обратного распространения ошибки

Backpropagation решает это в два прохода:

- прямой проход — данные идут через сеть, вычисляется предсказание и значение ошибки;
- обратный проход — ошибку «пускают назад» по сети, и с помощью правила дифференцирования сложной функции (цепного правила) для каждого веса вычисляют градиент.

правило цепи: $dL/dw = (dL/dy) \cdot (dy/dz) \cdot (dz/dw)$

Затем веса корректируют шагом градиентного спуска. Алгоритм эффективен: он переиспользует уже посчитанные производные, не пересчитывая их заново для каждого веса.

Современный взгляд

Сеть представляют как граф потока вычислений. Дифференцируемое программирование (автоматическое дифференцирование, реализованное в PyTorch и TensorFlow) автоматически считает все производные по этому графу — программисту не нужно выводить их вручную.

ПРИМЕР

Сеть предсказала 0,9, верный ответ — 0. Ошибка большая. Обратный проход распределяет «вину» за эту ошибку по всем весам слой за слоем, и шаг градиентного спуска чуть подправляет каждый вес в нужную сторону.

ЗАПОМНИ ГЛАВНОЕ

Запомни: прямой проход → ошибка, обратный проход → градиенты по цепному правилу. Просят описать два прохода и роль цепного правила.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Многослойная нейронная сеть

Полносвязная нейронная сеть прямого распространения — композиция слоёв линейных преобразований и нелинейных активаций. Для слоя l :

$$\begin{aligned}z^{\wedge}(\square) &= W^{\wedge}(\square) \cdot a^{\wedge}(\square-1) + b^{\wedge}(\square) \\a^{\wedge}(\square) &= \sigma(z^{\wedge}(\square))\end{aligned}$$

Параметры сети — веса $W^{\wedge}(l)$ и смещения $b^{\wedge}(l)$; типичные активации σ — ReLU, сигмоида, гиперболический тангенс, GELU. На последнем слое классификации применяют softmax.

Функция потерь и градиентный спуск

Обучение состоит в минимизации функции потерь $L(\theta)$ (перекрёстная энтропия для классификации, MSE для регрессии) методом стохастического градиентного спуска (SGD) или его вариантов (Momentum, Adam, RMSProp):

$$\theta_{t+1} = \theta_t - \alpha \cdot \nabla L(\theta_t; \text{mini-batch})$$

Алгоритм обратного распространения ошибки (backpropagation)

Эффективно вычисляет градиент ∇L по всем параметрам сети с помощью цепного правила дифференцирования сложной функции. Работает в два прохода:

- **прямой проход:** входные данные распространяются от первого слоя к последнему, на каждом слое сохраняются промежуточные активации $a^{\wedge}(l)$ и предактивации $z^{\wedge}(l)$; вычисляется итоговое предсказание и значение потерь L ;
- **обратный проход:** вычисляются производные $\delta^{\wedge}(l) = \partial L / \partial z^{\wedge}(l)$ рекуррентно от последнего слоя к первому по правилу

$$\begin{aligned}\delta^{\wedge}(L) &= \nabla_{a^{\wedge}(L)} L \cdot \sigma'(z^{\wedge}(L)) \\ \delta^{\wedge}(\square) &= (W^{\wedge}(\square+1))^{\top} \cdot \delta^{\wedge}(\square+1) \odot \sigma'(z^{\wedge}(\square))\end{aligned}$$

Градиенты по параметрам:

$$\begin{aligned}\partial L / \partial W^{\wedge}(\square) &= \delta^{\wedge}(\square) \cdot (a^{\wedge}(\square-1))^{\top} \\ \partial L / \partial b^{\wedge}(\square) &= \delta^{\wedge}(\square)\end{aligned}$$

Граф потока вычислений и дифференцируемое программирование

В современных фреймворках (PyTorch, TensorFlow, JAX) сеть представляется как **граф потока вычислений** — ориентированный граф, узлы которого — операции, а рёбра — поток тензоров. Автоматическое дифференцирование (autograd) обходит этот граф в обратном порядке и вычисляет градиенты по всем входам без необходимости выводить формулы вручную — это парадигма **дифференцируемого программирования**.

18. Свёрточные нейронные сети (CNN)

ПРОСТЫМИ СЛОВАМИ

CNN учитывают, что в картинке важны локальные узоры, и за счёт повторного использования фильтров эффективно их распознают.

Проблема обычных сетей на изображениях

Изображение — огромная таблица пикселей. Если подать его в обычную полносвязную сеть, число параметров будет неподъёмным, и сеть не учтёт, что соседние пиксели связаны, а объект может находиться в любом месте кадра.

Операция свёртки

Свёртка — это маленький фильтр (ядро, окно, например 3×3), который скользит по всему изображению. В каждом положении он вычисляет взвешенную сумму пикселей под собой и обнаруживает локальный признак: край, угол, пятно цвета, текстуру.

Два ключевых принципа

- разделяемые веса: один и тот же фильтр применяется ко всем участкам изображения — это резко сокращает число параметров;
- локальность: нейрон смотрит не на всё изображение, а на небольшую область (рецептивное поле).

Слои подвыборки (pooling)

Слой пулинга (например, max-pooling) уменьшает размер карты признаков, оставляя самое сильное значение в каждой области. Это снижает вычисления и даёт устойчивость к небольшим сдвигам.

Иерархия признаков

Слои выстраиваются иерархически: первые свёрточные слои распознают простые края, следующие — формы и части объектов, глубокие — целые объекты. Главное преимущество CNN — инвариантность к положению: признак распознаётся, где бы он ни находился в кадре.

ПРИМЕР

Распознавание кошки на фото. Первые слои находят края и усы, средние — глаза и уши, глубокие собирают это в «морду кошки». Кошка узнаётся в любом углу кадра — благодаря скользящим фильтрам.

ЗАПОМНИ ГЛАВНОЕ

Запомни: свёртка = скользящий фильтр, разделяемые веса, инвариантность к сдвигу; pooling уменьшает размер. Просьба объяснить, почему CNN лучше обычных сетей для изображений.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Специфика задач машинного обучения на изображениях

Изображение представляется тензором размера $H \times W \times C$ (высота, ширина, число каналов). Для среднего фото это сотни тысяч пикселей. Прямое применение полносвязной сети невозможно по двум причинам:

- огромное число параметров (на каждый пиксель — отдельный вес) ведёт к катастрофическому переобучению;
- полносвязная сеть не учитывает локальную структуру изображения (соседние пиксели связаны больше, чем удалённые);
- она не обладает инвариантностью к сдвигу — объект, сместившийся в кадре, выглядит для сети совершенно по-другому.

Операция свёртки

Свёртка применяет небольшое ядро (фильтр) размера $k \times k$, скользящее по всему изображению. В каждой позиции вычисляется взвешенная сумма пикселей под ядром плюс смещение:

$$y(i, j) = \sum_{(u, v)} K(u, v) \cdot x(i + u, j + v) + b$$

Параметры: размер ядра, шаг (stride), отступы (padding), число фильтров. Каждый фильтр обнаруживает определённый локальный признак.

Ключевые принципы CNN

- **разделяемые веса (weight sharing):** один и тот же фильтр применяется ко всем позициям изображения. Резко сокращает число параметров и обеспечивает инвариантность к сдвигу;
- **локальная связность:** нейрон смотрит лишь на небольшую область — рецептивное поле, а не на всё изображение;
- **иерархия признаков:** через стек свёрточных слоёв сеть учится представлениям от простых (края, углы) к сложным (части объектов, целые объекты).

Слои подвыборки (pooling)

Слой pooling уменьшает пространственное разрешение карты признаков, оставляя характерное значение в каждой области:

- max-pooling — максимум по окну (популярнее);
- average-pooling — среднее по окну;
- global pooling — по всей карте, сводит к вектору.

Pooling даёт инвариантность к мелким сдвигам и сокращает вычисления.

Архитектура CNN

Типичная сеть: $[\text{Conv} \rightarrow \text{активация} \rightarrow \text{Pool}] \times n \rightarrow \text{Flatten} \rightarrow \text{FC} \rightarrow \text{softmax}$. Современные архитектуры — VGG, ResNet (с skip-connections), Inception, EfficientNet, Vision Transformers.

Преимущества CNN на изображениях

- параметров на порядки меньше, чем у полносвязной сети;
- сеть автоматически обучается полезным признакам, без ручного инженерного конструирования (feature engineering);
- инвариантность к сдвигу: объект распознаётся независимо от его положения в кадре.

19. Рекуррентные нейронные сети. Модуль LSTM

ПРОСТЫМИ СЛОВАМИ

RNN дают сети память о прошлом, а LSTM делает эту память долгой и управляемой.

Зачем нужны рекуррентные сети

Обычная сеть обрабатывает каждый объект независимо. Но для текста, речи, временных рядов важен порядок и контекст. Рекуррентная сеть (RNN) обрабатывает последовательность элемент за элементом, передавая на каждый шаг скрытое состояние — «память» о предыдущих элементах.

$$h_t = f(W \cdot x_t + U \cdot h_{t-1} + b)$$

Скрытое состояние h_t зависит и от текущего входа x_t , и от состояния на прошлом шаге $h_{(t-1)}$.

Проблема обучения RNN

Простые RNN обучают вариантом backpropagation «во времени». На длинных последовательностях возникает проблема затухающего (или взрывающегося) градиента: сигнал ошибки, проходя через много шагов, экспоненциально ослабевает или растёт. Из-за этого сеть «забывает» далёкий контекст и не улавливает длинные зависимости.

Модуль LSTM

LSTM (долгая краткосрочная память) решает проблему особой конструкцией. Помимо скрытого состояния LSTM хранит состояние ячейки — «магистраль памяти», по которой информация течёт почти без искажений. Управляют ею вентили (гейты):

- вентиль забывания решает, что из памяти стереть;
- входной вентиль решает, какую новую информацию записать;
- выходной вентиль решает, что из памяти выдать наружу.

Благодаря вентилям LSTM удерживает важную информацию на десятки и сотни шагов. Похожая, но более простая архитектура — GRU.

ПРИМЕР

Перевод предложения: чтобы правильно согласовать слово в конце, сеть должна помнить подлежащее из начала. Простая RNN это «забудет», LSTM — удержит благодаря состоянию ячейки и вентилям.

ЗАПОМНИ ГЛАВНОЕ

Запомни: RNN = память через скрытое состояние; беда — затухающий градиент; LSTM лечит её вентилями и состоянием ячейки. Просят назвать вентили.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Рекуррентные нейронные сети (RNN)

RNN обрабатывают последовательность $x_1, x_2, \dots, x_T$ элемент за элементом, поддерживая скрытое состояние (память) h_t , передающееся через шаги:

$$\begin{aligned} h_t &= \sigma(W_x \cdot x_t + W_h \cdot h_{(t-1)} + b), \\ y_t &= g(W_y \cdot h_t + b_y) \end{aligned}$$

Состояние h_t резюмирует всю предыдущую историю последовательности до момента t .

Принципы обучения

RNN обучают алгоритмом **backpropagation through time (BPTT)**: последовательность «разворачивают» во времени и применяют обычный backpropagation к получившейся глубокой сети, веса которой повторяются на каждом шаге.

Сложности применения

- **затухающий градиент** (vanishing gradient): при распространении ошибки через много шагов градиент экспоненциально убывает — сеть «забывает» далёкий контекст и не улавливает длинные зависимости;
- **взрывающийся градиент**: при некоторых весах градиент экспоненциально растёт; борются обрезанием градиента (gradient clipping);

- трудности параллелизации обучения: шаги последовательно зависят друг от друга;
- конечная пропускная способность скрытого состояния ограничивает объём запоминаемой информации.

Модуль LSTM (Long Short-Term Memory)

LSTM решает проблему затухающего градиента за счёт особой архитектуры с состоянием ячейки C_t и тремя обучаемыми **вентильми (gates)**:

- **вентиль забывания** $f_t = \sigma(W_f \cdot [h_{(t-1)}, x_t] + b_f)$ — решает, какую часть прошлой памяти стереть;
- **входной вентиль** $i_t = \sigma(W_i \cdot [h_{(t-1)}, x_t] + b_i)$ — решает, какую новую информацию записать;
- **выходной вентиль** $o_t = \sigma(W_o \cdot [h_{(t-1)}, x_t] + b_o)$ — решает, какую часть памяти выдать наружу.

Обновление ячейки памяти и скрытого состояния:

```
C~_t = tanh(W_C · [h_(t-1), x_t] + b_C)
C_t = f_t ⊙ C_(t-1) + i_t ⊙ C~_t
h_t = o_t ⊙ tanh(C_t)
```

Состояние ячейки C_t — «магистраль» памяти, по которой информация течёт с минимальными искажениями: благодаря линейности обновления градиент не затухает. LSTM удерживает контекст на сотни шагов. Упрощённый вариант — **GRU** (Gated Recurrent Unit) с двумя вентильми.

Современные альтернативы RNN — **трансформеры** (Transformer) с механизмом внимания (attention), не требующие последовательной обработки.

20. Обработка текста. Модель Word2vec

ПРОСТЫМИ СЛОВАМИ

Word2vec кодирует смысл слова компактным вектором, выученным из контекстов его употребления.

Проблема представления текста

Компьютер работает с числами. Чтобы применять машинное обучение к тексту, слова нужно превратить в числовые векторы. Наивный способ (один бит на слово, «one-hot») даёт огромные разреженные векторы, в которых все слова одинаково «далеки» друг от друга — смысл не отражён.

Дистрибутивная семантика

Ключевая идея: смысл слова определяется его окружением. Слова, встречающиеся в похожих контекстах, близки по значению. «Кошка» и «собака» часто стоят рядом со словами «корм», «питомец», «хозяин» — значит, они семантически близки.

Матрица совместной встречаемости

Ранний подход: построить большую матрицу, где для каждой пары слов указано, как часто они встречаются рядом. Минус — матрица огромна и разрежена.

Модель Word2vec

Word2vec представляет каждое слово коротким плотным вектором (например, 300 чисел). Векторы обучает неглубокая нейросеть в одном из двух режимов:

- CBOW — предсказывает слово по его соседям;

- Skip-gram — предсказывает соседей по слову.

В обученном пространстве близкие по смыслу слова оказываются рядом, и работает векторная арифметика смыслов:

вектор(король) – вектор(мужчина) + вектор(женщина) $\approx$ вектор(королева)

ПРИМЕР

После обучения Word2vec слова «Москва» и «Париж» оказываются рядом (оба — столицы), а разность «Париж» – «Франция» примерно равна «Москва» – «Россия». Так в числах закодированы отношения между понятиями.

ЗАПОМНИ ГЛАВНОЕ

Запомни: дистрибутивная гипотеза, плотные векторы, режимы CBOW и Skip-gram. Просят объяснить идею «смысл из контекста».

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Задачи обработки текста

Компьютер работает с числами, а текст требует векторного представления. Базовые задачи NLP: классификация документов, анализ тональности, распознавание именованных сущностей, машинный перевод, генерация текста.

Дистрибутивная семантика

Дистрибутивная гипотеза (Z. Harris, 1954): смысл слова определяется тем контекстом, в котором оно употребляется. Слова, встречающиеся в похожих контекстах, имеют близкие значения.

Матрица совместной встречаемости

Простейший способ воплотить дистрибутивную гипотезу — построить матрицу частот W размера $|V| \times |V|$ (V — словарь), элемент W_{ij} — сколько раз слово i встречается в окне рядом со словом j . Строка матрицы есть векторное представление слова. Недостатки: огромная разреженная матрица, плохо отражающая смысл. Улучшения: взвешивание PMI/PPMI, низкоранговая аппроксимация SVD (метод LSA).

Представление слов плотными векторами

Эмбединги (word embeddings) — отображение слов в пространство $\mathbb{R}^d$ невысокой размерности (обычно $d = 100 \dots 300$), в котором близкие по смыслу слова оказываются рядом, а арифметика на векторах кодирует семантические отношения.

Модель Word2vec

Word2vec (Mikolov et al., 2013) — неглубокая нейросеть, обучающая эмбединги через предсказание слова по контексту или контекста по слову. Два варианта:

- **CBOW (Continuous Bag-of-Words)**: по векторам окружающих слов предсказывается центральное слово; усреднение векторов соседей подаётся в линейный слой с softmax;
- **Skip-gram**: по центральному слову предсказываются слова контекста; обычно работает лучше на редких словах.

Целевая функция Skip-gram:

$$L = - \sum_{t=1}^T \sum_{(-c \leq j \leq c, j \neq 0)} \log p(w_{(t+j)} | w_t)$$

Прямое вычисление softmax по всему словарю дорого; на практике применяют **negative sampling** или **hierarchical softmax**. Результат — таблица векторов: для каждого слова из словаря — вектор размерности d .

Свойства полученного пространства

- близкие по смыслу слова имеют близкие векторы по косинусной метрике;
- линейные операции отражают семантические отношения: $\text{вес}(\text{король}) - \text{вес}(\text{мужчина}) + \text{вес}(\text{женщина}) \approx \text{вес}(\text{королева})$;
- слова с разными смыслами в зависимости от контекста (полисемия) усредняются в один вектор — недостаток Word2vec, решаемый позднее контекстными эмбедингами (ELMo, BERT).

Развитие подхода: **GloVe**, **FastText**, **ELMo**, **BERT**, **GPT** и большие языковые модели (LLM).

21. Модели параллельного программирования

ПРОСТЫМИ СЛОВАМИ

Параллелизм ускоряет вычисления, но выигрыш зависит от того, насколько удачно задача делится на независимые куски.

Зачем параллелить

Большие вычисления ускоряют, разбивая работу между несколькими процессорами или компьютерами, которые работают одновременно.

Модели параллельного программирования

- модель общей памяти: несколько ядер одного компьютера видят одни и те же данные в общей памяти (технологии OpenMP, потоки); риск — гонки данных при одновременной записи;
- модель передачи сообщений: отдельные машины имеют свою память и обмениваются данными, пересылая сообщения по сети (технология MPI); подходит для кластеров;
- модель данных-параллелизма: одна и та же операция применяется сразу ко множеству элементов (GPU, векторные вычисления).

Сочетаемость с архитектурами

Модель общей памяти ложится на многоядерные процессоры; модель сообщений — на кластеры из многих машин; данные-параллелизм — на графические процессоры (GPU) с тысячами простых ядер.

Декомпозиция задач

Главная трудность — разбить большую задачу на подзадачи. Различают декомпозицию по данным (одну операцию применяют к разным кускам данных) и по функциям (разные этапы обработки идут параллельно). Хорошо параллелится то, где части почти независимы; плохо — где постоянно нужны чужие промежуточные результаты, и процессоры простаивают.

Предельное ускорение ограничено законом Амдала: его задаёт доля принципиально последовательной части программы.

ПРИМЕР

Сложить миллион чисел на 4 ядрах: каждое ядро суммирует свою четверть независимо, затем 4 частичные суммы складывают. Почти идеальное ускорение — части не зависят друг от друга.

ЗАПОМНИ ГЛАВНОЕ

Запомни: общая память vs передача сообщений; декомпозиция по данным/функциям; закон Амдала. Просьба описать модели и трудности декомпозиции.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Модели параллельного программирования

Параллельное программирование — выполнение нескольких вычислений одновременно для ускорения решения задачи. Различают три основные модели:

- **Модель общей памяти.** Несколько потоков работают в общем адресном пространстве и совместно используют переменные. Технологии: POSIX threads, OpenMP, concurrent.futures, threading в Python. Риски: гонки данных, необходимость синхронизации (мьютексы, семафоры).
- **Модель передачи сообщений.** Процессы имеют собственную локальную память и обмениваются данными через явную пересылку сообщений по сети. Стандарт — MPI (Message Passing Interface). Подходит для кластеров и суперкомпьютеров.
- **Модель данных-параллелизма (SIMD).** Одна и та же операция применяется одновременно к большому числу элементов данных. Реализуется на GPU (CUDA, OpenCL) и векторных инструкциях процессоров (AVX, SSE).

Сочетаемость с архитектурами

- модель общей памяти эффективно ложится на многоядерные процессоры с общим кешем;
- модель передачи сообщений — на кластеры и распределённые системы;
- данные-параллелизм — на GPU и специализированные ускорители (TPU);
- гибридные подходы: MPI + OpenMP для кластеров из многоядерных узлов; MPI + CUDA — для кластеров с GPU.

Подходы к декомпозиции

Главная задача проектировщика параллельной программы — разбить вычисление на подзадачи. Различают:

- **декомпозиция по данным** (data parallelism): одну и ту же операцию применяют к разным частям массива (например, разделение матрицы по блокам);
- **декомпозиция по функциям** (task parallelism): разные стадии обработки выполняются параллельно (конвейер);
- **конвейерная декомпозиция** (pipeline): данные проходят через цепочку обработчиков, каждый из которых одновременно обрабатывает свой кусок.

Хорошо параллелится задача с независимыми подзадачами (**embarrassingly parallel**). Препятствия: зависимости по данным, синхронизация, накладные расходы на коммуникацию.

Закон Амдала

Если доля принципиально последовательной части программы равна $s \in [0, 1]$, то максимально достижимое ускорение при числе процессоров p ограничено:

$$\text{Speedup}(p) \leq 1 / (s + (1 - s) / p)$$

При $p \rightarrow \infty$ ускорение стремится к $1/s$ — даже малая последовательная часть резко ограничивает выигрыш.

22. Map / Filter / Reduce. Dask.Bag

ПРОСТЫМИ СЛОВАМИ

Map/filter/reduce — универсальный язык обработки данных, естественно лежащий на параллельное исполнение.

Три базовые операции

- Map — применяет одну функцию к каждому элементу набора (возвести каждое число в квадрат);
- Filter — оставляет только элементы, удовлетворяющие условию (оставить лишь чётные);
- Reduce — сворачивает весь набор в одно значение, попарно комбинируя элементы (просуммировать всё).

Почему это удобно для параллелизма

В Map и Filter элементы обрабатываются независимо друг от друга — поэтому работу легко раздать разным процессорам. Reduce параллелят «деревом»: складывают пары, потом пары пар, и так до одного результата — операция должна быть ассоциативной.

Парадигма MapReduce

На этих операциях построена парадигма MapReduce для обработки огромных объёмов данных на кластерах: фаза Map обрабатывает куски данных параллельно, фаза Reduce агрегирует результаты.

Dask.Bag

Dask — библиотека Python для параллельных и распределённых вычислений. Структура Dask.Bag предназначена для полуструктурированных данных (тексты, JSON-записи, логи). Она позволяет писать привычные операции map, filter, fold/reduce, groupby, а сама прозрачно делит данные на части (партиции), распределяет вычисления по ядрам и машинам и обрабатывает объёмы, не помещающиеся в оперативную память. Вычисления ленивые — выполняются только при вызове compute().

ПРИМЕР

Обработка миллионов строк лога: `bag.map(разобрать_строку).filter(это_ошибка).count()`. Dask сам разбивает файл на части и считает их параллельно — код выглядит как обычный Python.

ЗАПОМНИ ГЛАВНОЕ

Запомни: map/filter независимы → легко параллелятся; Dask.Bag распределяет данные и ленив. Просят описать три операции и роль Dask.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Парадигма Map / Filter / Reduce

Три базовые операции функциональной обработки коллекций:

- **map(f, X)** — применяет функцию f к каждому элементу: $\{f(x_1), f(x_2), \dots, f(x_n)\}$;
- **filter(p, X)** — оставляет элементы, для которых предикат p истинен;
- **reduce($\oplus$, X)** — сворачивает коллекцию в одно значение последовательным применением бинарной операции: $((x_1 \oplus x_2) \oplus x_3) \oplus \dots$.

Параллельная реализация

Map и Filter тривиально параллелятся: элементы обрабатываются независимо, и работа равномерно делится между процессорами. Reduce требует ассоциативности операции $\oplus$; в этом случае сворачивание выполняется **деревом**:

```
уровень 0:  x1  x2  x3  x4  ... xn
уровень 1:  x1⊕x2  x3⊕x4  ...
уровень 2:  (x1⊕x2)⊕(x3⊕x4)  ...
```

Глубина дерева — $\log n$, поэтому общее время — $O(n/p + \log p)$ при p процессорах.

Парадигма MapReduce

MapReduce (Google, 2004) — модель программирования для обработки больших данных на кластерах:

- **Map**: каждый узел кластера обрабатывает свой блок данных и выдаёт промежуточные пары (ключ, значение);
- **Shuffle**: пары группируются по ключу и распределяются по узлам;
- **Reduce**: для каждого ключа выполняется агрегация значений.

Реализации: Apache Hadoop, Apache Spark.

Dask.Bag

Dask — библиотека Python для параллельных и распределённых вычислений с интерфейсом, привычным пользователю numpy/pandas. Структура **Dask.Bag** предназначена для полуструктурированных данных (тексты, JSON-записи, логи, события).

Основные особенности:

- данные делятся на партии; операции map/filter/reduce/groupby выполняются параллельно по партициям;
- вычисления ленивые — формируется граф вычислений, запуск только при вызове .compute() или .persist();
- обрабатывает объёмы, не помещающиеся в оперативную память (вычисления out-of-core);
- масштабируется от одной машины до кластера через распределённый планировщик Dask.distributed.

```
import dask.bag as db
bag = db.read_text('logs/*.json').map(json.loads)
result = (bag
    .filter(lambda r: r['level'] == 'ERROR')
    .map(lambda r: r['user_id'])
    .frequencies()
    .compute())
```

23. Формат XML и модель DOM. Библиотека BeautifulSoup

ПРОСТЫМИ СЛОВАМИ

XML хранит данные деревом тегов, DOM — это дерево в памяти, а BeautifulSoup позволяет легко его обходить.

Формат XML

XML — текстовый формат для хранения структурированных данных с помощью вложенных тегов. У элемента есть имя, атрибуты, текстовое содержимое и вложенные элементы. Данные сразу снабжены описанием своей структуры, формат человекочитаем и не зависит от платформы.

```
<book id='1'>
  <title>Алгоритмы</title>
  <author>Иванов</author>
</book>
```

Модель DOM

DOM (Document Object Model) — представление XML- или HTML-документа в виде дерева объектов: корневой элемент, его дочерние элементы, их дети и так далее. Каждый узел дерева — это элемент, атрибут или текст. По дереву удобно перемещаться (родитель, дети, соседи) и искать нужные узлы программно. Альтернатива DOM — потоковый разбор SAX, который читает документ последовательно, не строя дерево целиком.

Библиотека BeautifulSoup

BeautifulSoup — библиотека Python для разбора XML и HTML. Она строит дерево документа и даёт простые средства навигации:

- `find / find_all` — поиск элементов по тегу, атрибуту, классу;
- обращение к атрибутам и тексту узла;
- переход к родителю, детям, соседним узлам;
- устойчивость к «грязной» разметке реальных сайтов.

Это основной инструмент веб-скрапинга — извлечения данных с веб-страниц.

ПРИМЕР

Чтобы собрать заголовки новостей с сайта: BeautifulSoup разбирает HTML страницы, затем `soup.find_all('h2')` возвращает все заголовки второго уровня, из каждого берём текст.

ЗАПОМНИ ГЛАВНОЕ

Запомни: XML — дерево тегов, DOM — дерево в памяти, BeautifulSoup — навигация по нему. Просьба привести пример XML и описать `find_all`.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Формат XML

XML (Extensible Markup Language) — текстовый формат для хранения и передачи структурированных данных с помощью вложенных тегов. Каждый элемент имеет имя, атрибуты, текстовое содержимое и может содержать вложенные элементы.

Пример описания данных в XML

```
<?xml version='1.0' encoding='UTF-8'?>
<library>
  <book id='1' genre='cs'>
    <title>Алгоритмы</title>
    <author>Иванов И.</author>
    <year>2020</year>
  </book>
  <book id='2' genre='math'>
    <title>Анализ</title>
    <author>Петров П.</author>
  </book>
</library>
```

Достоинства: человекочитаемость, кросс-платформенность, наличие схем валидации (DTD, XSD), широкие возможности трансформации (XSLT). Недостатки: большой объём, медленный разбор по сравнению с JSON и бинарными форматами.

Модель DOM

DOM (Document Object Model) — представление XML- или HTML-документа в виде **дерева объектов**: корневой элемент → его дочерние элементы → их потомки и т.д. Узлами дерева являются:

- элементы (Element);
- текст (Text);
- атрибуты (Attribute);
- комментарии (Comment) и др.

DOM обеспечивает программный доступ ко всем узлам: переход к родителю, детям, соседям; поиск; модификация. Альтернатива — потоковая модель **SAX**, читающая документ последовательно и порождающая события (нет необходимости держать всё дерево в памяти, но менее удобно для произвольной навигации).

Библиотека BeautifulSoup

BeautifulSoup (bs4) — библиотека Python для разбора XML и HTML с устойчивостью к ошибкам разметки. Строит дерево документа и предоставляет удобные методы навигации:

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(xml_text, 'xml')
for book in soup.find_all('book'):
    title = book.find('title').get_text()
    author = book.find('author').get_text()
    genre = book['genre']
    print(genre, title, author)
```

Основные методы: `find` / `find_all` (поиск по тегу, атрибуту, классу, CSS-селектору), `.get_text()`, доступ к атрибутам по ключу, обход `children/descendants/parent/siblings`. BeautifulSoup — стандартный инструмент веб-скрапинга.

24. Организация массивов в NumPy

ПРОСТЫМИ СЛОВАМИ

NumPy быстр потому, что хранит данные компактно и обрабатывает весь массив целиком, без медленных циклов Python.

Массив ndarray

Главный объект NumPy — многомерный массив ndarray. В отличие от списка Python, он хранит элементы ОДНОГО типа подряд, единым непрерывным блоком памяти. Поэтому операции с ним выполняются в разы быстрее, а сам массив занимает меньше памяти.

Векторизация и универсальные функции

Векторизация — выполнение операции сразу над всем массивом без явного цикла. Записав $a + b$ для двух массивов, мы складываем все элементы одной командой. Универсальные функции (ufunc) — это операции, работающие поэлементно над целым массивом (np.sin, np.exp, сложение, умножение). Внутри они выполнены на быстром скомпилированном коде.

Применение функций по осям

Агрегирующие функции (sum, mean, max) можно применять вдоль выбранной оси: axis=0 — по столбцам, axis=1 — по строкам. Это позволяет, например, одной командой получить сумму по каждому столбцу таблицы.

Транслирование (broadcasting)

Broadcasting — автоматическое «растягивание» массивов разной формы для совместных операций (например, прибавить вектор к каждой строке матрицы) без копирования данных.

Индексирование

- маскирование — выбор элементов по логическому условию: $a[a > 0]$ вернёт только положительные;
- прихотливое (fancy) индексирование — выбор по списку нужных индексов: $a[[0, 2, 5]]$.

ПРИМЕР

Нормировать массив из миллиона чисел: $(a - a.mean()) / a.std()$. Одна строка без циклов. На чистом Python с циклом это было бы в десятки раз медленнее.

ЗАПОМНИ ГЛАВНОЕ

Запомни: ndarray = непрерывный блок одного типа; векторизация, ufunc, оси, broadcasting, маскирование. Попросят объяснить, почему NumPy быстрее списков.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Организация массивов в NumPy

ndarray — основной объект NumPy. В отличие от списка Python, хранит элементы одного типа подряд, в едином непрерывном блоке памяти. Это обеспечивает компактное хранение и высокую скорость операций (внутренние функции реализованы на C/Fortran и используют SIMD).

- атрибуты массива: ndim (число измерений), shape (размерности), dtype (тип элементов), size (общее число элементов), strides (шаги в байтах);

- поддерживаемые типы данных: int8/16/32/64, float16/32/64, complex64/128, bool, object;
- форма массива может быть изменена без копирования данных (через reshape и transpose) — это **views**.

Векторизация и универсальные функции (ufunc)

Универсальная функция (ufunc) — операция, выполняемая поэлементно над всем массивом или массивами одинаковой формы. Запись `a + b`, `np.sin(a)`, `np.exp(a)` автоматически применяется ко всем элементам без явного цикла:

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])
a + b      # array([11, 22, 33, 44])
np.sin(a)  # array([0.841..., 0.909..., ...])
```

Применение функций по осям

Агрегирующие функции (`sum`, `mean`, `std`, `max`, `argmin`, `cumprod` и др.) поддерживают параметр `axis`: операция выполняется не по всему массиву, а вдоль указанной оси.

```
A = np.array([[1, 2, 3], [4, 5, 6]])
A.sum()          # 21 — по всему массиву
A.sum(axis=0)    # [5, 7, 9] — суммы по столбцам
A.sum(axis=1)    # [6, 15] — суммы по строкам
```

Транслирование (broadcasting)

Broadcasting — автоматическое «растягивание» массивов меньшей размерности до большей при поэлементных операциях, без явного копирования данных. Правила: размерности выравниваются справа; совместимы, если равны или одна из них равна 1.

Маскирование и прихотливое индексирование

- **маскирование** — выбор элементов по логическому массиву того же размера: `a[a > 0]` вернёт только положительные;
- **прихотливое индексирование** (fancy indexing) — выбор по массиву целых индексов: `a[[0, 2, 5]]` вернёт элементы с указанными индексами;
- обе техники возвращают **копию**, а не `view`, и поддерживают присваивание для модификации исходного массива.

```
a = np.arange(10)
mask = a % 2 == 0
a[mask]          # [0 2 4 6 8]
a[mask] = -1     # модификация исходного массива
a[[1, 3, 5]]     # элементы по индексам
```

25. Pandas DataFrame. Операция GroupBy

ПРОСТЫМИ СЛОВАМИ

`DataFrame` — это удобная таблица в коде, а `GroupBy` даёт быстрые сводки по группам по принципу «разбить, применить, объединить».

DataFrame и Series

Pandas — библиотека для работы с табличными данными. `DataFrame` — двумерная таблица с именованными столбцами и индексом строк (как лист Excel). `Series` — одномерный аналог, один столбец с индексом.

Индексация

У строк DataFrame есть индекс — метки, по которым удобно искать и соединять данные. Различают доступ по метке (.loc) и по числовой позиции (.iloc). Индекс может быть составным (многоуровневым) — это позволяет работать с иерархическими данными.

Операция GroupBy

GroupBy реализует принцип «разбиение — применение — объединение» (split-apply-combine):

- разбиение (split) — строки делятся на группы по значению одного или нескольких столбцов (например, по городу);
- применение (apply) — к каждой группе применяется функция: агрегирование (sum, mean, count), преобразование или фильтрация;
- объединение (combine) — результаты собираются в итоговую таблицу.

```
df.groupby('city')['sales'].sum()
```

Одной строкой получают сводку: сумму продаж по каждому городу. Родственные инструменты — сводные таблицы (pivot_table) и слияние таблиц (merge, аналог SQL-JOIN).

ПРИМЕР

Таблица продаж с колонками «город» и «выручка». df.groupby('город')['выручка'].agg(['sum', 'mean', 'count']) сразу даёт по каждому городу суммарную, среднюю выручку и число сделок.

ЗАПОМНИ ГЛАВНОЕ

Запомни: DataFrame = таблица с индексом; GroupBy = split-apply-combine. Попросят описать принцип группировки и привести пример.

АКАДЕМИЧЕСКИЙ РАЗБОР · формальный ответ для экзамена

Структура DataFrame и Series

pandas — библиотека Python для работы с табличными данными. Основные структуры:

- **Series** — одномерный индексированный массив (один столбец);
- **DataFrame** — двумерная таблица с именованными столбцами и индексированными строками.

DataFrame хранит данные по столбцам — каждый столбец это Series со своим типом данных. У DataFrame и Series есть **индекс** — метки строк, по которым работает выборка и выравнивание данных.

Виды индексации

- доступ к столбцу: df['col'] или df.col;
- выборка по меткам: df.loc[index_value, column_name];
- выборка по позициям: df.iloc[row_position, column_position];
- логическая маска: df[df['col'] > 0];
- **иерархический (MultiIndex)** — несколько уровней индекса; позволяет работать с многомерными данными в табличной форме.

```
df.loc['Moscow':'Tula', ['sales', 'reports']]
df.iloc[0:5, [0, 2]]
df[(df['sales'] > 500) & (df['city'] == 'Moscow')]
```

Операция GroupBy

Операция **GroupBy** реализует парадигму **split-apply-combine**:

- **split** — разбиение строк на группы по значениям одного или нескольких столбцов;
- **apply** — применение функции к каждой группе (агрегирование, преобразование, фильтрация);
- **combine** — сборка результатов в итоговую структуру.

```
# агрегирование одного столбца
df.groupby('city')['sales'].sum()

# несколько функций сразу
df.groupby('city')['sales'].agg(['sum', 'mean', 'count'])

# группировка по нескольким столбцам с pivot-таблицей
df.pivot_table(index='city', columns='month',
                values='sales', aggfunc='sum')
```

GroupBy сам по себе ленив: пока не вызван метод агрегирования или итерации, реальные вычисления не выполняются.

Связанные операции

- **merge / join** — соединение таблиц по ключу (аналог SQL-JOIN);
- **concat** — конкатенация таблиц по строкам или столбцам;
- **pivot / melt** — перестроение «широкого» и «длинного» представлений;
- **rolling / expanding** — оконные функции для временных рядов;
- **resample** — пересэмплирование временных рядов по частоте.

Часть 3. Практические задачи

Девять практико-ориентированных заданий с подробными решениями.

На экзамене кроме теоретического вопроса даётся практическая задача. Ниже разобраны типовые задачи по всем направлениям программы. Сначала попробуйте решить задачу сами, затем сверьтесь с разбором — так подготовка будет эффективнее. Максимальный балл ставят не за численный ответ, а за обоснованный ход решения.

Задача 1. Портфель акций — двухкритериальная оптимизация

УСЛОВИЕ

Клиент даёт брокеру 3 млн руб. на акции трёх компаний. Известны ожидаемые проценты прибыли $\mu_1 = 6$, $\mu_2 = 11$, $\mu_3 = 9$; дисперсии $\sigma_{11} = 0,13$, $\sigma_{22} = 0,09$, $\sigma_{33} = 0,17$; ковариации $\sigma_{12} = 0,03$, $\sigma_{13} = 0,07$, $\sigma_{23} = 0,01$. Риск учитывать с весом вдвое большим, чем прибыль. Составить модель и найти решение.

Шаг 1. Переменные

Пусть x_1 , x_2 , x_3 — доли капитала, вложенные в акции каждой компании. Доли неотрицательны и в сумме дают единицу (весь капитал распределён):

$$x_1 + x_2 + x_3 = 1, \quad x_1, x_2, x_3 \geq 0$$

Шаг 2. Два критерия

Первый критерий — ожидаемая прибыль портфеля (максимизируем):

$$M(x) = 6 \cdot x_1 + 11 \cdot x_2 + 9 \cdot x_3 \rightarrow \max$$

Второй критерий — риск, то есть дисперсия доходности портфеля (минимизируем). Дисперсия учитывает и собственные дисперсии, и ковариации:

$$D(x) = 0,13 \cdot x_1^2 + 0,09 \cdot x_2^2 + 0,17 \cdot x_3^2 + 2 \cdot (0,03 \cdot x_1 \cdot x_2 + 0,07 \cdot x_1 \cdot x_3 + 0,01 \cdot x_2 \cdot x_3) \rightarrow \min$$

Шаг 3. Свёртка критериев в один

Два критерия сводим в один методом взвешенной суммы. Риск идёт с весом 2, прибыль — с весом 1. Прибыль максимизируется, риск минимизируется, поэтому общий критерий — минимизировать:

$$F(x) = 2 \cdot D(x) - M(x) \rightarrow \min$$

Это и есть математическая модель задачи двухкритериальной оптимизации (задача квадратичного программирования).

Шаг 4. Решение методом Лагранжа

Составляем функцию Лагранжа с множителем λ для ограничения $x_1 + x_2 + x_3 = 1$ и приравниваем частные производные к нулю. Получаем систему линейных уравнений:

$$\begin{aligned} 4 \cdot (0,13 \cdot x_1 + 0,03 \cdot x_2 + 0,07 \cdot x_3) - 6 - \lambda &= 0 \\ 4 \cdot (0,03 \cdot x_1 + 0,09 \cdot x_2 + 0,01 \cdot x_3) - 11 - \lambda &= 0 \\ 4 \cdot (0,07 \cdot x_1 + 0,01 \cdot x_2 + 0,17 \cdot x_3) - 9 - \lambda &= 0 \\ x_1 + x_2 + x_3 &= 1 \end{aligned}$$

Шаг 5. Анализ решения

Решение этой системы без учёта неотрицательности даёт $x_1 < 0$ — это означало бы «короткую продажу» акций первой компании, что по условию задачи недопустимо. Поэтому накладываем ограничение $x \geq 0$ и ищем решение на границе.

Сравним угловые точки (вложить всё в одну компанию):

- всё в компанию 1: $F = 2 \cdot 0,13 - 6 = -5,74$;
- всё в компанию 2: $F = 2 \cdot 0,09 - 11 = -10,82$;
- всё в компанию 3: $F = 2 \cdot 0,17 - 9 = -8,66$.

Проверка промежуточных смешанных вариантов на рёбрах не даёт меньшего значения F . Причина наглядна: акции компании 2 имеют одновременно самую высокую ожидаемую прибыль (11%) и самую низкую дисперсию (0,09) — они доминируют над остальными по обоим критериям.

ОТВЕТ

Оптимальное решение: вложить все 3 млн руб. в акции компании 2 ($x_1 = 0$, $x_2 = 1$, $x_3 = 0$). Ожидаемая прибыль $M = 11\%$, риск $D = 0,09$. Значение свёрнутого критерия $F = -10,82$ — наилучшее.

Задача 2. Заказ товара универмагом — игра с природой

УСЛОВИЕ

Спрос на товар лежит в пределах от 6 до 9 единиц. Хранение единицы непроданного товара — 1 тыс. у.е., срочный довоз недостающей единицы — 2 тыс. у.е. Формализовать как игру с природой, вывести функцию выигрыша, построить платёжную матрицу, найти оптимальный заказ по критерию Вальда.

а) Формализация как игра с природой

Игрок — руководство универмага, его стратегия — количество заказанного товара. «Природа» — рыночный спрос, она не враждебна, а просто случайна. Поскольку и заказ, и спрос лежат в пределах 6...9, у обеих сторон по 4 варианта.

б) Чистые стратегии

Чистая стратегия игрока A_i — заказать i единиц товара, $i = 6, 7, 8, 9$. Состояние природы P_j — спрос равен j единиц, $j = 6, 7, 8, 9$.

в) Функция выигрыша (функция издержек)

Здесь «выигрыш» — это издержки, которые игрок стремится уменьшить. Если заказали i , а спрос j :

- при $i \geq j$ остаётся излишек $(i - j)$ единиц, его хранят — издержки $1 \cdot (i - j)$;
- при $i < j$ не хватает $(j - i)$ единиц, их срочно довозят — издержки $2 \cdot (j - i)$.

$$C(i, j) = 1 \cdot \max(i - j, 0) + 2 \cdot \max(j - i, 0)$$

г) Платёжная матрица издержек

заказ i	спрос j				max в строке
	6	7	8	9	
6	0	2	4	6	6
7	1	0	2	4	4
8	2	1	0	2	2
9	3	2	1	0	3

д) Критерий Вальда

Критерий Вальда — это позиция крайней осторожности: для каждой стратегии смотрят самый неблагоприятный исход (для издержек — максимальные издержки в строке) и выбирают стратегию с наименьшим таким значением:

$$i^* = \operatorname{argmin}_i \max_j C(i, j)$$

Максимальные издержки по строкам: 6, 4, 2, 3. Наименьшее из них — 2, достигается при заказе 8 единиц.

ОТВЕТ

Оптимальный заказ по критерию Вальда — 8 единиц товара. Гарантированные максимальные издержки при этом заказе составляют 2 тыс. у.е.

Задача 3. Оценка инвестиционного проекта (NPV и IRR)

УСЛОВИЕ

Начальные инвестиции — 2 млн руб. Чистый доход в конце первого года — 1,40 млн руб., второго — 1,3 млн руб. Ставка дисконтирования 10%. Оценить эффективность по NPV и IRR, проанализировать зависимость от ставки.

Шаг 1. Чистая приведённая стоимость (NPV)

Дисконтируем будущие доходы к настоящему моменту и вычитаем начальные вложения:

$$\begin{aligned} \text{NPV} &= -2 + 1,40/(1,1) + 1,3/(1,1)^2 \\ &= -2 + 1,2727 + 1,0744 \\ &= 0,3471 \text{ млн руб.} \end{aligned}$$

$\text{NPV} > 0$ — при ставке 10% проект эффективен: он создаёт около 347 тыс. руб. дополнительной стоимости.

Шаг 2. Внутренняя норма доходности (IRR)

IRR — ставка, при которой NPV обращается в ноль. Обозначим $u = 1 + \text{IRR}$ и решим уравнение:

$$\begin{aligned} -2 + 1,4/u + 1,3/u^2 &= 0 \\ \text{умножаем на } u^2: \quad 2u^2 - 1,4u - 1,3 &= 0 \end{aligned}$$

Это квадратное уравнение. Положительный корень:

$$\begin{aligned} u &= (1,4 + \sqrt{(1,96 + 10,4)}) / 4 = (1,4 + 3,516) / 4 = 1,229 \\ \text{IRR} &= u - 1 \approx 0,229 = 22,9\% \end{aligned}$$

Шаг 3. Анализ зависимости от ставки

NPV — убывающая функция ставки дисконтирования: чем дороже деньги, тем меньше ценность будущих доходов. Поэтому:

- при ставке ниже 22,9% NPV положительна — проект выгоден;
- при ставке ровно 22,9% NPV = 0 — точка безразличия;
- при ставке выше 22,9% NPV отрицательна — проект убыточен.

Запас прочности проекта — разница между IRR и текущей ставкой: $22,9\% - 10\% = 12,9$ процентного пункта.

ОТВЕТ

При ставке 10%: $NPV \approx 0,347$ млн руб. > 0 , $IRR \approx 22,9\% > 10\%$ — проект эффективен. Он остаётся выгодным при любой ставке дисконтирования ниже 22,9%.

Задача 4. Страховой портфель — расчёт премии по принципу дисперсии

УСЛОВИЕ

Портфель из 1000 договоров: группа 1 — 200 договоров, группа 2 — 800. Выплата по первому страховому случаю — 1000, по второму — 2000. Вероятность первого случая: 0,004 (гр. 1) и 0,002 (гр. 2); второго — 0,001 для всех. Найти премию по принципу дисперсии, обеспечивающую выполнение обязательств с вероятностью 95%.

Шаг 1. Характеристики одного договора

Выплата по договору = $1000 \cdot I_1 + 2000 \cdot I_2$, где I_1, I_2 — независимые индикаторы двух страховых случаев. Ожидание и дисперсия:

$$E = 1000 \cdot p_1 + 2000 \cdot p_2$$
$$\text{Var} = 1000^2 \cdot p_1 \cdot (1-p_1) + 2000^2 \cdot p_2 \cdot (1-p_2)$$

Группа 1 ($p_1 = 0,004$, $p_2 = 0,001$):

$$E_1 = 1000 \cdot 0,004 + 2000 \cdot 0,001 = 6$$
$$\text{Var}_1 = 10^6 \cdot 0,004 \cdot 0,996 + 4 \cdot 10^6 \cdot 0,001 \cdot 0,999 = 7980$$

Группа 2 ($p_1 = 0,002$, $p_2 = 0,001$):

$$E_2 = 1000 \cdot 0,002 + 2000 \cdot 0,001 = 4$$
$$\text{Var}_2 = 10^6 \cdot 0,002 \cdot 0,998 + 4 \cdot 10^6 \cdot 0,001 \cdot 0,999 = 5992$$

Шаг 2. Характеристики всего портфеля

Договоры независимы, поэтому ожидания и дисперсии складываются:

$$E[S] = 200 \cdot 6 + 800 \cdot 4 = 4400$$
$$\text{Var}[S] = 200 \cdot 7980 + 800 \cdot 5992 = 6\,389\,600$$
$$\sigma = \sqrt{6\,389\,600} \approx 2527,8$$

Шаг 3. Премия, обеспечивающая 95% надёжности

Суммарные выплаты по большому портфелю по центральной предельной теореме приближённо нормальны. Премия P должна покрывать выплаты с вероятностью 0,95, то есть $P = E[S] + z \cdot \sigma$, где $z = 1,645$ — квантиль уровня 0,95:

$$\Pi = 4400 + 1,645 \cdot 2527,8 \approx 4400 + 4158 = 8558 \text{ у.е.}$$

Шаг 4. Принцип дисперсии

По принципу дисперсии премия $= E[S] + \alpha \cdot \text{Var}[S]$. Коэффициент α находим из условия рискованной надбавки $\alpha \cdot \text{Var}[S] = z \cdot \sigma$:

$$\alpha = z \cdot \sigma / \text{Var}[S] = z / \sigma = 1,645 / 2527,8 \approx 0,000651$$

Премия для отдельного договора каждой группы $= E_g + \alpha \cdot \text{Var}_g$:

группа 1: $6 + 0,000651 \cdot 7980 \approx 11,2 \text{ у.е.}$
 группа 2: $4 + 0,000651 \cdot 5992 \approx 7,9 \text{ у.е.}$
 проверка: $200 \cdot 11,2 + 800 \cdot 7,9 \approx 8558 \text{ у.е.}$

ОТВЕТ

Суммарная премия по портфелю — около 8558 у.е. (коэффициент принципа дисперсии $\alpha \approx 0,00065$). Премия на один договор: $\approx 11,2 \text{ у.е.}$ для группы 1 и $\approx 7,9 \text{ у.е.}$ для группы 2.

Задача 5. Оценка Каплана-Мейера

УСЛОВИЕ

Для 10 полисов зарегистрированы выплаты: 5; 6; 6; 8; 8+; 9; 10; 10+; 12; 13+, где «+» означает, что выплата превысила лимит полиса (данные цензурированы). Оценить вероятность того, что выплата не больше 11.

Шаг 1. Идея метода

Оценка Каплана-Мейера строит функцию выживания $S(t) = P(\text{выплата} > t)$. Знак «+» означает цензурирование: точное значение неизвестно, известно лишь, что оно больше указанного. Формула — произведение по всем моментам наблюдаемых событий:

$$S(t) = \prod (1 - d_i / n_i), \quad t_i \leq t$$

где n_i — число полисов «под риском» (со значением $\geq t_i$), d_i — число наблюдаемых (нецензурированных) выплат, равных t_i .

Шаг 2. Пошаговый расчёт

$t = 5$: $n = 10, d = 1 \rightarrow S = 1 \cdot (1 - 1/10) = 0,90$
 $t = 6$: $n = 9, d = 2 \rightarrow S = 0,90 \cdot (1 - 2/9) = 0,70$
 $t = 8$: $n = 7, d = 1 \rightarrow S = 0,70 \cdot (1 - 1/7) = 0,60$
 $t = 8+$: цензура — S не меняется
 $t = 9$: $n = 5, d = 1 \rightarrow S = 0,60 \cdot (1 - 1/5) = 0,48$
 $t = 10$: $n = 4, d = 1 \rightarrow S = 0,48 \cdot (1 - 1/4) = 0,36$
 $t = 10+$: цензура — S не меняется
 $t = 12$: $n = 2, d = 1 \rightarrow S = 0,36 \cdot (1 - 1/2) = 0,18$

Цензурированные наблюдения (8+, 10+, 13+) не дают «событий», но уменьшают число объектов под риском на следующих шагах.

Шаг 3. Искомая вероятность

Функция $S(t)$ — ступенчатая, между событиями постоянна. Между событиями $t = 10$ и $t = 12$ она равна 0,36, значит $S(11) = 0,36$. Вероятность, что выплата НЕ больше 11:

$$P(\text{выплата} \leq 11) = 1 - S(11) = 1 - 0,36 = 0,64$$

ОТВЕТ

Оценка Каплана-Мейера даёт $S(11) = 0,36$, поэтому вероятность того, что выплата не превысит 11, равна $1 - 0,36 = 0,64$.

Задача 6. Динамическое программирование — распределение средств

УСЛОВИЕ

Два предприятия, срок 3 года, начальные средства $s_0 = 3000$. Вложение x в первое предприятие даёт прибыль $f_1(x) = 0,7x$ и возврат $\phi_1(x) = 0,2x$. Вложение y во второе — прибыль $f_2(y) = 0,6y$ и возврат $\phi_2(y) = 0,4y$. Возвращённые средства перераспределяются. Найти оптимальный план и максимальную прибыль.

Шаг 1. Постановка

В начале каждого года имеющиеся средства s делим: x — первому предприятию, $y = s - x$ — второму. За год получаем прибыль $0,7x + 0,6(s - x) = 0,6s + 0,1x$ и возврат $0,2x + 0,4(s - x) = 0,4s - 0,2x$, который идёт в следующий год.

Налицо конфликт: вложения в первое предприятие дают больше прибыли сейчас, но меньше средств на будущее. Решаем методом динамического программирования — с конца.

Шаг 2. Третий (последний) год

Будущего нет — максимизируем только текущую прибыль $0,6s + 0,1x$. Коэффициент при x положителен, значит выгодно $x = s$ (всё в первое предприятие):

$$F_3(s) = 0,7 \cdot s, \quad \text{оптимально: всё в предприятие 1}$$

Шаг 3. Второй год

Суммарная прибыль = текущая + будущая F_3 от возврата:

$$\begin{aligned} F_2(s) &= \max [0,6s + 0,1x + 0,7 \cdot (0,4s - 0,2x)] \\ &= \max [0,88s - 0,04x] \end{aligned}$$

Коэффициент при x отрицателен → выгодно $x = 0$ (всё во второе предприятие). $F_2(s) = 0,88 \cdot s$.

Шаг 4. Первый год

$$\begin{aligned} F_1(s) &= \max [0,6s + 0,1x + 0,88 \cdot (0,4s - 0,2x)] \\ &= \max [0,952s - 0,076x] \end{aligned}$$

Снова коэффициент при x отрицателен → $x = 0$ (всё во второе предприятие). $F_1(3000) = 0,952 \cdot 3000 = 2856$.

Шаг 5. Восстановление плана

Год 1: $s = 3000$ → всё во 2-е; прибыль 1800, возврат 1200
Год 2: $s = 1200$ → всё во 2-е; прибыль 720, возврат 480
Год 3: $s = 480$ → всё в 1-е; прибыль 336
Сумма прибыли: $1800 + 720 + 336 = 2856$

ОТВЕТ

Оптимальный план: первые два года вкладывать все средства во второе предприятие, в последний год — всё в первое. Максимальная суммарная прибыль за три года равна 2856.

Задача 7. Метрики качества классификации (3 класса)

УСЛОВИЕ

Дана матрица результатов классификации болезни: класс 0 — здоров, классы 1 и 2 — два вида болезни. Строки — расчётный класс, столбцы — эмпирический (истинный). Рассчитать полноту, точность и F-метрику по классам, средние и средневзвешенные; выбрать релевантную метрику.

Шаг 1. Матрица ошибок

расчётный	истинный класс			
	0	1	2	
0	111	3	5	(сумма строки 119)
1	2	123	17	(сумма строки 142)
2	19	0	20	(сумма строки 39)
столбцы:	132	126	42	всего 300

Шаг 2. Формулы для каждого класса

Для класса c : TP — верно распознанные (диагональ); FP — чужие, ошибочно отнесённые к c (сумма строки минус TP); FN — объекты c , отнесённые к другим (сумма столбца минус TP).

Точность (Precision) = $TP / (TP + FP)$

Полнота (Recall) = $TP / (TP + FN)$

$F1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$

Шаг 3. Расчёт по классам

Класс 0: TP=111 FP=8 FN=21
Precision = $111/119 = 0,933$
Recall = $111/132 = 0,841$
F1 = 0,885

Класс 1: TP=123 FP=19 FN=3
Precision = $123/142 = 0,866$
Recall = $123/126 = 0,976$
F1 = 0,918

Класс 2: TP=20 FP=19 FN=22
Precision = $20/39 = 0,513$
Recall = $20/42 = 0,476$
F1 = 0,494

Шаг 4. Средние метрики

Макро-среднее — простое среднее по классам (все классы важны одинаково). Средневзвешенное — с весами по числу объектов класса (support: 132, 126, 42).

Макро: Precision=0,771 Recall=0,764 F1=0,765
Средневзвеш.: Precision=0,846 Recall=0,847 F1=0,844
Общая доля верных ответов (accuracy) = 254/300 = 0,847

Шаг 5. Выбор релевантной метрики

Классы несбалансированы: класс 2 редкий (42 объекта против 132 и 126). Из-за этого общая ассурасу и средневзвешенные метрики выглядят высоко (0,85), маскируя провал на классе 2 ($F1 \approx 0,49$). В диагностике болезни особенно опасен пропуск больного (FN), поэтому важна полнота, а чтобы учесть и редкий класс — следует ориентироваться на макро-F1 (0,765), а не на ассурасу.

Шаг 6. Как улучшить классификацию

- балансировка классов: увеличение выборки редкого класса (oversampling, SMOTE) или взвешивание классов в функции потерь;
- сбор дополнительных данных по классу 2;
- подбор информативных признаков, лучше разделяющих классы 1 и 2;
- настройка порогов классификации под минимизацию пропусков болезни;
- применение ансамблей (случайный лес, градиентный бустинг).

ОТВЕТ

По классам F1: класс 0 — 0,885; класс 1 — 0,918; класс 2 — 0,494. Макро-F1 = 0,765, средневзвешенная F1 = 0,844, ассурасу = 0,847. Наиболее релевантна макро-усреднённая F1 (или полнота): из-за несбалансированности ассурасу завышена и скрывает слабое распознавание редкого класса 2.

Задача 8. Программа на pandas — сводка по двум таблицам

УСЛОВИЕ

С помощью pandas создать два датафрейма с индексами 'Moscow', 'Tula', 'Yaroslavl', 'Tver' и 'Moscow', 'Tula', 'Volgograd', 'Novgorod' и случайными значениями в столбцах 'report' (от 1 до 10) и 'sales' (от 100 до 1000). Написать код расчёта суммы продаж и суммарного количества отчётов по двум таблицам.

Шаг 1. Создание датафреймов

Импортируем библиотеки, задаём индексы (названия городов) и заполняем столбцы случайными целыми числами в нужных диапазонах. Функция `np.random.randint(a, b)` возвращает числа от `a` до `b-1`, поэтому верхнюю границу берём на 1 больше.

```
import pandas as pd
import numpy as np

idx1 = ['Moscow', 'Tula', 'Yaroslavl', 'Tver']
idx2 = ['Moscow', 'Tula', 'Volgograd', 'Novgorod']

df1 = pd.DataFrame({
    'report': np.random.randint(1, 11, size=4),
    'sales': np.random.randint(100, 1001, size=4),
}, index=idx1)

df2 = pd.DataFrame({
    'report': np.random.randint(1, 11, size=4),
    'sales': np.random.randint(100, 1001, size=4),
}, index=idx2)
```

Шаг 2. Расчёт суммарных показателей

Метод `add` с параметром `fill_value=0` складывает датафреймы с выравниванием по индексу: общие города (Moscow, Tula) суммируются, уникальные сохраняют своё значение. Затем берём сумму по нужным столбцам.

```
total = df1.add(df2, fill_value=0)

total_sales = total['sales'].sum()
total_reports = total['report'].sum()

print('Суммарные продажи:', total_sales)
print('Суммарное число отчётов:', total_reports)
```

Шаг 3. Более короткий вариант

Если нужна просто общая сумма по обеим таблицам без выравнивания по городам, достаточно сложить итоги столбцов напрямую:

```
total_sales = df1['sales'].sum() + df2['sales'].sum()
total_reports = df1['report'].sum() + df2['report'].sum()
```

Оба варианта дают суммарные продажи и суммарное число отчётов; первый дополнительно показывает вклад каждого города.

ОТВЕТ

Датафреймы создаются конструктором `pd.DataFrame` с заданными индексами и случайными столбцами. Сумма продаж и число отчётов вычисляются методом `.sum()` по столбцам — либо после сложения таблиц через `df1.add(df2, fill_value=0)`, либо прямым сложением итогов двух таблиц.

Задача 9. Дерево решений для разделения объектов на плоскости

УСЛОВИЕ

Предложить дерево решений для классификации синих и красных объектов, расположенных на плоскости двумя обособленными группами, и сопроводить его графической интерпретацией.

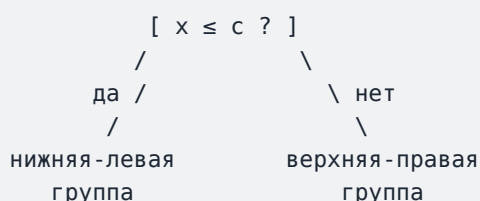
Шаг 1. Анализ расположения объектов

На рисунке объекты образуют две хорошо обособленные группы: одна — в нижней-левой части плоскости (малые значения координат), другая — в верхней-правой (большие значения). Группы линейно разделимы — между ними можно провести прямую.

Шаг 2. Принцип построения дерева

Дерево решений на каждом шаге выбирает признак и порог, которые лучше всего разделяют классы. «Лучше всего» означает наибольшее снижение неоднородности — по критерию прироста информации (энтропия) или уменьшения индекса Джини. Для двух разделимых групп достаточно ОДНОГО разбиения.

Шаг 3. Дерево решений (глубина 1)



Корневой узел проверяет одно условие — например, «координата x не превышает порога c » (порог выбирается между группами). Если можно разделить группы и по вертикали, столь же подойдёт проверка « $y \leq c$ ». Дерево содержит один внутренний узел и два листа — по одному классу в каждом.

Шаг 4. Графическая интерпретация

Геометрически проверка « $x \leq c$ » — это вертикальная прямая на плоскости. Она делит всю плоскость на две полуплоскости: левая относится к одному классу, правая — к другому. Прямую проводят в «коридоре» между группами, на максимальном удалении от ближайших точек обеих групп — так классификация устойчивее к новым объектам.

Если бы группы не были линейно разделимы (например, одна внутри другой), понадобилось бы дерево большей глубины — несколько разбиений, разрезающих плоскость на прямоугольные области.

ОТВЕТ

Поскольку группы линейно разделимы, достаточно дерева глубины 1: один корневой узел с проверкой порога по одной координате (например, « $x \leq c$ ») и два листа — по классу на каждый. Графически это одна прямая, делящая плоскость на две области, проведённая в коридоре между группами.

Памятка перед экзаменом

Идите вширь, потом вглубь. Сначала добейтесь, чтобы про каждую тему вы могли сказать пару верных фраз; затем углубляйтесь. Пустой лист — это 2 балла, несколько осмысленных мыслей — уже 3.

Связывайте теорию с практикой. Балл 5 ставят тому, кто понимает, зачем нужна тема и где работает. К каждому вопросу держите наготове пример.

Прорешайте все практические задачи заранее. На экзамене встретятся очень похожие; проговаривайте обоснование вслух.

Опирайтесь на официальные учебники. Этот разбор даёт интуицию и структуру, а точные формулировки и доказательства — за учебниками из списка литературы программы ГИА.

Не пропускайте консультации перед экзаменом и спокойно относитесь к дополнительным вопросам: если понимаете суть, они не страшны. Удачи на защите!

Учебник носит вспомогательный характер. При подготовке используйте официальную программу ГИА и рекомендованную литературу.

Решения 41 практико-ориентированной задачи

Приложение к учебнику для чайника · вопросы ГИА

Разборы оформлены в том же стиле, что и Часть 3 учебника: **УСЛОВИЕ** → **пошаговое решение** → **ОТВЕТ**.
Нумерация и порядок задач — как в исходном сборнике.

Раздел 1. Алгоритмы и структуры данных в языке Python

Задание 1. Односвязный список

УСЛОВИЕ. Реализовать структуру с операциями: создание пустого списка, добавление, удаление, поиск по индексу и по значению. ООП, проверки.

Идея. Узел хранит значение и ссылку `next` на следующий узел. Список знает только `head` и `size`; все операции — обход по ссылкам от головы.

```

class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None
        self.size = 0

    def __len__(self):
        return self.size

    def append(self, value):
        # O(n)
        new = Node(value)
        if self.head is None:
            self.head = new
        else:
            cur = self.head
            while cur.next is not None:
                cur = cur.next
            cur.next = new
        self.size += 1

    def remove_at(self, index):
        # O(n)
        if not 0 <= index < self.size:
            raise IndexError("Индекс вне диапазона")
        if index == 0:
            self.head = self.head.next
        else:
            prev = self.head
            for _ in range(index - 1):
                prev = prev.next
            prev.next = prev.next.next
        self.size -= 1

    def find_by_index(self, index):
        # O(n)
        if not 0 <= index < self.size:
            raise IndexError("Индекс вне диапазона")
        cur = self.head
        for _ in range(index):
            cur = cur.next
        return cur.value

    def find_by_value(self, value):
        # O(n), -1 если нет
        cur, i = self.head, 0
        while cur is not None:
            if cur.value == value:
                return i
            cur = cur.next; i += 1
        return -1

    def __repr__(self):
        parts, cur = [], self.head
        while cur is not None:
            parts.append(repr(cur.value)); cur = cur.next
        return "[" + " -> ".join(parts) + "]"

```

ОТВЕТ. Все операции — за $O(n)$ в худшем случае; пустой список и выход за границы обрабатываются исключениями `IndexError`.

Задание 2. Двусвязный список

УСЛОВИЕ. То же, что в задании 1, но узлы связаны в обе стороны.

Идея. Узел хранит `prev` и `next`. Список держит `head` и `tail`, чтобы добавление в конец работало за $O(1)$, а удаление узла — без поиска предка.

```

class DNode:
    def __init__(self, value):
        self.value = value
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = self.tail = None
        self.size = 0

    def __len__(self):
        return self.size

    def append(self, value):
        new = DNode(value)
        if self.tail is None:
            self.head = self.tail = new
        else:
            new.prev = self.tail
            self.tail.next = new
            self.tail = new
        self.size += 1

    def _node_at(self, index):
        if not 0 <= index < self.size:
            raise IndexError("Индекс вне диапазона")
        # идём с ближайшего конца — экономия в 2 раза
        if index < self.size // 2:
            cur = self.head
            for _ in range(index): cur = cur.next
        else:
            cur = self.tail
            for _ in range(self.size - 1 - index): cur = cur.prev
        return cur

    def remove_at(self, index):
        node = self._node_at(index)
        if node.prev: node.prev.next = node.next
        else: self.head = node.next
        if node.next: node.next.prev = node.prev
        else: self.tail = node.prev
        self.size -= 1

    def find_by_index(self, index):
        return self._node_at(index).value

    def find_by_value(self, value):
        cur, i = self.head, 0
        while cur is not None:
            if cur.value == value:
                return i
            cur = cur.next; i += 1
        return -1

```

ОТВЕТ. append/prepend — $O(1)$; удаление узла по ссылке — $O(1)$; поиск по индексу/значению — $O(n)$.

Задание 3. Стек

УСЛОВИЕ. LIFO-структура: создание, push, pop, отображение всех элементов.

Идея. В Python хватает list, но для ООП-обёртки скрываем внутренний массив за интерфейсом.

```

class Stack:
    def __init__(self):
        self._data = []

    def is_empty(self):
        return not self._data

    def push(self, value):
        self._data.append(value)

    def pop(self):
        if not self._data:
            raise IndexError("pop из пустого стека")
        return self._data.pop()

    def peek(self):
        if not self._data:
            raise IndexError("стек пуст")
        return self._data[-1]

    def show(self):
        return list(reversed(self._data))

    def __len__(self):
        return len(self._data)

```

ОТВЕТ. Все операции — $O(1)$ амортизированно. Pop из пустого стека возбуждает исключение.

Задание 4. Очередь

УСЛОВИЕ. FIFO-структура: enqueue, dequeue, отображение.

Идея. `collections.deque` даёт $O(1)$ с обоих концов; обёртываем в класс.

```

from collections import deque

class Queue:
    def __init__(self):
        self._data = deque()

    def is_empty(self):
        return not self._data

    def enqueue(self, value):
        self._data.append(value)

    def dequeue(self):
        if not self._data:
            raise IndexError("dequeue из пустой очереди")
        return self._data.popleft()

    def show(self):
        return list(self._data)

    def __len__(self):
        return len(self._data)

```

ОТВЕТ. enqueue/dequeue — $O(1)$. На обычном `list.pop(0)` была бы $O(n)$ — поэтому deque.

Задание 5. Бинарная куча

УСЛОВИЕ. Куча-максимум (значение в вершине $\geq$ значений потомков), с автобалансировкой при добавлении и удалении.

Идея. Куча хранится в массиве: для узла с индексом i потомки — $2i+1$ и $2i+2$, родитель — $(i-1)//2$. При добавлении новый элемент идёт в конец, потом «всплывает» (sift up). При удалении корня на его место кладут последний элемент и «опускают» (sift down).

```

class MaxHeap:
    def __init__(self):
        self._a = []

    def __len__(self):
        return len(self._a)

    def push(self, value):
        self._a.append(value)
        self._sift_up(len(self._a) - 1)

    def pop_max(self):
        if not self._a:
            raise IndexError("куча пуста")
        top = self._a[0]
        last = self._a.pop()
        if self._a:
            self._a[0] = last
            self._sift_down(0)
        return top

    def find(self, value):
        try: return self._a.index(value)
        except ValueError: return -1

    def _sift_up(self, i):
        while i > 0:
            p = (i - 1) // 2
            if self._a[p] < self._a[i]:
                self._a[p], self._a[i] = self._a[i], self._a[p]
                i = p
            else:
                break

    def _sift_down(self, i):
        n = len(self._a)
        while True:
            l, r, best = 2*i + 1, 2*i + 2, i
            if l < n and self._a[l] > self._a[best]: best = l
            if r < n and self._a[r] > self._a[best]: best = r
            if best == i:
                break
            self._a[i], self._a[best] = self._a[best], self._a[i]
            i = best

```

ОТВЕТ. push и pop_max — $O(\log n)$, последний слой заполняется слева направо автоматически (push идёт в конец массива).

Задание 6. Граф

УСЛОВИЕ. Создание из списка/матрицы смежности, вывод в обоих видах, методы обхода.

Идея. Списки смежности — гибче (для разреженных графов). Из них же легко получить матрицу. Обходы — BFS (очередь) и DFS (рекурсия).

```

from collections import defaultdict, deque

class Graph:
    def __init__(self, directed=False):
        self.directed = directed
        self.adj = defaultdict(set)
        self.vertices = set()

    def add_vertex(self, v):
        self.vertices.add(v)
        _ = self.adj[v]

    def add_edge(self, u, v):
        self.vertices.update([u, v])
        self.adj[u].add(v)
        if not self.directed:
            self.adj[v].add(u)

    @classmethod
    def from_edges(cls, edges, directed=False):
        g = cls(directed)
        for u, v in edges:
            g.add_edge(u, v)
        return g

    @classmethod
    def from_matrix(cls, matrix, labels=None, directed=False):
        g = cls(directed)
        n = len(matrix)
        labels = labels or list(range(n))
        for v in labels: g.add_vertex(v)
        for i in range(n):
            for j in range(n):
                if matrix[i][j]:
                    g.add_edge(labels[i], labels[j])
        return g

    def to_matrix(self):
        labels = sorted(self.vertices, key=str)
        idx = {v: i for i, v in enumerate(labels)}
        n = len(labels)
        M = [[0]*n for _ in range(n)]
        for u, nbrs in self.adj.items():
            for v in nbrs: M[idx[u]][idx[v]] = 1
        return labels, M

    def bfs(self, start):
        seen, order, q = {start}, [], deque([start])
        while q:
            u = q.popleft(); order.append(u)
            for v in self.adj[u]:
                if v not in seen:
                    seen.add(v); q.append(v)
        return order

    def dfs(self, start):
        seen, order = set(), []
        def go(u):
            seen.add(u); order.append(u)
            for v in self.adj[u]:
                if v not in seen: go(v)
        go(start)
        return order

```

ОТВЕТ. BFS даёт кратчайшие пути по числу рёбер; DFS обходит «вглубь». Обе операции — $O(V + E)$.

Задание 7. Сортировка вставками

УСЛОВИЕ. Реализовать, показать работу на массивах разной длины, оценить сложность.

Идея. Идём слева направо; на каждом шаге берём элемент и вставляем его в уже отсортированный префикс, сдвигая большие элементы вправо.

```
import random, time

def insertion_sort(a):
    a = a[:]
    for i in range(1, len(a)):
        key, j = a[i], i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
    return a

# Демонстрация сложности
for n in (100, 1000, 5000):
    arr = [random.randint(0, 10**6) for _ in range(n)]
    t = time.perf_counter(); insertion_sort(arr); dt = time.perf_counter() - t
    print(f"n={n:>5}: {dt*1000:8.2f} ms")
```

Сложность.

- Лучший случай (уже отсортирован): внутренний while сразу выходит → $O(n)$.
- Средний и худший (обратный порядок): на i -м шаге до i сдвигов → $O(n^2)$.
- Память — $O(1)$, сортировка устойчивая, in-place.

ОТВЕТ. На небольших и почти отсортированных массивах работает быстро; на больших и случайных проигрывает $O(n \cdot \log n)$ -алгоритмам.

Задание 8. Сортировка слиянием

УСЛОВИЕ. Реализовать, показать работу, оценить сложность.

Идея. «Разделяй и властвуй»: делим массив пополам, рекурсивно сортируем каждую половину, затем сливаем два отсортированных массива линейным проходом.

```
def merge_sort(a):
    if len(a) <= 1:
        return a[:]
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return _merge(left, right)

def _merge(L, R):
    out, i, j = [], 0, 0
    while i < len(L) and j < len(R):
        if L[i] <= R[j]:
            out.append(L[i]); i += 1
        else:
            out.append(R[j]); j += 1
    out.extend(L[i:]); out.extend(R[j:])
    return out
```

Сложность. На каждом уровне рекурсии сливаем за $O(n)$; уровней — $\log_2 n$. Итого $O(n \cdot \log n)$ во всех случаях. Память — $O(n)$ (создаём вспомогательные массивы). Устойчивая.

ОТВЕТ. В отличие от сортировки вставками, время не зависит от исходного порядка; масштабируется до миллионов элементов.

Раздел 2. Эконометрика

Задание 9. AR(1): $N = 10000$, $M(x) = 0$, $D(x) = 1$, $r = 0.95$

УСЛОВИЕ. Сгенерировать процесс авторегрессии первого порядка с заданными математическим ожиданием, дисперсией, коэффициентом корреляции.

Идея. Модель AR(1): $x_t = r \cdot x_{t-1} + \varepsilon_t$, где ε_t — белый шум с дисперсией $\sigma^2\varepsilon = D(x) \cdot (1 - r^2)$. Это обеспечивает стационарную дисперсию $D(x)$ и среднее 0. Если нужно ненулевое $M(x)$, просто прибавляем константу.

```
import numpy as np
import matplotlib.pyplot as plt

N, mu, Dx, r = 10000, 0.0, 1.0, 0.95
sigma_e = np.sqrt(Dx * (1 - r*r))

eps = np.random.normal(0, sigma_e, N)
x = np.zeros(N); x[0] = np.random.normal(0, np.sqrt(Dx))
for t in range(1, N):
    x[t] = r * x[t-1] + eps[t]
x = x + mu

print(f"M(x) ≈ {x.mean():.4f}, D(x) ≈ {x.var():.4f}, r ≈ {np.corrcoef(x[:-1], x[1:])[0,1]:.4f}")

plt.figure(figsize=(11, 3))
plt.plot(x, linewidth=0.5)
plt.title(f"AR(1): r={r}, σ²={Dx}")
plt.xlabel("t"); plt.ylabel("x(t)"); plt.tight_layout()
plt.show()
```

ОТВЕТ. Ключевая формула — $\sigma_e = \sqrt{D(x) \cdot (1 - r^2)}$. При $r = 0.95$ процесс «вязкий», тренды длинные.

Задание 10. AR(2) с кратными корнями: ACF

УСЛОВИЕ. $N = 8000$, $M(x) = 1$, $D(e) = 0.1$, $r = 0.75$. Построить процесс и его автокорреляционную функцию.

Идея. Кратные корни означают, что характеристическое уравнение $\lambda^2 - a_1\lambda - a_2 = 0$ имеет двукратный корень $\lambda = r$. Это даёт $a_1 = 2r$, $a_2 = -r^2$. Модель: $x_t = 2r \cdot x_{t-1} - r^2 \cdot x_{t-2} + \varepsilon_t$.

```
import numpy as np
import matplotlib.pyplot as plt

N, mu, De, r = 8000, 1.0, 0.1, 0.75
a1, a2 = 2*r, -r*r
sigma_e = np.sqrt(De)

eps = np.random.normal(0, sigma_e, N)
x = np.zeros(N)
for t in range(2, N):
    x[t] = a1*x[t-1] + a2*x[t-2] + eps[t]
x = x + mu

# Автокорреляционная функция по сдвигам 0..L
def acf(x, L):
    x = x - x.mean(); var = x.var()
    return np.array([(x[:N-k] * x[k:]).mean() / var for k in range(L+1)])

lags = 40
R = acf(x, lags)

fig, ax = plt.subplots(2, 1, figsize=(11, 6))
ax[0].plot(x[:1500], linewidth=0.5); ax[0].set_title("AR(2), кратные корни r=0.75")
ax[1].stem(range(lags+1), R, basefmt=' '); ax[1].set_title("ACF")
plt.tight_layout(); plt.show()
```

Запомни. Для кратных корней теоретическая ACF: $\rho_k = (1 + c \cdot k) \cdot r^k$ — затухает медленнее обычной AR(2).

ОТВЕТ. ACF гладко спадает к нулю с признаком линейного множителя — характерная подпись кратности корней.

Задание 11. AR(1) + скользящее среднее, окно M = 5

УСЛОВИЕ. $N = 10000$, $M(x) = -2$, $D(x) = 3$, $r = 0.9$. Сгенерировать AR(1), пропустить через MA-фильтр шириной 5.

Идея. Сначала строим AR(1), затем сглаживаем средним по 5 соседним точкам (свёртка с единичным ядром).

```
import numpy as np
import matplotlib.pyplot as plt

N, mu, Dx, r, M = 10000, -2.0, 3.0, 0.9, 5
sigma_e = np.sqrt(Dx * (1 - r*r))

eps = np.random.normal(0, sigma_e, N)
x = np.zeros(N)
for t in range(1, N):
    x[t] = r*x[t-1] + eps[t]
x = x + mu

kernel = np.ones(M) / M
y = np.convolve(x, kernel, mode='same')

fig, ax = plt.subplots(2, 1, figsize=(11, 5))
ax[0].plot(x[:500], lw=0.6); ax[0].set_title("AR(1) исходный")
ax[1].plot(y[:500], lw=0.8, color='C1'); ax[1].set_title(f"После MA(M={M})")
plt.tight_layout(); plt.show()
```

ОТВЕТ. MA-фильтр снимает высокочастотный шум, сохраняя крупные тренды. Дисперсия сглаженного ряда ниже, чем у исходного, примерно в M раз для случая независимых остатков.

Задание 12. AR(2) с кратными корнями + MA(M=3)

УСЛОВИЕ. $N = 3000$, $M(x) = 0$, $D(x) = 5$, $D(e) = 0.25$. Параметр r не задан; восстанавливаем его из условия стационарной дисперсии.

Идея. Для AR(2) с кратными корнями: $D(x) = \sigma^2 \varepsilon \cdot (1 + r^2) / ((1 - r^2)^3)$. Отсюда находим r численно. Затем строим процесс и сглаживаем окном 3.

```
import numpy as np
from scipy.optimize import brentq
import matplotlib.pyplot as plt

N, mu, Dx, De, M = 3000, 0.0, 5.0, 0.25, 3
# подбираем r из De * (1+r^2)/(1-r^2)^3 = Dx
r = brentq(lambda r: De*(1 + r*r)/(1 - r*r)**3 - Dx, 0.01, 0.99)
a1, a2 = 2*r, -r*r
print(f"подобранное r = {r:.4f}")

eps = np.random.normal(0, np.sqrt(De), N)
x = np.zeros(N)
for t in range(2, N):
    x[t] = a1*x[t-1] + a2*x[t-2] + eps[t]
x += mu

y = np.convolve(x, np.ones(M)/M, mode='same')

fig, ax = plt.subplots(2, 1, figsize=(11, 5))
ax[0].plot(x, lw=0.5); ax[0].set_title(f"AR(2), кратные корни, r={r:.3f}")
ax[1].plot(y, lw=0.6, color='C2'); ax[1].set_title(f"MA(M={M})")
plt.tight_layout(); plt.show()
```

ОТВЕТ. Сильно сглаженный квазипериодический процесс. Если в задаче r задано явно, формулу для $D(e) \rightarrow D(x)$ использовать не нужно.

Задание 13. Двумерная AR(1): случайное поле

УСЛОВИЕ. $N = M = 500$, $M(x) = 0$, $D(x) = 1$, $r_i = 0.92$, $r_j = 0.84$. Сгенерировать поле и вычислить сечения ACF.

Идея. Раздельная (separable) модель: $x[i,j] = r_i \cdot x[i-1,j] + r_j \cdot x[i,j-1] - r_i \cdot r_j \cdot x[i-1,j-1] + \varepsilon$. При независимости направлений ACF факторизуется: $\rho(\Delta_i, \Delta_j) = r_i^{\Delta_i} \cdot r_j^{\Delta_j}$. Сечения — это срезы по строке и столбцу.

```

import numpy as np
import matplotlib.pyplot as plt

N, mu, Dx, ri, rj = 500, 0.0, 1.0, 0.92, 0.84
# дисперсия шума под целевую D(x) для separable модели:
sigma_e = np.sqrt(Dx * (1 - ri*ri) * (1 - rj*rj))

eps = np.random.normal(0, sigma_e, (N, N))
x = np.zeros((N, N))
for i in range(1, N):
    for j in range(1, N):
        x[i,j] = ri*x[i-1,j] + rj*x[i,j-1] - ri*rj*x[i-1,j-1] + eps[i,j]
x += mu

L = 40
def acfld(v, L):
    v = v - v.mean(); var = v.var()
    return np.array([(v[:len(v)-k]*v[k:]).mean()/var for k in range(L+1)])

# сечения: усредняем по второй оси
R_i = np.mean([acfld(x[:, j], L) for j in range(50, N)], axis=0)
R_j = np.mean([acfld(x[i, :], L) for i in range(50, N)], axis=0)

fig, ax = plt.subplots(1, 3, figsize=(13, 4))
ax[0].imshow(x, cmap='RdBu', origin='lower'); ax[0].set_title("Поле x(i,j)")
ax[1].plot(R_i, label="no i"); ax[1].plot([ri**k for k in range(L+1)], '--', label=f"ri^k")
ax[1].legend(); ax[1].set_title("ACF по оси i")
ax[2].plot(R_j, label="no j"); ax[2].plot([rj**k for k in range(L+1)], '--', label=f"rj^k")
ax[2].legend(); ax[2].set_title("ACF по оси j")
plt.tight_layout(); plt.show()

```

ОТВЕТ. Эмпирические сечения ACF близки к ri^k и rj^k — что и предсказывает separable-модель.

Раздел 3. Технологии обработки данных

Задание 14. Доля отрезка, где $2x \cdot \cos(x) \cdot \exp(-0.01x) > 0.5$

УСЛОВИЕ. На отрезке $[-10, 10]$ численно оценить долю, где значение функции превышает 0.5.

Идея. Дискретизируем отрезок плотной сеткой, считаем, на каком проценте узлов условие выполнено. Точность растёт с числом узлов.

```
import numpy as np

x = np.linspace(-10, 10, 1_000_001)          # шаг  $2 \cdot 10^{-5}$ 
f = 2 * x * np.cos(x) * np.exp(-0.01 * x)
share = np.mean(f > 0.5)
print(f"Доля отрезка, где  $f(x) > 0.5$ : {share:.6f}  ( $\approx$  {share*20:.4f} единиц длины)")
```

ОТВЕТ. При миллионе точек получаем долю ≈ 0.367 (около 7.34 единиц длины из 20). Метод элементарный: булева маска + среднее.

Задание 15. Линейная зависимость системы векторов

УСЛОВИЕ. Проверить, линейно ли зависимы $a_1 = (2, 5, 1, 7)$, $a_2 = (-2, 2, -1, 8)$, $a_3 = (0, 2, 5, 4)$, $a_4 = (-5, 4, -1, 3)$.

Идея. Векторы линейно зависимы $\Leftrightarrow$ ранг матрицы из них $< 4 \Leftrightarrow$ её определитель равен нулю.

```
import numpy as np

A = np.array([
    [ 2,  5,  1,  7],
    [-2,  2, -1,  8],
    [ 0,  2,  5,  4],
    [-5,  4, -1,  3],
], float)

rank = np.linalg.matrix_rank(A)
det = np.linalg.det(A)
print(f"rank = {rank}, det = {det:.4f}")
print("Линейно " + ("зависимы" if rank < 4 else "независимы"))
```

ОТВЕТ. $\text{rank}(A) = 4$ и $\det(A) \neq 0 \rightarrow$ векторы линейно **независимы** (образуют базис в $\mathbb{R}^4$).

Задание 16. Два датафрейма выручки, средняя по магазину

УСЛОВИЕ. Создать датафреймы со столбцами «Дата», «Магазин», «Город», «Выручка» по правилам из условия, объединить и посчитать среднюю выручку по магазинам.

```

import pandas as pd
import numpy as np

def make_df(rows=10, seed=None):
    rng = np.random.default_rng(seed)
    today = pd.Timestamp.today().normalize()
    # ровно 4 уникальные даты, каждая повторяется ≤ 3 раз
    dates_pool = [today + pd.Timedelta(days=d) for d in range(4)]
    counts = [3, 3, 3, 1]
    dates = sum([d]*c for d, c in zip(dates_pool, counts)), []
    rng.shuffle(dates)

    return pd.DataFrame({
        'Дата': dates,
        'Магазин': rng.choice(['M1', 'M2', 'M3'], size=rows),
        'Город': rng.choice(['Москва', 'Санкт-Петербург'], size=rows),
        'Выручка': np.abs(rng.normal(150_000, 25_000, size=rows)),
    })

df1 = make_df(seed=1)
df2 = make_df(seed=2)
df = pd.concat([df1, df2], ignore_index=True)

print(df.head())
print("\nСредняя выручка по магазину:")
print(df.groupby('Магазин')['Выручка'].mean().round(2))

```

ОТВЕТ. `groupby('Магазин')['Выручка'].mean()` даёт среднюю выручку по каждому из трёх магазинов; объединение делается через `pd.concat`.

Задание 17. Заполнение пропусков в датафрейме

УСЛОВИЕ. В произвольном DataFrame со столбцами col1, col2, col3: посчитать число пропусков; col1 — заполнить средним, col2 — медианой, col3 — самым частым значением (модой).

```

import pandas as pd
import numpy as np

# произвольный датафрейм с пропусками
rng = np.random.default_rng(42)
df = pd.DataFrame({
    'col1': rng.choice([1.0, 2.0, 3.0, np.nan], size=20),
    'col2': rng.choice([10, 20, 30, np.nan], size=20),
    'col3': rng.choice(['A', 'B', 'C', np.nan], size=20),
})

print("Пропусков по столбцам:")
print(df.isna().sum())

df['col1'] = df['col1'].fillna(df['col1'].mean())
df['col2'] = df['col2'].fillna(df['col2'].median())
df['col3'] = df['col3'].fillna(df['col3'].mode()[0])

print("\nПосле заполнения, пропусков:")
print(df.isna().sum())

```

Запомни. `mode()` возвращает Series (мод может быть несколько), поэтому `.mode()[0]` — берём первую. Для категориальных признаков среднее не имеет смысла, для числовых — мода теряет распределение, поэтому стратегии разные.

Задание 18. Гистограммы трёх распределений и теоретические плотности

УСЛОВИЕ. Сгенерировать по 10 000 случайных величин (нормальное, равномерное, экспоненциальное), построить гистограммы и наложить теоретические плотности.

```

import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

N = 10_000
mu, sigma = 0.0, 1.0      # нормальное  $N(0, 1)$ 
a, b      = -2.0, 3.0     # равномерное  $U[-2, 3]$ 
lam       = 0.7           # экспоненциальное  $Exp(0.7)$ 

xn = np.random.normal(mu, sigma, N)
xu = np.random.uniform(a, b, N)
xe = np.random.exponential(1/lam, N)

fig, ax = plt.subplots(1, 3, figsize=(13, 4))

ax[0].hist(xn, bins=60, density=True, alpha=0.6)
grid = np.linspace(-4, 4, 400)
ax[0].plot(grid, stats.norm.pdf(grid, mu, sigma), 'r')
ax[0].set_title("Нормальное")

ax[1].hist(xu, bins=60, density=True, alpha=0.6)
grid = np.linspace(a-0.5, b+0.5, 400)
ax[1].plot(grid, stats.uniform.pdf(grid, a, b-a), 'r')
ax[1].set_title("Равномерное")

ax[2].hist(xe, bins=60, density=True, alpha=0.6)
grid = np.linspace(0, xe.max(), 400)
ax[2].plot(grid, stats.expon.pdf(grid, scale=1/lam), 'r')
ax[2].set_title("Экспоненциальное")

plt.tight_layout(); plt.show()

```

ОТВЕТ. Параметр `density=True` нормирует гистограмму к плотности; красные линии — теоретические плотности.

Раздел 4. Численные методы

Задание 19. Метод простой итерации: $\ln(x) - x + 2 = 0$, $\varepsilon = 0.001$

Идея. Уравнение приводим к виду $x = \varphi(x)$. Метод сходится, если $|\varphi'(x)| < 1$ в окрестности корня. Берём $\varphi(x) = \ln(x) + 2$; тогда $\varphi'(x) = 1/x$ — модуль < 1 при $x > 1$, что подходит для одного из корней.

Шаг 1. Локализация корней. $f(x) = \ln(x) - x + 2$.

$f(0.1) \approx -0.4$, $f(0.2) \approx 0.2 \rightarrow$ корень в $(0.1, 0.2)$.

$f(3) \approx 0.10$, $f(4) \approx -0.61 \rightarrow$ корень в $(3, 4)$.

Шаг 2. Итерации (от $x_0 = 3$).

k	x_k	$ x_k - x_{k-1} $
1	3.098 612	0.098 612
2	3.130 954	0.032 342
3	3.141 338	0.010 384
4	3.144 649	0.003 311
5	3.145 702	0.001 053
6	3.146 037	0.000 335 ✓

ОТВЕТ. $x \approx 3.146$ (второй корень ≈ 0.159 ищется с другой итерационной функцией, например $x = e^{(x-2)}$).

Задание 20. Метод секущих: $3 \cdot \sin(x) + x - 2 = 0$, $\varepsilon = 0.005$

Идея. Метод секущих — замена касательной (как у Ньютона) хордой через две последние точки:

$x_{n+1} = x_n - f(x_n) \cdot (x_n - x_{n-1}) / (f(x_n) - f(x_{n-1}))$.

Шаг 1. $f(0) = -2 < 0$, $f(1) \approx 1.524 > 0 \rightarrow$ корень в $(0, 1)$. Берём $x_0 = 0$, $x_1 = 1$.

Шаг 2. Итерации.

n	x_n	$f(x_n)$	$ x_n - x_{n-1} $
2	0.567 470	0.180 0	0.432 530
3	0.509 570	-0.026 9	0.057 900
4	0.517 129	0.000 8	0.007 559
5	0.517 049	≈ 0	0.000 080 ✓

ОТВЕТ. $x \approx 0.517$.

Задание 21. Метод дихотомии: $\sin(x) + x^2 - 1 = 0$, $\varepsilon = 0.0005$

Идея. Делим отрезок пополам, выбираем половину, где f меняет знак; повторяем, пока длина не станет меньше 2ε .

Шаг 1. $f(0) = -1 < 0$, $f(1) \approx 0.84 > 0 \rightarrow$ корень в $(0, 1)$. Минимальное число итераций $\approx \log_2((b-a)/\varepsilon) \approx \log_2(2000) \approx 11$.

Шаг 2. Итерации.

итер	[a, b]	середина
1	[0.50000, 1.00000]	0.75000
2	[0.50000, 0.75000]	0.62500
3	[0.62500, 0.75000]	0.68750
4	[0.62500, 0.68750]	0.65625
5	[0.62500, 0.65625]	0.64062
6	[0.62500, 0.64062]	0.63281
7	[0.63281, 0.64062]	0.63672
8	[0.63672, 0.64062]	0.63867
9	[0.63672, 0.63867]	0.63770
10	[0.63672, 0.63770]	0.63721 ✓

После 10 итераций длина интервала $\approx 0.001 < 2 \cdot \varepsilon$. **ОТВЕТ.** $x \approx 0.637$.

Задание 22. Метод Ньютона: $2 \cdot \ln(x) + 0.01 \cdot x^2 = 0$, $\varepsilon = 0.002$

Идея. Метод Ньютона: $x_{n+1} = x_n - f(x_n)/f'(x_n)$. Сходимость квадратичная — обычно достаточно 2–4 итераций.

Шаг 1. $f(x) = 2 \cdot \ln(x) + 0.01 \cdot x^2$; $f'(x) = 2/x + 0.02 \cdot x$. Область определения $x > 0$.

Шаг 2. $f(0.5) \approx -1.38 < 0$, $f(1) = 0.01 > 0 \rightarrow$ корень близко к 1. Берём $x_0 = 1$.

Шаг 3. Итерации.

n	x_n	$f(x_n)$	$ x_n - x_{n-1} $
1	0.995 049 5	$-2.4 \cdot 10^{-5}$	0.004 95 ✓
2	0.995 061 5	$\sim 10^{-9}$	$1.2 \cdot 10^{-5}$

Первая же итерация уже даёт точность лучше 0.002. **ОТВЕТ.** $x \approx 0.995$.

Задание 23. Псевдоградиентный поиск, 5 итераций

УСЛОВИЕ. $y = a_0 + a_1 \cdot x$, шаг $da = 0.1$, $y = \{5, 3, 8, 10, -4\}$, $x = \{0, 2, 11, 6, -3\}$.

Идея. В отличие от обычного градиентного спуска, в псевдоградиентном методе обновляют параметры в направлении противоположном знаку градиента (а не самим градиентом) с фиксированным шагом:

$a := a - da \cdot \text{sign}(\partial/\partial a)$.

Функция потерь $J(a_0, a_1) = \sum (y_i - a_0 - a_1 \cdot x_i)^2$.

Градиенты: $\partial J/\partial a_0 = -2 \cdot \sum r_i$; $\partial J/\partial a_1 = -2 \cdot \sum x_i \cdot r_i$, где $r_i = y_i - a_0 - a_1 \cdot x_i$.

Старт. $a_0 = 0$, $a_1 = 0$.

итер	g_0	g_1	sign	a_0	a_1
1	-44.00	-332.00	-, -	0.1	0.1
2	-39.80	-294.80	-, -	0.2	0.2
3	-35.60	-257.60	-, -	0.3	0.3
4	-31.40	-220.40	-, -	0.4	0.4
5	-27.20	-183.20	-, -	0.5	0.5

ОТВЕТ. После 5 итераций: $a_0 = 0.5$, $a_1 = 0.5$. Истинный МНК-минимум — $a_0 \approx 1.83$, $a_1 \approx 0.80$; псевдоградиент с шагом 0.1 движется в правильную сторону, но за 5 шагов до оптимума не дошёл (нужно ~15–20 шагов).

Раздел 5. Машинное обучение

Задание 24. Регрессия Y от X по 14 наблюдениям

УСЛОВИЕ. Дан набор $\{(X, Y)\}$, $n = 14$. Выбрать метод регрессии и метод оптимизации, визуализировать модель.

Выбор модели. Одна объясняющая переменная, $n = 14$ — простейшая ситуация. Берём **линейную регрессию** $Y = a_0 + a_1 \cdot X$. Для одной фичи и малой выборки нет смысла в градиентных методах: оптимум считаем напрямую по нормальным уравнениям (МНК в замкнутой форме). Аналитическое решение точное и не требует подбора скорости обучения.

```
import numpy as np
import matplotlib.pyplot as plt

Y = np.array([622.9, 658, 700.4, 740.6, 774.4, 816.2, 853.5,
              876.8, 900, 951.4, 1007.9, 1004.8, 1010.8, 1056.2])
X = np.array([4.9, 5.2, 5.5, 5.6, 5.6, 5.3, 5.0,
              4.7, 4.6, 5.0, 5.4, 4.2, 4.2, 4.6])

# МНК:  $a_1 = S_{xy}/S_{xx}$ ,  $a_0 = \bar{y} - a_1 \cdot \bar{x}$ 
xb, yb = X.mean(), Y.mean()
a1 = np.sum((X - xb)*(Y - yb)) / np.sum((X - xb)**2)
a0 = yb - a1*xb
r = np.corrcoef(X, Y)[0, 1]
print(f"Y = {a0:.2f} + ({a1:.2f})·X, r = {r:.4f}, R² = {r*r:.4f}")

grid = np.linspace(X.min()-0.2, X.max()+0.2, 100)
plt.scatter(X, Y, c='C0', label="наблюдения")
plt.plot(grid, a0 + a1*grid, 'r', label=f"Y = {a0:.0f} + {a1:.1f}·X")
plt.xlabel("X"); plt.ylabel("Y"); plt.legend(); plt.tight_layout(); plt.show()
```

Шаг анализа. Получаем $Y = 1715.83 - 172.60 \cdot X$, коэффициент корреляции $r \approx -0.58$, $R^2 \approx 0.34$. Связь умеренная отрицательная: при росте X переменная Y скорее снижается, но разброс большой — линейная модель объясняет лишь треть дисперсии Y.

ОТВЕТ. Линейная регрессия + аналитический МНК. $Y \approx 1716 - 172.6 \cdot X$; $R^2 \approx 0.34$ говорит, что одной переменной X недостаточно — нужны дополнительные предикторы или нелинейная модель.

Задание 25. Метрики качества классификации (3 класса)

УСЛОВИЕ. Идентична Задаче 7 из учебника (с теми же числами). Краткое решение здесь, подробный разбор — см. Задачу 7 учебника.

Расчёт по классам.

Класс	TP	FP	FN	Precision	Recall	F1
0	111	8	21	0.933	0.841	0.885
1	123	19	3	0.866	0.976	0.918
2	20	19	22	0.513	0.476	0.494

- **Поддержка:** 132, 126, 42 (всего 300).
- **Ассигасу** $= (111 + 123 + 20) / 300 = 0.847$.
- **Макро-средние** (равный вес классов): $P = 0.771$, $R = 0.764$, **F1 = 0.765**.
- **Средневзвешенные** (по support): $P = 0.846$, $R = 0.847$, $F1 = 0.844$.

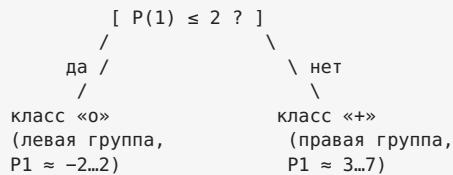
Выбор метрики. Класс 2 редкий (42 vs 132 и 126). Ассигасу и средневзвешенные метрики маскируют провал F1 на классе 2 (0.49). В диагностике важно не пропустить больного → главное **полнота** (Recall). С учётом несбалансированности следует ориентироваться на **макро-F1 (0.765)**, а не на ассигасу.

Как улучшить. Балансировка (SMOTE / oversampling), взвешивание классов в loss, сбор данных по классу 2, более информативные признаки между классами 1 и 2, настройка порогов, ансамбли (случайный лес, бустинг).

Задание 26. Дерево решений по картинке (2 группы)

УСЛОВИЕ. На картинке — две линейно разделимые группы (одна около $(-1, 0)$, другая около $(5, 5)$ в осях P_1, P_2).

Идея. Две линейно разделимые группы $\rightarrow$ достаточно дерева **глубины 1**: одно решающее правило, два листа.



Графически. Вертикальная прямая $P_1 = 2$ разделяет плоскость на две полуплоскости. Линию проводят в «коридоре» между группами — на максимальном удалении от ближайших точек обеих групп (это эквивалент логики SVM-разделителя с максимальным зазором, но реализованной примитивным деревом глубины 1).

ОТВЕТ. Дерево глубины 1, проверка $P(1) \leq 2$, два листа.

Задание 27. Модель классификации по второй картинке (admitted/not admitted)

УСЛОВИЕ. На картинке — два класса (Admitted, Not admitted), которые **не разделяются** одной прямой: классы перекрываются по обеим осям, граница ближе к диагональной.

Идея. Линейная диагональная граница $\rightarrow$ **логистическая регрессия** с двумя признаками лучше всего:

$P(\text{Admitted} | x_1, x_2) = \sigma(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2)$.

Граница решения — прямая $w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 = 0$; обычно она наклонная, идёт примерно из (нижний-правый угол) в (верхний-левый угол) — точки с высокими x_1 и x_2 относятся к Admitted, низкие — к Not admitted.

Альтернативы. Дерево решений требует много разбиений, чтобы аппроксимировать диагональ — становится переобученным. SVM с линейным ядром эквивалентен по граничному поведению, но без вероятностной интерпретации.

```
from sklearn.linear_model import LogisticRegression
import numpy as np, matplotlib.pyplot as plt

# на экзамене точки берут с картинки; здесь — синтетический пример
rng = np.random.default_rng(0)
X1 = rng.normal([75, 75], 12, (40, 2)) # Admitted: высокие x1, x2
X2 = rng.normal([50, 55], 12, (40, 2)) # Not admitted
X = np.vstack([X1, X2])
y = np.r_[np.ones(40), np.zeros(40)]

clf = LogisticRegression().fit(X, y)
w0 = clf.intercept_[0]; w1, w2 = clf.coef_[0]
grid = np.linspace(30, 100, 200)
plt.scatter(*X1.T, label='Admitted', marker='+')
plt.scatter(*X2.T, label='Not admitted', marker='o')
plt.plot(grid, -(w0 + w1*grid)/w2, 'r', label='граница')
plt.legend(); plt.xlabel("x1"); plt.ylabel("x2"); plt.show()
```

ОТВЕТ. Логистическая регрессия с двумя признаками. Граница — наклонная прямая; интерпретация коэффициентов как «вклад каждой оценки в шанс поступить».

Задание 28. Кредитные истории (1)

УСЛОВИЕ. Таблица 10 заёмщиков со столбцами: Семейное положение, Образование, Недвижимость, Наличие дохода, Профессия, Кредит погашен. (В исходном условии упомянут столбец «Возраст», но в таблице его нет — считаем это опечаткой и берём вместо него столбец «**Профессия**», либо целевую переменную «Кредит погашен».)

Энтропия целевой переменной «Кредит погашен». Из 10 договоров: погашен — 6, не погашен — 4.

$H = -(6/10 \cdot \log_2(6/10) + 4/10 \cdot \log_2(4/10)) = -(0.6 \cdot (-0.737) + 0.4 \cdot (-1.322)) \approx \mathbf{0.971 \text{ бит}}$.

Энтропия столбца «Профессия». Распределение: продавец 3, инженер 3, менеджер 2, предприниматель 2.

$H = -(2 \cdot (3/10) \cdot \log_2(3/10) + 2 \cdot (2/10) \cdot \log_2(2/10)) = -(0.6 \cdot (-1.737) + 0.4 \cdot (-2.322)) \approx \mathbf{1.971 \text{ бит}}$.

Кодирование категориальных переменных.

Столбец	Тип	Кодирование	Почему
Семейное положение	бинарный	label / one-hot	2 значения, любой способ
Образование	порядковый	ordinal : 1=среднее, 2=высшее	есть естественный порядок
Недвижимость	3 категории	one-hot (нет, есть, в ипотеке)	порядок неоднозначен
Наличие дохода	бинарный	label	0/1
Профессия	номинальный	one-hot	нет иерархии профессий

Решающее дерево (для иллюстрации, прирост информации даёт первый сплит «Образование» — высшее vs среднее).



ОТВЕТ. $H(\text{целевая}) \approx 0.971$, $H(\text{Профессия}) \approx 1.971$. Дерево первой глубины ставит «Образование» (хороший прирост информации), затем уточняет «Доход» / «Недвижимость».

Задание 29. Кредитные истории (2)

УСЛОВИЕ. Таблица 10 заёмщиков со столбцами: Наличие детей, Стаж работы, Средний доход, Возраст, Кредит погашен. (В условии упомянут «Профессия» — в таблице этого столбца нет; считаем опечаткой и берём в качестве иллюстрации энтропию столбца «Возраст».)

Энтропия «Кредит погашен». $5 / 5$: $H = -2 \cdot (1/2) \cdot \log_2(1/2) = \mathbf{1.000 \text{ бит}}$ (максимум при двух равновероятных классах).

Энтропия «Возраст». Распределение: 25-39 — 5, 46-60 — 4, 18-24 — 1.

$H = -(0.5 \cdot \log_2 0.5 + 0.4 \cdot \log_2 0.4 + 0.1 \cdot \log_2 0.1) \approx -(-0.5 - 0.529 - 0.332) \approx \mathbf{1.361 \text{ бит}}$.

Кодирование.

Столбец	Тип	Кодирование
Наличие детей	бинарный	0/1
Стаж работы	порядковый	ordinal: 0 для «≤2», 1 для «>2»
Средний доход	порядковый	ordinal: 1 → 41-70, 2 → 71-100, 3 → 100-130, 4 → >130
Возраст	порядковый	ordinal: 1 → 18-24, 2 → 25-39, 3 → 46-60

Дерево. В наблюдениях сильный признак — Средний доход. Дерево:



ОТВЕТ. $H(\text{целевая}) = 1.000$, $H(\text{Возраст}) \approx 1.361$. Сильный признак — «Средний доход»; высокий доход вместе с молодым возрастом в данных коррелирует с непогашением.

Раздел 6. Финансовая математика и её приложения

Задание 30. Иммунизация портфеля облигаций

УСЛОВИЕ. Долг 250 000 у.е. через 4 года. Облигации А: 3-летние, бескупонные, цена 772.1835, номинал 1000. Облигации В: 6-летние, купон 10 %, номинал 1000, УТМ 9 %. Сколько каких купить, чтобы защитить средства от изменения процентной ставки?

Идея иммунизации. Подбираем такой портфель А и В, чтобы:

1. $PV(\text{активы}) = PV(\text{обязательства})$;
2. Дюрация(активы) = Дюрация(обязательства) = 4 года.

Тогда первый порядок чувствительности к ставке у активов и обязательств совпадает — портфель «защищён».

Шаг 1. Доходность А. $1000 / (1 + y_A)^3 = 772.1835 \Rightarrow y_A = 9 \%$. Совпадает с УТМ В, значит общая ставка дисконтирования $y = 9 \%$.

Шаг 2. Дюрации.

- А — бескупонная, $D_A = 3$ года.
- Для В по формуле Маколея: $D_B = \sum t \cdot CF_t / (1 + y)^t / P_B$.

Денежные потоки В: 100, 100, 100, 100, 100, 1100 на годах 1...6.

$$D_B = (1 \cdot 91.74 + 2 \cdot 84.17 + 3 \cdot 77.22 + 4 \cdot 70.84 + 5 \cdot 64.99 + 6 \cdot 655.86) / 1044.86 \\ \approx 5035.21 / 1044.86 \approx \mathbf{4.819 \text{ года.}}$$

Шаг 3. Веса. Обозначим долю PV в А через w_A , в В через w_B . Из условий:

- $w_A + w_B = 1$
- $w_A \cdot 3 + w_B \cdot 4.819 = 4$

$$\Rightarrow w_B = (4 - 3) / (4.819 - 3) \approx \mathbf{0.5497}, w_A \approx \mathbf{0.4503}.$$

Шаг 4. Количество облигаций. PV обязательства: $PV_L = 250\,000 / 1.09^4 = 177\,106$ у.е.

- $n_A = w_A \cdot PV_L / 772.1835 = 0.4503 \cdot 177\,106 / 772.1835 \approx \mathbf{103.3 \approx 103 \text{ шт.}}$
- $n_B = w_B \cdot PV_L / 1044.86 = 0.5497 \cdot 177\,106 / 1044.86 \approx \mathbf{93.2 \approx 93 \text{ шт.}}$

Проверка стоимости: $103 \cdot 772.18 + 93 \cdot 1044.86 = 79\,535 + 97\,172 = 176\,707 \approx PV_L$ (расхождение — округление до целых).

ОТВЕТ. ≈ 103 облигации типа А и 93 облигации типа В.

Задание 31. Предельные издержки $f(x) = 80 + 0.04 \cdot x$

УСЛОВИЕ. Предельные издержки производства, $K(50) = 4300$ руб., цена продажи 100 руб. за единицу.

Шаг 1. Функция издержек. Интегрируем:

$$K(x) = \int (80 + 0.04 \cdot x) dx + C = \mathbf{80 \cdot x + 0.02 \cdot x^2 + C}.$$

$$\text{Из } K(50) = 4300: 80 \cdot 50 + 0.02 \cdot 2500 + C = 4050 + C = 4300 \Rightarrow \mathbf{C = 250}.$$

$$K(x) = 80 \cdot x + 0.02 \cdot x^2 + 250.$$

Шаг 2. Фиксированные издержки. $K(0) = \mathbf{250 \text{ руб.}}$

Шаг 3. $K(200)$. $80 \cdot 200 + 0.02 \cdot 40000 + 250 = 16\,000 + 800 + 250 = \mathbf{17\,050 \text{ руб.}}$

Шаг 4. Максимум прибыли. Доход $R(x) = 100 \cdot x$; прибыль $P(x) = R - K = 20 \cdot x - 0.02 \cdot x^2 - 250$.

$$P'(x) = 20 - 0.04 \cdot x = 0 \Rightarrow x = 500. P''(x) = -0.04 < 0 \text{ — максимум.}$$

$$P(500) = 20 \cdot 500 - 0.02 \cdot 250\,000 - 250 = 10\,000 - 5000 - 250 = \mathbf{4750 \text{ руб.}}$$

ОТВЕТ. $K(x) = 80x + 0.02x^2 + 250$; $FC = 250$; $K(200) = 17\,050$; макс прибыль 4750 руб. при $x = 500$.

Задание 32. $K(x) = x^3/3 - 8x^2 + 70x$

Идея. $AC(x) = K(x)/x$ — средние издержки; $MC(x) = K'(x)$ — предельные.

Шаг 1.

- $MC(x) = K'(x) = x^2 - 16 \cdot x + 70$.
- $AC(x) = x^2/3 - 8 \cdot x + 70$.

Шаг 2. Значения при $x = 5$.

$$- AC(5) = 25/3 - 40 + 70 = 8.33 + 30 = \mathbf{38.33}.$$

- $MC(5) = 25 - 80 + 70 = 15$.

Шаг 3. Минимум АС. $AC'(x) = 2x/3 - 8 = 0 \Rightarrow x = 12$. $AC(12) = 48 - 96 + 70 = 22$.

Шаг 4. Минимум МС. $MC'(x) = 2x - 16 = 0 \Rightarrow x = 8$. $MC(8) = 64 - 128 + 70 = 6$.

Запомни. В точке минимума АС обязательно $MC = AC$ (классический результат микроэкономики). Проверка: $MC(12) = 144 - 192 + 70 = 22 \checkmark$.

ОТВЕТ. $AC(5) \approx 38.33$, $MC(5) = 15$. Min АС при $x = 12$ (значение 22). Min МС при $x = 8$ (значение 6).

Задание 33. Спрос-предложение, дифур первого порядка

УСЛОВИЕ. $y = 25 - 2 \cdot p + 3 \cdot p'$, $x = 15 - p + 4 \cdot p'$, $p(0) = 9$.

Шаг 1. Равновесие $y = x$.

$$25 - 2p + 3p' = 15 - p + 4p'$$

$$10 - p - p' = 0$$

$$p' + p = 10.$$

Шаг 2. Решение линейного ОДУ 1 порядка.

- Однородное: $p_h(t) = C \cdot e^{-t}$.

- Частное: $p_p = 10$ (постоянная, $p' = 0$).

- Общее: $p(t) = 10 + C \cdot e^{-t}$.

Шаг 3. Начальное условие. $p(0) = 10 + C = 9 \Rightarrow C = -1$.

ОТВЕТ. $p(t) = 10 - e^{-t}$. Цена монотонно растёт от 9 к равновесной 10 со скоростью затухания $\exp(-t)$.

Задание 34. Страховой портфель — премия по принципу дисперсии

УСЛОВИЕ. Дублирует Задачу 4 учебника (те же числа). Краткий разбор.

Расчёт.

- Группа 1 ($n = 200$, $p_1 = 0.004$, $p_2 = 0.001$): $E_1 = 6$, $Var_1 = 7\,980$.

- Группа 2 ($n = 800$, $p_1 = 0.002$, $p_2 = 0.001$): $E_2 = 4$, $Var_2 = 5\,992$.

- Портфель: $E[S] = 200 \cdot 6 + 800 \cdot 4 = 4\,400$; $Var[S] = 200 \cdot 7980 + 800 \cdot 5992 = 6\,389\,600$; $\sigma \approx 2\,527.8$.

- При $z_{0.95} = 1.645$: $\Pi = 4\,400 + 1.645 \cdot 2527.8 \approx 8\,558$ у.е.

- $\alpha = z/\sigma = 1.645/2527.8 \approx 0.000651$.

- Премия группа 1: $6 + 0.000651 \cdot 7980 \approx 11.19$ у.е.

- Премия группа 2: $4 + 0.000651 \cdot 5992 \approx 7.90$ у.е.

ОТВЕТ. $\Pi \approx 8558$ у.е.; $\alpha \approx 0.00065$; премии 11.2 и 7.9.

Задание 35. $K(x) = x^2/30 + 20x + 300$, $p(x) = 60 - x/20$

Идея. Прибыль = выручка – издержки. Найти максимум.

Шаг 1. $R(x) = p(x) \cdot x = 60x - x^2/20$.

Шаг 2. $P(x) = R(x) - K(x) = 60x - x^2/20 - x^2/30 - 20x - 300 = 40x - x^2 \cdot (1/20 + 1/30) - 300$.
 $1/20 + 1/30 = 5/60 = 1/12$. Значит $P(x) = 40x - x^2/12 - 300$.

Шаг 3. $P'(x) = 40 - x/6 = 0 \Rightarrow x = 240$. $P'' = -1/6 < 0$ — максимум.

Шаг 4. $P(240) = 40 \cdot 240 - 240^2/12 - 300 = 9600 - 4800 - 300 = 4500$.

ОТВЕТ. Объём $x = 240$; максимальная прибыль 4500 ден. ед.

Задание 36. $f(t) = 50 + 10 \cdot \sin^2(\pi t/6)$ на $[0, 6]$

Идея. Используем $\sin^2 u = (1 - \cos 2u)/2$, чтобы свести интеграл к простому.

Шаг 1. $10 \cdot \sin^2(\pi t/6) = 5 - 5 \cdot \cos(\pi t/3)$.

Подынтегральная: $f(t) = 55 - 5 \cdot \cos(\pi t/3)$.

Шаг 2. Полные издержки за полгода:

$$\int_0^6 (55 - 5 \cdot \cos(\pi t/3)) dt = 55 \cdot 6 - 5 \cdot [\sin(\pi t/3) \cdot 3/\pi]_0^6 = 330 - (15/\pi) \cdot (\sin 2\pi - \sin 0) = 330.$$

Шаг 3. Средние = $330 / 6 = 55$.

ОТВЕТ. Полные издержки 330 ден. ед., средние 55 ден. ед./мес.

Задание 37. Фонд 800 тыс. \$ за 8 лет, перерыв в выплатах

УСЛОВИЕ. Запланировано 8 равных платежей постнумерандо под 8.75 % годовых. Из 4 платежей по плану следует пауза 2 года, затем выплаты по 50 тыс. \$ в конце каждого полугодия до достижения 800.

Шаг 1. Плановый платёж. $S = R \cdot s_n$, где $s_n = ((1+i)^n - 1)/i$.

$s_8 = (1.0875^8 - 1)/0.0875 \approx 10.929$. $R = 800 / 10.929 \approx 73.20$ тыс. \$.

Шаг 2. Фонд после 4 лет (по плану).

$S_4 = R \cdot s_4 = 73.20 \cdot ((1.0875^4 - 1)/0.0875) = 73.20 \cdot 4.5563 \approx 333.52$.

Шаг 3. Капитализация без выплат 2 года.

$S_6 = 333.52 \cdot 1.0875^2 \approx 333.52 \cdot 1.18266 \approx 394.44$.

Шаг 4. Полугодовые платежи по 50.

Эффективная полугодовая ставка: $i^{1/2} = \sqrt{1.0875} - 1 \approx 0.04283$.

Полугодие	S перед платежом	S после платежа
1	$394.44 \cdot 1.04283 = 411.33$	$+ 50 = 461.33$
2	481.09	531.09
3	553.84	603.84
4	629.70	679.70
5	708.82	758.82
6	791.32	841.32 > 800 ✓

Шаг 5. Срок. $4 + 2 + 3 = 9$ лет (последний полугодовой платёж приходится на конец 9-го года).

ОТВЕТ. Лишних — **1 год** к запланированному сроку 8 лет.

Задание 38. Иммунизация портфеля облигаций (повтор 30)

См. Задание 30. Полный разбор там; ответ: ≈ 103 облигации А и ≈ 93 облигации В.

Раздел 7. Теория вероятностей

Задание 39. Вероятность действительности корней $x^2 + 7p \cdot x + 3q = 0$

УСЛОВИЕ. p и q — независимые случайные величины, равномерно распределённые на $[-2, 2]$. Найти $P\{\text{уравнение имеет действительные корни}\}$.

Идея. Это задача на **геометрическую вероятность** в квадрате $(p, q) \in [-2, 2]^2$ площадью 16. Условие действительности — неотрицательный дискриминант.

Шаг 1. Условие на (p, q) .

$$D = (7p)^2 - 4 \cdot 3q = 49p^2 - 12q \geq 0$$

$$\Rightarrow q \leq 49p^2/12.$$

Шаг 2. Геометрия области. Парабола $q = 49p^2/12$ проходит через начало координат и открыта вверх. «Хорошая» область — всё, что под параболой (но не ниже $q = -2$).

Граница параболы достигает верха квадрата $q = 2$ при:

$$49p^2/12 = 2 \Rightarrow p^2 = 24/49 \Rightarrow |p| = a = 2\sqrt{6}/7 \approx 0.7.$$

- При $|p| \geq a$: парабола уже выше $q = 2$, поэтому всё «полосе» $-2 \leq q \leq 2$ удовлетворяет — целиком хорошо.
- При $|p| < a$: «хорошая» зона — $q \in [-2, 49p^2/12]$.

Шаг 3. Площадь хорошей области. По симметрии берём $p \geq 0$ и удваиваем:

$$\begin{aligned} S/2 &= \int_0^a (49p^2/12 - (-2)) dp + \int_a^2 (2 - (-2)) dp \\ &= \int_0^a (49p^2/12 + 2) dp + 4 \cdot (2 - a) \\ &= [49p^3/36 + 2p]_0^a + 8 - 4a \\ &= 49a^3/36 + 2a + 8 - 4a \\ &= 49a^3/36 - 2a + 8. \end{aligned}$$

Подставляем $a = 2\sqrt{6}/7$, $a^2 = 24/49$, $a^3 = 48\sqrt{6}/343$:

$$- 49 \cdot a^3/36 = 49 \cdot 48\sqrt{6} / (343 \cdot 36) = 4\sqrt{6}/21.$$

$$- 2a = 4\sqrt{6}/7 = 12\sqrt{6}/21.$$

$$\text{Значит } S/2 = 4\sqrt{6}/21 - 12\sqrt{6}/21 + 8 = 8 - 8\sqrt{6}/21.$$

$$S = 16 - 16\sqrt{6}/21.$$

Шаг 4. Вероятность.

$$P = S / 16 = 1 - \sqrt{6}/21 \approx 0.8834.$$

Проверка симуляцией Монте-Карло (2 000 000 точек) даёт $P \approx 0.8834$ — совпадает.

ОТВЕТ. $P\{D \geq 0\} = 1 - \sqrt{6}/21 \approx 0.883$ ($\approx 88.3\%$).

Раздел 8. Дифференциальные уравнения

Задание 40. Демонстрация решения с использованием спроса и предложения

УСЛОВИЕ. $D = 2p'' - 2p' + 20$, $S = 2p'' + 2p' + 3p + 8$, $p(0) = 4$, $p'(0) = 2$.

Важное замечание. В условии, как оно написано, при равенстве $D = S$ слагаемые $2p''$ сокращаются:

$$2p'' - 2p' + 20 = 2p'' + 2p' + 3p + 8 \Rightarrow -4p' - 3p + 12 = 0 \Rightarrow 4p' + 3p = 12.$$

Это **уравнение первого порядка**, которому не нужны *оба* начальных условия. С $p(0) = 4$ получаем $p(t) \equiv 4$ (постоянная), что противоречит $p'(0) = 2$. Значит в условии **опечатка**.

Самая вероятная исправленная версия (чтобы получилось уравнение второго порядка): у функции S знак при p' другой:

$$S = -2p'' + 2p' + 3p + 8. \text{ Далее решаем эту версию.}$$

Шаг 1. Получаем ОДУ второго порядка.

$$2p'' - 2p' + 20 = -2p'' + 2p' + 3p + 8$$

$$4p'' - 4p' - 3p + 12 = 0$$

$$4p'' - 4p' - 3p = -12.$$

Шаг 2. Однородное. Характеристическое: $4\lambda^2 - 4\lambda - 3 = 0$.

$$D = 16 + 48 = 64 \Rightarrow \lambda = (4 \pm 8)/8 = 3/2 \text{ или } -1/2.$$

$$p_h(t) = C_1 \cdot e^{(3t/2)} + C_2 \cdot e^{(-t/2)}.$$

Шаг 3. Частное. Ищем константу $p_p = A$. Тогда $-3A = -12 \Rightarrow A = 4$.

Шаг 4. Общее. $p(t) = 4 + C_1 \cdot e^{(3t/2)} + C_2 \cdot e^{(-t/2)}$.

Шаг 5. Константы.

$$-p(0) = 4 + C_1 + C_2 = 4 \Rightarrow C_1 + C_2 = 0.$$

$$-p'(t) = (3/2) \cdot C_1 \cdot e^{(3t/2)} - (1/2) \cdot C_2 \cdot e^{(-t/2)}.$$

$$-p'(0) = (3/2) \cdot C_1 - (1/2) \cdot C_2 = 2 \Rightarrow 3 \cdot C_1 - C_2 = 4.$$

$$\text{Сложение: } 4 \cdot C_1 = 4 \Rightarrow C_1 = 1, C_2 = -1.$$

ОТВЕТ. $p(t) = 4 + e^{(3t/2)} - e^{(-t/2)}$. Цена монотонно растёт (компонента $e^{(3t/2)}$ преобладает), что говорит о неустойчивости рынка в этой модели.

Если же в условии опечатки нет, и слагаемые $2p''$ действительно сокращаются, то возникает противоречие между двумя начальными условиями — задача поставлена некорректно.

Задание 41. $y' = m \cdot I \cdot p(y) \cdot y$, $p(y) = (5 + 3 \cdot e^{(-y)})/y$, $m = 0.6$, $I = 0.4$, $y(0) = 1$

Идея. Подставляем $p(y)$ и видим, что y сокращается — уравнение становится почти автономным. После замены $u = e^y$ оно превращается в линейное относительно du/dt .

Шаг 1. Подстановка.

$$y' = 0.6 \cdot 0.4 \cdot ((5 + 3 \cdot e^{(-y)})/y) \cdot y = 0.24 \cdot (5 + 3 \cdot e^{(-y)}) = 1.2 + 0.72 \cdot e^{(-y)}.$$

Шаг 2. Разделяем переменные.

$$dy / (1.2 + 0.72 \cdot e^{(-y)}) = dt.$$

Шаг 3. Замена $u = e^y$. Тогда $du = e^y \cdot dy = u \cdot dy \Rightarrow dy = du/u$, $e^{(-y)} = 1/u$.

Левая часть:

$$(du/u) / (1.2 + 0.72/u) = (du/u) / ((1.2 \cdot u + 0.72)/u) = du / (1.2 \cdot u + 0.72).$$

Шаг 4. Интегрирование.

$$\int du / (1.2 \cdot u + 0.72) = \int dt$$

$$(1/1.2) \cdot \ln|1.2 \cdot u + 0.72| = t + C$$

$$\ln|1.2 \cdot u + 0.72| = 1.2 \cdot t + C'$$

$$1.2 \cdot u + 0.72 = K \cdot e^{(1.2 \cdot t)}, \text{ где } K — \text{константа.}$$

Шаг 5. Начальное условие $y(0) = 1$. Тогда $u(0) = e^1 = e$.

$$1.2 \cdot e + 0.72 = K \Rightarrow K = 1.2 \cdot (e + 0.6).$$

Шаг 6. Возврат к y .

$$1.2 \cdot e^y + 0.72 = 1.2 \cdot (e + 0.6) \cdot e^{(1.2 \cdot t)}$$

$$e^y = (e + 0.6) \cdot e^{(1.2 \cdot t)} - 0.6$$

$$y(t) = \ln((e + 0.6) \cdot e^{(1.2 \cdot t)} - 0.6).$$

Проверка. При $t = 0$: $\ln(e + 0.6 - 0.6) = \ln e = 1 \checkmark$. Численное интегрирование исходного ОДУ (`scipy.solve_ivp`) даёт совпадение до 6 знаков после запятой при $t = 0.5, 1.0, 1.5, 2.0$.

OTBET. $y(t) = \ln((e + 0.6) \cdot e^{(1.2 \cdot t)} - 0.6).$

Заключение

В документе разобраны все 41 задачи из исходного сборника. Стил ь соответствует учебнику: для каждой задачи приведены условие, идея, пошаговый разбор и итоговый ответ; для численных и кодовых задач — рабочий Python-код, для аналитических — формулы и проверки.

Полезные сквозные приёмы, которые пригождаются почти в каждом разделе:

- **Для AR-процессов.** Дисперсия шума должна быть согласована с целевой дисперсией: $\sigma^2\epsilon = D(x) \cdot (1 - r^2)$ для AR(1).
- **Для численных методов.** Сначала локализовать корень (таблица знаков), потом выбрать метод. Дихотомия — самый надёжный и медленный, Ньютон — быстрый, но требует хорошего старта, секущие — компромисс.
- **Для финансовой математики.** Иммунизация требует двух условий: совпадения приведённой стоимости и совпадения дюрации.
- **Для геометрических вероятностей.** Свести задачу к отношению площадей/объёмов; проверить ответ Монте-Карло — занимает несколько строк и страхует от арифметической ошибки.
- **Для ОДУ-моделей рынка.** Если в условии «гладкие» сокращения, перепроверь, не делает ли это уравнение вырожденным — частая опечатка в постановках.